

Statistische Programmierung

Hochschule Kempten

Linus Konstanzer

2026-10-01

Inhaltsverzeichnis

Vorwort	5
Ziel dieses Skripts?	5
Hilfe, mein Code macht nicht das, was ich will!	5
Aufbau des Manuskripts	6
1 (Kurz)Einführung in R	8
1.1 Installieren von R und RStudio	8
1.1.1 Arbeiten mit Arbeitspfaden und R Projekten	12
1.2 Einfache R Befehle und Funktionen	15
1.2.1 Vektoren, Datensätze und Rechenoperationen	15
1.3 Arbeiten mit Datensätzen	20
1.3.1 Daten Importieren	20
1.3.2 Einfache Datenmanipulation und Visualisierung	20
1.3.3 Anpassen von Datensätzen	23
1.3.4 Faktor Variablen	26
1.4 Datenvisualisierung mit {ggplot2}	28
1.5 Übungsaufgaben	32
1.5.1 R als Taschenrechner	32
1.5.2 Listen und Vektoren	33
1.5.3 Datensätze	34
1.5.4 Grafiken	35
2 Lineare Regression	36
2.1 Der Palmer-Penguin-Datensatz	36
2.1.1 Deskriptive Statistiken	37
2.2 Die einfache lineare Regression	39
2.2.1 Schätzung der Modellparameter in R	44
2.2.2 Das Gütemaß R^2	48
2.3 Die multiple lineare Regression	49
2.3.1 Multiple lineare Regression in R	50
2.3.2 Interpretation und Vergleich der Koeffizienten	52
2.3.3 Modellannahmen	54
2.3.4 Gütemaße	77
2.4 Übungsaufgaben	79
2.4.1 Explorative Datenanalyse	79

2.4.2	Einfache lineare Regression	79
2.4.3	Kennzahlen	79
2.4.4	Multiple Lineare Regression	80
3	Weiterführende Konzepte des Statistischen Lernens	82
3.1	Training und Test Split	82
3.1.1	Training- und Testsplit in R	84
3.2	Cross-Validation	85
3.2.1	Cross-Validation in R	87
3.3	Umgang mit fehlenden Daten	88
3.3.1	Imputation	89
3.4	Merkmalsselektion mithilfe von Lasso-Regression	93
3.5	Übungsaufgaben	95
4	Regressionsbäume	98
4.1	Grundidee	98
4.1.1	Darstellung von Bäumen und Vorhersagen	98
4.1.2	Vorteile gegenüber linearen Modellen	101
4.2	Top-Down-Greedy	103
4.2.1	Rekursive und Greedy-Eigenschaft des Algorithmus	104
4.2.2	Stoppkriterien und Pruning	105
4.3	Wichtigkeit der Variablen in Baummodellen	108
4.4	Regressionsbäume in R	108
4.4.1	Schätzung der Parameter	108
4.4.2	Darstellung des Baums	111
4.4.3	Wichtigkeit der Variablen	112
4.4.4	Berechnung der Improvement-Werte	113
4.5	Erweiterungen	115
4.5.1	Erstellung eines Bootstrap-Datasets	115
4.5.2	Trainieren der Baummodelle und Aggregation	116
4.5.3	Random Forests in R	116
4.6	Übungsaufgaben	118
4.6.1	Theorieaufgaben	118
4.6.2	R Aufgaben	122
5	Klassifikation	125
5.1	Einführung	125
5.2	Evaluation eines binären Klassifikationsmodells	125
5.2.1	Confusionmatrix	125
5.2.2	ROC-Kurve	128
5.3	Die Logistische Regression	129
5.3.1	Modellgleichung	129
5.3.2	Interpretation der Koeffizienten	131

5.3.3	Statistische Signifikanz	133
5.4	Klassifikationsbäume	133
5.5	Klassifikation in R	134
5.5.1	Logistische Regression	135
5.5.2	Klassifikationsbäume	142
5.6	Übungsaufgaben	145

Vorwort

Dieses Vorlesungsskript entsteht im Zuge der Veranstaltung Statistische Programmierung mit R in Kooperation mit [Prof. Dr. Roland Jeske](#) an der Hochschule Kempten. Da dieses Manuskript noch in Arbeit ist, sind Änderungen unter Vorbehalt möglich und wahrscheinlich.

Ziel dieses Skripts?

In fast allen Übungen werden wir mit der (statistischen) Programmiersprache R arbeiten. Statistische Programmierung lebt von der Arbeit mit Daten – sei es die explorative Datenanalyse (EDA) oder das Entwickeln von statistischen Modellen für Vorhersagen. R ist besonders für EDA geeignet, da es sich durch verschiedene Zusatzpakete (Libraries) wie das Tidyverse und Tidymodels auszeichnet. Zum erfolgreichen Abschließen dieses Moduls ist es deshalb unerlässlich, dass man sich in der Vorlesung sowie der Übung mit der Programmiersprache R auseinander setzt. Sowohl die theoretischen Konzepte als auch die Anwendung sollen hierbei durch die verschiedenen Übungen vertieft werden.

Hilfe, mein Code macht nicht das, was ich will!

Debuggen, auch bekannt als die Kunst, herauszufinden, warum sich der Code nicht wie erwartet verhält, spielt beim Programmieren eine wesentliche Rolle. Auch nach jahrelanger Übung werden (gelegentlich) Fehler beim Programmieren auftreten.

Es gibt viele, mehr oder weniger effektive, Ansätze zur Lösung von programmiertechnischen Problemen im Rahmen dieses Kurses. Die meisten Probleme werden von der Form

Ich habe den Code bei mir in ein R Skript kopiert und es funktioniert nicht.

sein.

Die meisten dieser Fehler bzw. Probleme lassen sich mithilfe der folgenden drei Ansätze lösen. Wichtig an dieser Stelle ist zu erwähnen, dass die meisten Foren, Websites und Dokumentationen für R in englischer Sprache verfasst wurden, weshalb es sich lohnt, die Fehler in der englischen Form zu untersuchen.

1. Viele der häufig auftretenden Fehlermeldungen wurden wahrscheinlich schon in Foren wie [Stack Overflow](#) diskutiert. Alternativ können die Suchergebnisse auch Websites wie [DataCamp](#) oder [GeeksforGeeks](#) enthalten. Diese Websites bieten für die meisten Probleme und Fehler eine schnelle Lösung, welche in der Regel einfach exakt wie vorgeschlagen in die eigenen Codeskripte übernommen werden kann.
2. Verwendung eines großen Sprachmodells wie Claude, ChatGPT, oder Gemini. Wenn es um Fragen zur Fehlersuche oder zur Programmierung im Allgemeinen geht, sind Sprachmodelle hilfreich. Indem man eine Frage als Prompt eingibt, wird in den meisten Fällen die Ausgabe der Sprachmodelle bereits eine zufriedenstellende Antwort liefern.
3. Die interne „Hilfe“-Funktion mag zwar als altmodisch gelten, ist aber ein äußerst effektiver Ansatz zur Fehlersuche. Sie kann wertvolle Erkenntnisse liefern, wenn sich eine Funktion nicht wie erwartet verhält. Wenn sich eine Funktion nicht wie erwartet verhält, öffnet die Eingabe von `?Funktionsname` im Konsolen-Panel die entsprechende Hilfeseite. Der Abschnitt `arguments` auf der Hilfeseite erklärt die verschiedenen Eingabeparameter einer Funktion. Der Abschnitt `value` beschreibt, was für einen Rückgabewert die Funktion hat. Beispiele für die Verwendung einer Funktion können im letzten Abschnitt der Hilfe Funktion gefunden werden.

Aufbau des Manuskripts

Jedes Kapitel beginnt, je nach Komplexität des Themas, mit einer mehr oder weniger technischen Zusammenfassung und Motivation. Nach diesen Zusammenfassungen folgen Beispiele, welche die Funktionen und Konzepte ergänzen. Die Übungsaufgaben finden sich im Anschluss an diese Beispiele.

1. Im ersten Kapitel werden wir die wichtigsten R-Konzepte wiederholen, welche in der Veranstaltung regelmäßig angewendet werden.
2. Im zweiten Kapitel betrachten wir die einfache und die multiple lineare Regression. Wir gehen dabei auf die Annahmen ein, diskutieren, warum diese wichtig sind und wie man prüft, ob diese Annahmen bei einem gegebenen Datensatz erfüllt sind, oder nicht.
3. Im dritten Kapitel setzen wir uns mit essenziellen Techniken der Regression auseinander, welche direkt mit dem Entwickeln von Modellen verwandt sind. Es geht hierbei um das Aufteilen der Daten in Trainings- und Testdaten, was robustere Vorhersagen für zuvor ungesehene Ausprägungen ermöglicht. Außerdem führen wir die Imputation von fehlenden Daten sowie die effiziente Merkmalsselektion ein.
4. Das vierte Kapitel beschäftigt sich mit Regressionsbäumen. Wir lernen, wie man einen Regressionsbaum auf verschiedene Arten darstellen kann, führen für das Fitten eines Baumes den Binary-Splitting-Algorithmus ein und lernen, wie man mit Regressionsbäumen in R umgeht. Zusätzlich diskutieren wir noch verschiedene Kriterien, welche beschreiben, wie gut ein geschätztes Baummodell ist.

5. Im fünften Kapitel setzen wir uns mit Klassifikationsaufgaben auseinander. Beginnend mit der logistischen Regression und Klassifikationsbäumen diskutieren wir verschiedene Modellansätze. Wir beschäftigen uns außerdem intensiv mit der Herleitung verschiedener Metriken für Klassifikationsaufgaben und damit, wie diese zu interpretieren sind.

1 (Kurz)Einführung in R

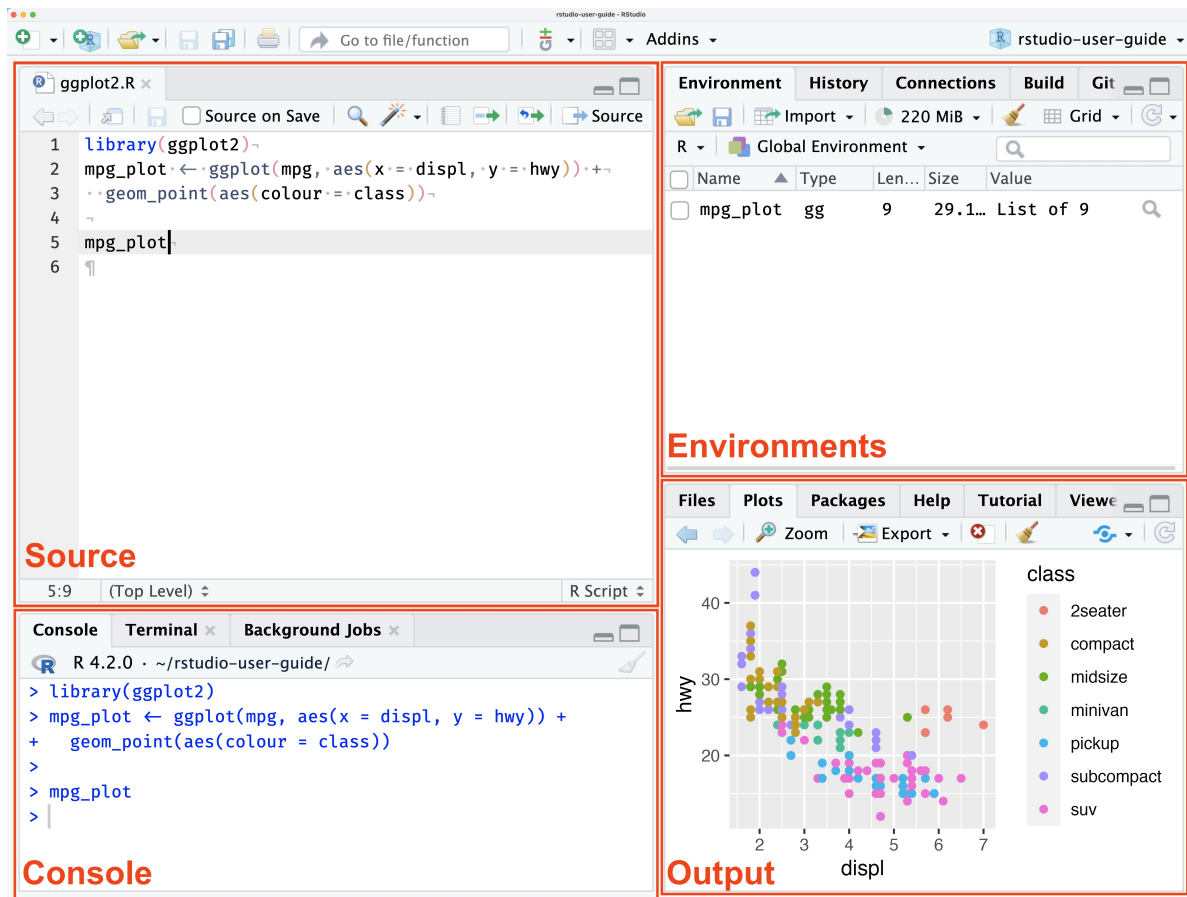
Eine vollumfängliche Wiederholung von R Konzepten welche im Grundstudium gelehrt werden würden den Rahmen dieses Manuskriptes Sprengen. Wir werden deshalb in dieser Übung den Fokus auf die Installation von R, RStudio, wichtigen Libraries und deren grundlegende Anwendung legen. Für eine Umfangreiche Einführung in R kann ich das [YARDS](#) Manuskript von Albert Rapp empfehlen.

1.1 Installieren von R und RStudio

In diesem Abschnitt wiederholen wir kurz die Installation von R und RStudio. Obwohl R (die Programmiersprache) mit einer vorinstallierten grafischen Benutzeroberfläche (GUI) installiert werden kann, werden wir stattdessen RStudio. RStudio kommt mit einigen Vorteilen gegenüber der internen R Gui, wie zum Beispiel das einsehen von Variablenwerten, Plots und Programmcode gleichzeitig. Es ist allerdings wichtig zu beachten, dass RStudio nur dann funktioniert, wenn R zuvor auch installiert wurde.

- R (für Windows) kann [hier](#) heruntergeladen werden.
- RStudio kann [hier](#) heruntergeladen werden.

Nach der erfolgreichen Installation von R und RStudio sollte sich beim Start von RStudio ein Fenster öffnen, das in etwa so aussieht wie das folgende (vgl. [RStudio User Guide](#)).

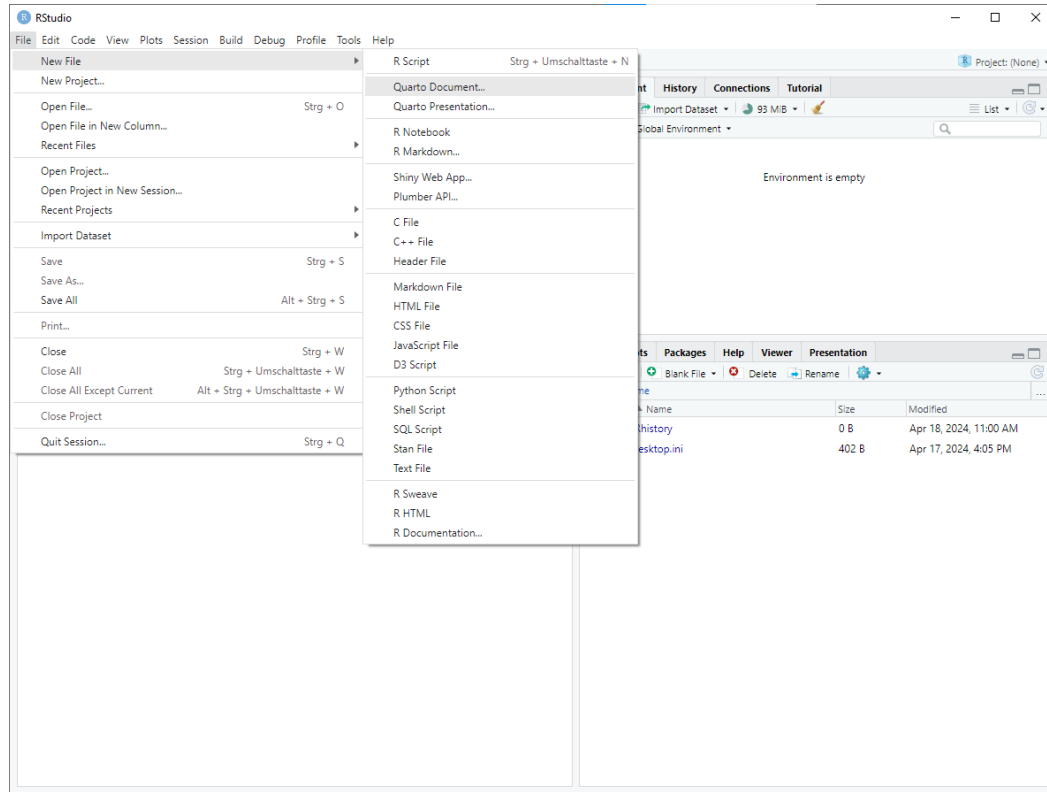


Das *Source* Panel zeigt ein `.R` File namens `ggplot2.R`. `.R` Dateien sind das standard R Format in welchen Code geschrieben und gespeichert wird. Eine Variante die mehr Ästhetik besitzt sind sogenannte Quarto Markdown Files (Endung `.qmd`) welche erlauben Text und Code zu kombinieren. Zum Beispiel wurde dieses Manuskript komplett mit Quarto geschrieben.

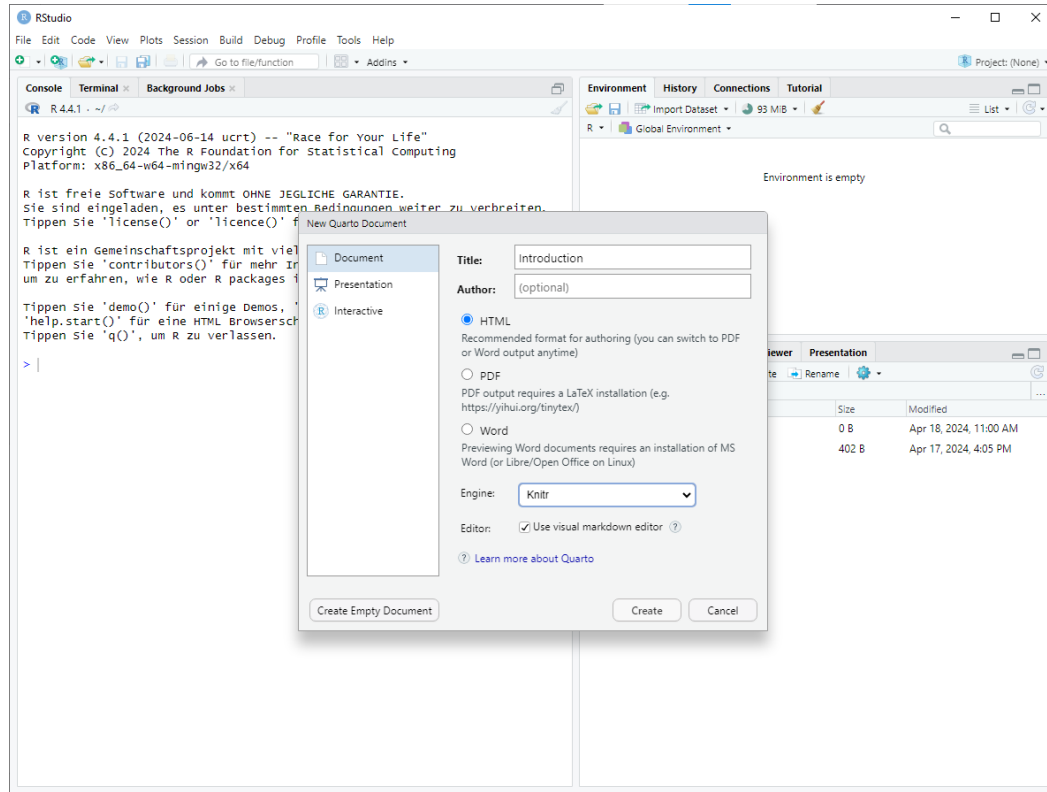
Kurzeinführung Quarto

Quarto ist in RStudio bereits vorinstalliert, weshalb das Anlegen von neuen Dokumenten sehr leicht ist.

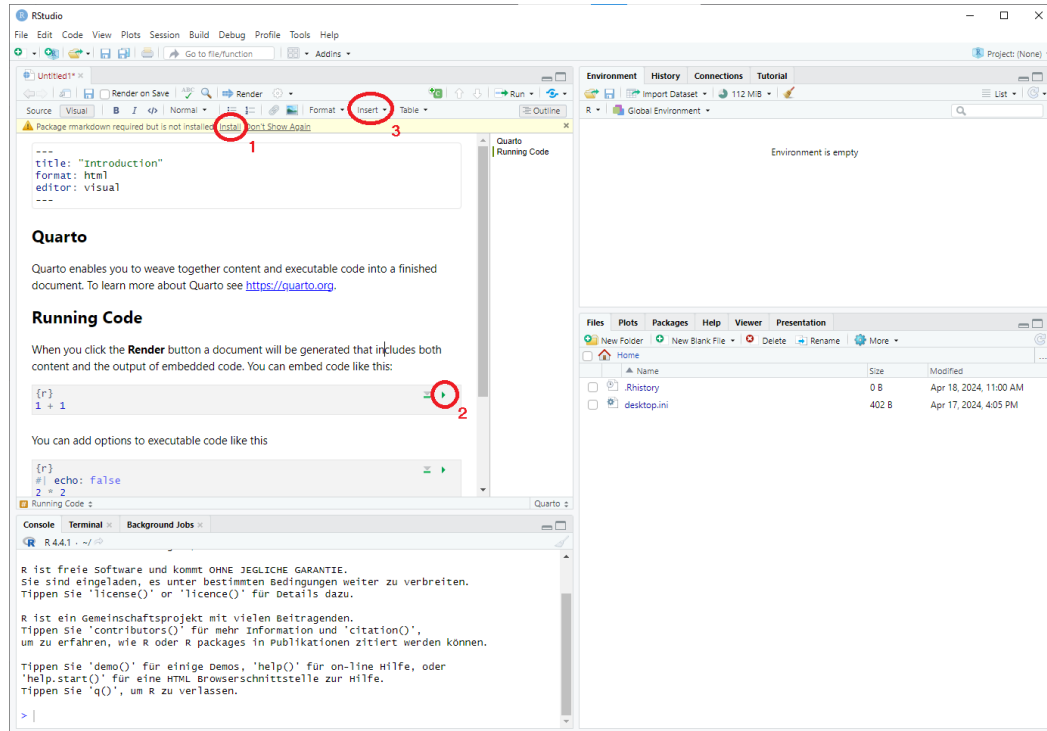
1. In RStudio, klicke auf `New File -> Quarto Document...` im `File` tab



2. Wenn das New Quarto Document Fenster sich öffnet kann man das Outputformat wählen. Das HTML output ist sehr umgänglich und kann mit jedem Browser verwendet werden. Durch das auswählen des create Knopfes öffnet sich dann ein neues Quarto Dokument.



3. Im Source Panel wird nun ein Quarto-Beispieldokument angezeigt, welches bearbeitet werden kann. Eventuell muss noch `rmarkdown` installiert werden, um eine korrekte Darstellung erzeugen zu können (vgl. Markierung 1). Das Quarto-Dokument kann dann durch Anklicken der entsprechenden Abschnitte geändert werden. R-Code-Zellen können durch Klicken auf den grünen Pfeil ausgeführt werden (vgl. Markierung 2). Um neue R-Code-Zellen zwischen Absätzen einzufügen, kann über die Registerkarte Einfügen Ausführbare Zelle -> R (vgl. Markierung 3) eine neue Code-Zelle .



Für weitere Informationen siehe z.B. [Quarto Guide](#).

Natürlich muss man sich nicht zwingend in Quarto einlesen und kann die folgenden Aufgaben auch in normalen .R Skripten ausführen.

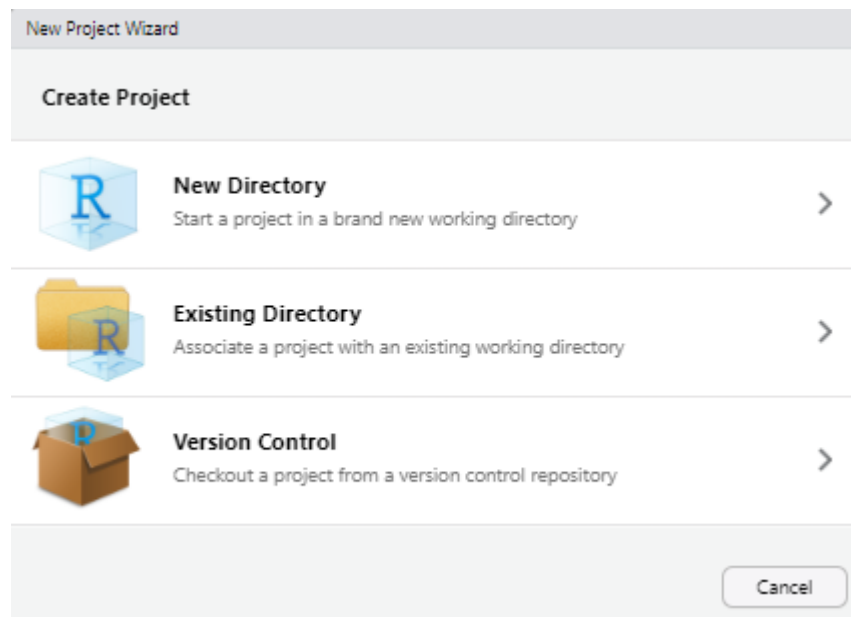
1.1.1 Arbeiten mit Arbeitspfaden und R Projekten

Nach dem Öffnen von RStudio kann durch den Befehl `getwd()` im Konsolen Panel der Pfad des aktuellen Arbeitsverzeichnis ausgegeben werden. Das Arbeitsverzeichnis ist zum Beispiel dann wichtig, wenn wir neue Dateien anlegen, oder Datensätze einlesen wollen. Wenn der Rückgabewert des Befehls `getwd()` zum Beispiel `"C:/Users/konstlinu/Documents/Vorlesungen/Data_Mining/"` lautet, kann R auf jede Datei im Verzeichnis `Data_Mining` zugreifen. Eine Möglichkeit, das Arbeitsverzeichnis zu ändern, ist die Verwendung des Befehls `setwd()`. Hierbei ist wichtig, entweder den absoluten neuen Pfad, oder den relativen neuen Pfad anzugeben. Falls das Ziel ist, in das `/Documents` Verzeichnis zu wechseln, so kann man durch die Angabe des relativen Pfades `setwd('../..../')` das Verzeichnis `/Documents` auswählen, sofern das aktuelle Verzeichnis `"C:/Users/konstlinu/Documents/Vorlesungen/Statistische_Programmierung/"` ist. Die `../` Notation wird verwendet um das übergeordnete Verzeichnis auszuwählen. Alternativ kann man auch einfach den absoluten Pfad `"C:/Users/konstlinu/Documents"` in den `setwd()`

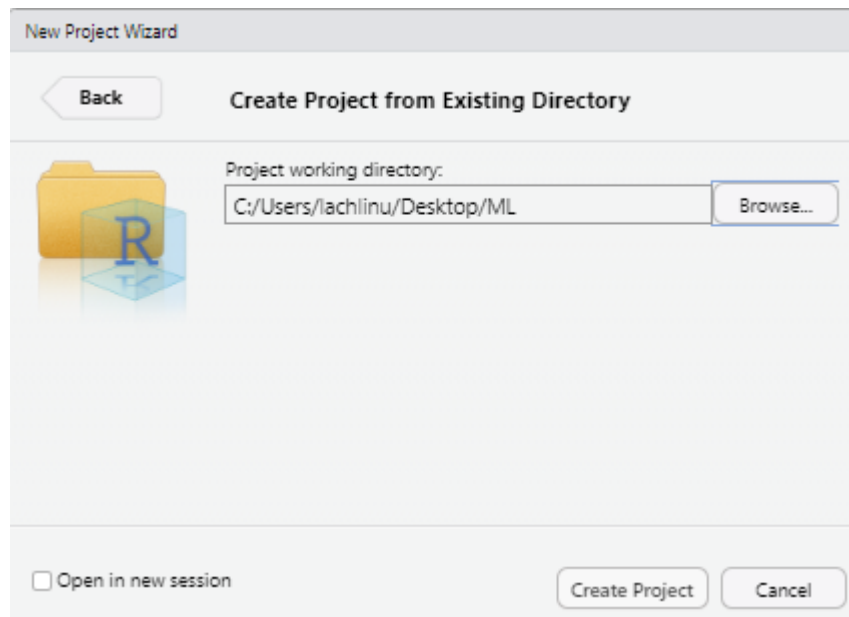
Befehl einfügen um das Verzeichnis entsprechend zu wechseln. Das manuelle Ändern des Verzeichnisses in jedem .R Skript kann schnell mühsam werden, daher ist die Einrichtung eines Projekts eine praktischere Alternative. RStudio-Projekte ermöglichen die Erstellung eines individuellen Arbeitsverzeichnisses für mehrere Kontexte. Zum Beispiel können wir R nicht nur zum Lösen von Aufgaben für statistische Programmierung verwenden, sondern auch für Abschlussarbeiten. Wenn man also zwei verschiedene Projekte einrichtet, kann man die Arbeitsverzeichnisse und Arbeitsbereiche für jedes Projekt individuell organisieren.

Das folgende Beispiel illustriert wie man ein Projekt einrichtet und verwendet:

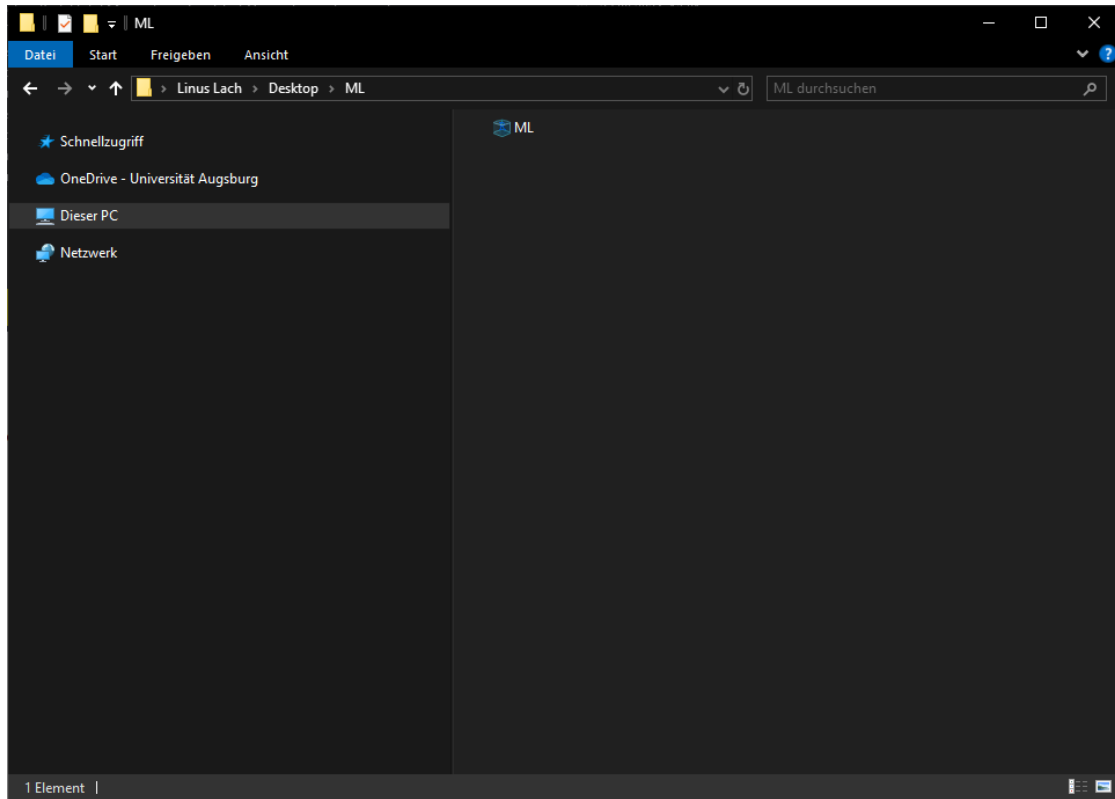
1. Über den **File** Tab, wähle **New Project**....



2. Durch das wählen von **Existing Directory** kann man durch den Explorer ein Verzeichnis wählen welches bereits existiert und dann durch den **Create Project** button ein neues Projekt in diesem Verzeichnis anlegen.



3. Das neue Projekt kann dann einfach angeklickt werden, wodurch sich ein neues RStudio Fenster öffnen sollte.



Sobald sich über das anklicken des Projekts ein neues RStudio Fenster geöffnet hat, wird durch das Ausführen des `getwd()`-Befehls der Arbeitspfad des Projekts ausgegeben. Nun kann auch jede Datei in diesem Verzeichnis direkt verwendet werden.

1.2 Einfache R Befehle und Funktionen

Nach der erfolgreichen Installation von R und RStudio widmen wir uns nun ein paar grundlegenden Programmierkonzepten in R.

1.2.1 Vektoren, Datensätze und Rechenoperationen

1.2.1.1 Vektoren

Variablen wie Vektoren, oder Arrays werden in R mit dem `<-` erstellt. Falls zum Beispiel das Ziel ist, einen Vektor zu erstellen, welcher die Werte 1, 2, 3, 4, 5 enthält, gibt es viele Verschiedene Möglichkeiten dies zu erreichen.

```
x1 <- c(1,2,3,4,5)
x2 <- 1:5
x3 <- seq(1,5)
```

Jeder der obigen Vektoren enthält die Zahlen 1-5. `x1` wurde durch die `c()` Funktion erstellt, welche einen Vektor entsprechend der übergebenen Argumente erzeugt. `x2` wurde mithilfe des `:` Operators erstellt, welcher einer ganzzahlige Sequenz der Zahlen beginnend mit der Zahl links des Operators bis zur Zahl rechts des Operators erzeugt. Die Erzeugung des Vektors `x3` basiert auf der selben Idee wie die Erstellung des Vektors `x2`, allerdings wurde hier explizit die Funktion `seq` (Sequence) verwendet um die Zahlenfolge zu generieren. Die `seq` Funktion erlaubt im Gegensatz zur Verwendung des `:` Operators einige weitere Argumente wie zum Beispiel das Argument `by`, oder `length.out`.

Der folgende Befehl erzeugt beispielsweise eine Sequenz von 1 bis 5 mit Schrittgröße 0.5.

```
seq(from = 1,to = 5,by = 0.5)
```

```
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

Durch das Spezifizieren von `length.out` kann man die Länge der Sequenz steuern. So wird im folgenden Beispiel eine Sequenz von Zahlen zwischen 1 und 5 erzeugt mit der Länge 20.

Die Zuweisung von Werten in Funktionen erfolgt im obigen Beispiel mit dem Symbol `=`. Die Zuweisung neuer Variablen außerhalb von Funktionen kann auch mit dem `=` Symbol erfolgen, aber es wird selten verwendet und außer in einigen pathologischen Fällen gibt es keinen Unterschied. Die meisten R-Benutzer bevorzugen jedoch die Zuweisung von Umgebungsvariablen mit `<-`, was in Funktionsaufrufen **nicht** funktioniert.

```
seq(1,5,length.out = 20)
```

```
[1] 1.000000 1.210526 1.421053 1.631579 1.842105 2.052632 2.263158 2.473684
[9] 2.684211 2.894737 3.105263 3.315789 3.526316 3.736842 3.947368 4.157895
[17] 4.368421 4.578947 4.789474 5.000000
```

Tip

Die Argumente einer Funktion müssen nicht immer übergeben werden. Im obigen Beispiel wurden die Benennungen `"from="` und `"to="` weggelassen. Das Weglassen der Argumentnamen beim Aufruf der Funktion ist lediglich der Faulheit der programmierenden Person geschuldet. In den meisten Fällen lohnt es sich Zwecks Lesbarkeit diese zu be-

nennen. Wichtig ist allerdings, beim Weglassen der Argumentnamen die vorgegebene Reihenfolge der Argumente einzuhalten. Falls beispielsweise der Befehl `seq(1,5,0.5)` die Folge `1.0, 1.5, ..., 5.0` ausgibt, dann ist der Rückgabewert des Befehls `seq(5,1,0.5)` nicht der gleiche! R geht davon aus, dass die Argumente in der Reihenfolge übergeben werden, in welcher sie auch in der Dokumentation festgelegt wurden. Im Falle der `seq()`-Funktion ist das `from`, `to`, `by`, `length.out`,

Der Abstand zwischen den einzelnen Folgenglieder der Folge ist hierbei gegeben durch

$$\frac{5 - 1}{20 - 1} = \frac{4}{19}.$$

Im Zähler ist steht die Länge des Intervalls welche durch **obere Grenze - untere Grenze** berechnet wird und im Nenner **Anzahl der Folgeglieder - 1**.

Das Zugreifen auf Listenelemente wird durch eckige Klammern ermöglicht. Möchte man zum Beispiel auf das zweite Element des Vektors `x2` zugreifen, so wendet man die folgende Notation an:

```
x2[2]
```

```
[1] 2
```

Innerhalb der eckigen Klammer steht der Index, bzw. die Indizes welche zurückgegeben werden sollen. So kann man auch zum Beispiel die ersten zwei Einträge des Vektors `x2` durch folgende Notation auswählen:

```
x2[1:2]
```

```
[1] 1 2
```

1.2.1.2 Erstellen von Datensätzen

Obwohl die meisten Datensätze, welche wir in der Übung verwendet bereits erstellt wurden, ist es sehr sinnvoll sich mit der Erstellung von eigenen Datensätzen auseinander zu setzen. Besonders wenn man auf Basis bereits existierender Datensätze Teildatensätze erstellen will, ist diese Kenntnis hilfreich.

Ein Grundlegender Befehl zum erzeugen von Datensätzen ist die `data.frame` Funktion.

```
data<- data.frame(
  x = seq(1,10,length.out = 18),
  y = letters[1:18],
  z = unlist(
    rep(
      strsplit("The quick brown fox jumps over the lazy dog",
        split = " "),
      2)
  )
)
```

Das Datensatz `data` enthält drei Spalten: `x`, `y`, `z` welche mit verschiedenen Werten erzeugt wurden. Die Spalte `x` wurde mithilfe der `seq` Funktion mit 18 Werten befüllt welche zwischen 1 und 10 liegen. Die Spalte `y` wurde mithilfe des vordefinierten `letters` Vektor befüllt. Dieser enthält die Buchstaben `a-z`. Hierbei wurden ebenso die 18 Einträge verwendet. In der letzten Spalte `z` wurden mehrere Funktionen verschachtelt auf einmal verwendet. Die innerste Funktion `strsplit` teilt einen gegebenen String (`<chr>`) Vektor in einzelne Komponenten auf. Die Teilung des langen Strings folgt durch das Argument `split`. Hier wurde als Split Argument das Leerzeichen `" "` übergeben. Dadurch wird der Satz `"The quick brown fox jumps over the lazy dog"` an jedem Leerzeichen getrennt und eine Liste der einzelnen Wörter zurückgegeben. Die `rep` (Repeat) Funktion wiederholt das übergebene Argument und wiederholt dieses gemäß der übergebenen Zahl. Da der Rückgabewert der `rep` Funktion in diesem Fall eine verschachtelte Liste ist (eine Liste mit zwei Elementen, wobei jedes dieser beiden Elemente wieder selbst eine Liste ist), wir die Verschachtlung über die `unlist` Funktion aufgehoben.

Der Datensatz hat somit folgende Struktur:

```
data
```

	x	y	z
1	1.000000	a	The
2	1.529412	b	quick
3	2.058824	c	brown
4	2.588235	d	fox
5	3.117647	e	jumps
6	3.647059	f	over
7	4.176471	g	the
8	4.705882	h	lazy
9	5.235294	i	dog
10	5.764706	j	The
11	6.294118	k	quick
12	6.823529	l	brown

```
13 7.352941 m fox
14 7.882353 n jumps
15 8.411765 o over
16 8.941176 p the
17 9.470588 q lazy
18 10.000000 r dog
```

Die Variablen eines Datensatzes könne über verschiedene Methoden ausgewählt und bearbeitet werden. Im nächsten Abschnitt werden hierfür einige dieser Methoden vorgestellt.

1.2.1.3 Rechenoperationen in R

R wurde ursprünglich als statistische Programmiersprache eingeführt, weshalb bereits viele Funktionen zum berechnen verschiedener Werte vordefiniert sind.

Arithmetische Rechenoperation funktionieren für Zahlenwerte, Zahlenvektoren und auch gemischten Vektoren.

```
5+2
```

```
[1] 7
```

```
seq(1:5)*2
```

```
[1] 2 4 6 8 10
```

```
seq(1:5)^2
```

```
[1] 1 4 9 16 25
```

```
cos(seq(0,pi,length.out=10))
```

```
[1] 1.0000000 0.9396926 0.7660444 0.5000000 0.1736482 -0.1736482
[7] -0.5000000 -0.7660444 -0.9396926 -1.0000000
```

Obige Beispiele enthalten nur eine kleine Selektion an Methoden welche verwendet werden können, um verschiedene Rechenoperationen durchzuführen. R Funktioniert gewissermaßen teilweise wie ein Taschenrechner, weshalb dieser Abschnitt auch nicht in der Tiefe ausgeführt werden muss.

1.3 Arbeiten mit Datensätzen

Im Zusammenhang mit statistischem Programmieren bilden Daten die Grundlage. Egal ob das Ziel ist das Wetter, künftige Aktienkurse oder die Grundmiete einer potenziellen Mietwohnung vorherzusagen - ohne hochwertige Daten versagen selbst die fortschrittlichsten Modelle. Die Realität sieht jedoch so aus, dass Daten, die wir aus dem Internet, von Servern oder aus Excel Tabellen beziehen, oft alles andere als makellos sind. So können fehlende Werte beispielsweise als `NA` (Not Available), `NaN` (Not a Number), `NULL` oder einfach als leere Zeichenkette „" kodiert sein. Aus diesem Grund ist es wichtig, dass die grundlegenden Datenmanipulationen in R nochmals diskutiert werden.

1.3.1 Daten Importieren

Viele Datensätze sind zwar bereits in R integriert, bzw. können über Libraries eingelesen werden, allerdings sind einige der Datensätze welche wir in der Übung verwenden nur durch externe Quellen verfügbar. Die meisten Daten werden im `csv` oder `txt` Format zur Verfügung gestellt. Es ist deshalb wichtig zu verstehen wie man diese Datensätze einliest bevor man tatsächlich mit diesen arbeitet.

Für die folgenden Übungen werden wir das Ramen Dataset untersuchen, welches entweder von der Data Science Plattform [Kaggle](#) oder dem untenstehenden Button heruntergeladen werden kann.

Der Ramen Datensatz enthält Informationen über verschiedene Instant Ramen Gerichte, welche besonders beliebt unter Studierenden während der Prüfungszeit sind.

Die Daten können dann zum Beispiel in einem Verzeichnis `Daten` gespeichert werden, welches im gleichen Verzeichnis liegt wie die Projektdatei. Um die Daten einzulesen verwendet man in diesem Fall die `read.csv` Funktion verwenden. Im folgenden Code Snippet wird der Ramen Datensatz aus dem Verzeichnis `/data` eingelesen und unter dem Namen `data_ramen` gespeichert.

```
data_ramen <- read.csv(file = "data/ramen-ratings.csv")
```

1.3.2 Einfache Datenmanipulation und Visualisierung

Für die meisten Übungen werden wir das `{tidyverse}` verwenden, welches eine Sammlung von Libraries enthält die das Arbeiten mit Daten stark vereinfachen. Um die Sammlung von Libraries zu installieren, kann der `install.packages()` Befehl verwendet werden:

```
install.packages("tidyverse")
```

Nach der Installation kann die Library dann mit dem `library` Befehl zur aktuellen R Session hinzugefügt werden.

```
library("tidyverse")
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Das `{tidyverse}` enthält eine Vielzahl wichtiger Libraries, von welchen wir vor allem die folgenden drei verwenden werden:

- `{ggplot2}` für das Erstellen von Grafiken,
- `{dplyr}` für Datenmanipulation,
- `{tibble}` für das effiziente Speichern von Daten.

Eine sehr gute Einführung in diese Library kann unter folgendem [Link](#) eingesehen werden. Der Entwickler hinter der Library hat ebenso ein kostenloses Buch namens [R for Data Science](#) geschrieben, welches eine sehr umfangreiche Einführung in die verschiedenen Libraries des `{tidyverse}` gibt.

Nach dem erfolgreichen Installieren und hinzufügen der Library können wir direkt die erste essentielle Funktion nutzen. Die `glimpse()` Funktion gibt einen kompakten Überblick über den Inhalt und die beinhaltenden Datentypen des unterliegenden Datensatzes.

```
glimpse(data_ramen,width =50)
```

```
Rows: 2,580
Columns: 7
$ Review.. <int> 2580, 2579, 2578, 2577, 2576, 2~
$ Brand    <chr> "New Touch", "Just Way", "Nissi~
$ Variety  <chr> "T's Restaurant Tantanmen ", "N~
$ Style    <chr> "Cup", "Pack", "Cup", "Pack", "~
$ Country  <chr> "Japan", "Taiwan", "USA", "Taiw~
$ Stars    <chr> "3.75", "1", "2.25", "2.75", "3~
$ Top.Ten  <chr> "", "", "", "", "", "", "", "", ~
```

Rows: 2,580 bedeutet, dass der Datensatz 2.580 Einträge hat, und **Columns:** 7 bedeutet, dass der Datensatz jeweils 7 Variablen enthält. Die erste Spalte enthält die Variablennamen, ihre Datentypen und einige Anfangswerte, wodurch der Datensatz eine klare Struktur erhält. Wir können bereits sehen, dass außer einer Variablen (**Review**) jede andere Variable vom Typ **chr** ist, was für *character* oder *string* steht.

1.3.2.1 Anpassen der Variablen

Beim Aufrufen der `glimpse()` Funktion ist direkt aufgefallen, dass die Variablennamen teilweise inkonsistent sind. Die erste Spalte hat zum Beispiel den Namen `"Review.."`. Um eine konsistente und einfache Namensgebung zu verwenden, können wir die `janitor::clean_names` Funktion verwenden.

Tip

Die obige Notation wird verwendet, wenn man zusätzlich zum Funktionsname auch noch die Library der Funktion benennen will. Besonders bei der Vermeidung von sogenannten 'namespace conflicts', bei welchen zwei Funktionen aus verschiedenen Libraries die gleichen Namen besitzen.

Auch hier gilt: Eventuell muss die `{janitor}` Library zuerst installiert werden, bevor die Funktion verwendet werden kann. Die Installation kann analog wie im Beispiel zur `{tidyverse}` Library durchgeführt werden.

Wenn die Library dann erfolgreich installiert wurde, können wir die Daten mit einer angepassten Version überschreiben:

```
data_ramen <- janitor::clean_names(dat = data_ramen)
```

Nach dem Abändern der Daten und einem erneuten Aufrufen der `glimpse()` Funktion kann die Änderung der Variablennamen eingesehen werden:

```
glimpse(data_ramen)
```

```
Rows: 2,580
Columns: 7
$ review <int> 2580, 2579, 2578, 2577, 2576, 2575, 2574, 2573, 2572, 2571, 25~
$ brand <chr> "New Touch", "Just Way", "Nissin", "Wei Lih", "Ching's Secret"~
$ variety <chr> "T's Restaurant Tantanmen ", "Noodles Spicy Hot Sesame Spicy H~
$ style <chr> "Cup", "Pack", "Cup", "Pack", "Pack", "Pack", "Cup", "Tray", "~
$ country <chr> "Japan", "Taiwan", "USA", "Taiwan", "India", "South Korea", "J~
$ stars <chr> "3.75", "1", "2.25", "2.75", "3.75", "4.75", "4", "3.75", "0.2~
$ top_ten <chr> "", "", "", "", "", "", "", "", "", "", "", "", "", "", "", ""~
```

1.3.3 Anpassen von Datensätzen

1.3.3.1 Die mutateFunktion

Um Anpassungen an den Daten vorzunehmen, bietet sich die `dplyr::mutate` Funktion perfekt an.

Die `mutate` Funktion kann entweder neue Spalten generieren, oder vorhandene Spalten modifizieren. Im folgenden Beispiel wird die `mutate` Funktion dafür verwendet die Variable `stars` vom Datentyp `<chr>` in `<dbl>` umzuwandeln und die Variable `top_ten` zu entfernen.

```
1 data_ramen <- data_ramen %>%  
2   mutate(  
3     stars = as.double(stars),  
4     top_ten = NULL  
5   )
```

```
Warning: There was 1 warning in `mutate()`.  
i In argument: `stars = as.double(stars)`.  
Caused by warning:  
! NAs introduced by coercion
```

1. In der ersten Zeile der Codezelle wird angegeben, dass der Datensatz `data_ramen` durch eine veränderte Version überschrieben werden soll.
2. Der ursprüngliche Datensatz `data_ramen` wird in der zweiten Zeile an die `mutate`Funktion übergeben.
3. In der dritten Zeile wird als erstes Argument die Variable `stars` durch die Ausgabe der Funktion `as.double` mit Argument `stars` überschrieben. Die Funktion `as.double` ist eine Funktion der `{base}` library.
4. In der vierten Zeile wird als weiteres Argument (separiert durch ein `,` aus der vorherigen Zeile) die den Variable `top_ten` entfernt, indem die Variable mit `NULL` überschrieben wird.

Das ausführen der vorherigen Code Zelle wirft folgende Warnung:

```
Warning message: There was 1 warning in mutate(). i In argument: stars =  
as.double(stars). Caused by warning: ! NAs introduced by coercion
```

Dies liegt daran, dass in der Variable `stars` fehlende Werte vorhanden sind. Durch das Anwenden der `as.double` Funktion wurden die Fehlenden werte in `NA` Werte umgewandelt, was aber erst mal kein Problem darstellt.

1.3.3.2 Der Pipe Operator %>%

In der Code Zelle

```
1 data_rahmen <- data_rahmen %>%  
2   mutate(...)
```

wurde in der ersten Zeile der sogenannte Pipe Operator %>% verwendet. Der Pipe Operator ist ein mächtiges Werkzeug aus der {magittr} Library welche erlaubt Verkettungen von Funktionen durchzuführen. Die Wortwörtliche übersetzung “Rohr” steht hierbei sinnbildlich für den Vorgang dass auf der einen Seite (links) z.B. Daten übergeben werden und auf der anderen Seite (rechts) nach den durchgeführten Transformationen ausgegeben werden. Die Stärke des Operators liegt hierbei in der Möglichkeit viele Operationen sequentiell durchzuführen und dabei die Lesbarkeit und Übersichtlichkeit zu bewahren.

So kann man zum Beispiel nach der initialen Transformation durch die `mutate` Funktion die `na.omit` Funktion auf die resultierenden Daten angewendet werden, um alle NA, bzw. NaN Werte zu entfernen.

```
data_rahmen <- data_rahmen %>%  
  mutate(  
    stars = as.double(stars),  
    top_ten = NULL  
  ) %>%  
  na.omit()
```

Durch die Anwendung des Pipe Operators muss auch der Datensatz im Obigen Beispiel nicht als Argument der Funktion übergeben werden. Ohne den Pipe Operator könnte man das obige Beispiel auch wie folgt implementieren:

```
data_rahmen <- mutate(na.omit(data_rahmen),  
                      stars = as.double(stars),  
                      top_ten = NULL)
```

1.3.3.3 Filtern, Gruppieren und Zusammenfassen von Datasets

Im vorherigen Abschnitt haben wir uns mit grundlegenden Funktionen für das Anpassen der Daten beschäftigt. Ein weiterer wichtiger Aspekt beim Umgang mit Daten ist die sogenannte Explorative Datenanalyse (EDA). Hierbei werden die unterliegenden Daten mithilfe von deskriptiven und induktiven Methoden untersucht. Wichtige Funktionen in diesem Kontext sind die `filter`, `group_by` und `summarise` Funktionen aus dem {dplyr} Package.

Angenommen wir wollen für das `data_ramen` Dataset herausfinden, wie viele Marken (brands) mehr als 30 Sorten (varieties) an Instant Ramen Nudeln anbieten. Intuitiv macht es Sinn die Einträge des Datensatzes zu Gruppieren, wobei jede Marke einer Gruppe entspricht. Nach der Bildung von Gruppen erzeugt durch die Marken, kann dann für jede Gruppe die Anzahl der Einträge innerhalb der Gruppen berechnet werden. Abschließend filtert man die resultierenden Ergebnisse nach dem Kriterium, dass mindestens 30 Einträge in einer Gruppe enthalten sein müssen. Diese beschriebene Heuristik lässt sich mithilfe des folgenden Skripts umsetzen:

```
1 data_ramen %>%
2   group_by(brand) %>%
3   summarise(num_var = n()) %>%
4   filter(num_var > 30)
```

```
# A tibble: 12 x 2
  brand      num_var
  <chr>      <int>
1 Indomie      53
2 Lucky Me!    34
3 Mama        71
4 Maruchan     76
5 Myojo       63
6 Nissin      381
7 Nongshim     98
8 Ottogi      45
9 Paldo       66
10 Samyang Foods 51
11 Vifon       33
12 Vina Acecook 34
```

Fassen wir die Schritte im obigen Skript nochmal zusammen:

1. In der ersten Zeile wird der Datensatz `data_ramen` durch den Pipe Operator in die folgenden Funktionen übergeben.
2. In der zweiten Zeile wird der `group_by` Befehl verwendet um die Einträge des Datensatzes nach der Variable `brand` zu Gruppieren.

Tip

Durch das alleinige Aufrufen der `group_by` Funktion mit der entsprechenden Gruppierungsvariable als Argument, wird im Output bereits angezeigt wie viele verschiedene Gruppen erstellt wurden.

```
data_ ramen %>%
  group_by(brand) %>%
  glimpse()
```

Rows: 2,577

Columns: 6

Groups: brand [355]

\$ review <int> 2580, 2579, 2578, 2577, 2576, 2575, 2574, 2573, 2572, 2571, 25~

\$ brand <chr> "New Touch", "Just Way", "Nissin", "Wei Lih", "Ching's Secret"~

\$ variety <chr> "T's Restaurant Tantanmen ", "Noodles Spicy Hot Sesame Spicy H~

\$ style <chr> "Cup", "Pack", "Cup", "Pack", "Pack", "Pack", "Cup", "Tray", "~

\$ country <chr> "Japan", "Taiwan", "USA", "Taiwan", "India", "South Korea", "J~

\$ stars <dbl> 3.75, 1.00, 2.25, 2.75, 3.75, 4.75, 4.00, 3.75, 0.25, 2.50, 5.~

so wurden im obigen Beispiel 355 verschiedene Gruppen gebildet, woraus wir schließen können, dass im Datensatz 355 verschiedene Marken enthalten sind (unter der Annahme, dass keine Duplikate enthalten sind).

3. Der `summarise` Befehl wird dann verwendet, um verschiedene Deskriptive Statistiken auf den Gruppen anzuwenden. In diesem Fall wird ähnlich wie bei der Verwendung der `mutate` Funktion eine neue Spalte `num_vars` erstellt welche in jeder Zeile den Output der Funktion `n` enthält. Die `dplyr::n` Funktion hat als Rückgabewert die Gruppengröße.

1.3.4 Faktor Variablen

Ein wichtiger Datentyp, der sowohl ordinale (Daten mit einem gewissen Ordnungsbegriff) als auch nominale Daten verarbeiten kann, sind so genannte Faktor Variablen (Datentyp `<fct>`).

Betrachten wir hierfür den folgenden Synthetischen Datensatz mit sieben Personen mit entsprechenden Altersgruppen und Augenfarben.

```
data_example <- tibble(
  names = c("Alice", "Bob", "Charlie", "Diana", "Eve", "Frank", "Grace"),
  age_groups = c("18-25", "<18", "26-35", "36-45", "18-25", "60+", "26-35"),
  eye_color = c("Blue", "Brown", "Green", "Hazel", "Brown", "Blue", "Green")
)
```

```
data_example
```

```
# A tibble: 7 x 3
```

```
  names    age_groups eye_color
  <chr>    <chr>      <chr>
```

1	Alice	18-25	Blue
2	Bob	<18	Brown
3	Charlie	26-35	Green
4	Diana	36-45	Hazel
5	Eve	18-25	Brown
6	Frank	60+	Blue
7	Grace	26-35	Green

Note

Im obigen Code Snippet wurde die `tibble::tibble` Funktion verwendet. Tibbles werden ähnlich wie `data.frame` Objekte zum speichern von Datensätzen verwendet. Tibbles haben allerdings zum Beispiel den Vorteil, dass sie bei der Ausgabe in der Konsole übersichtlicher dargestellt werden.

Da die Variable `age_group` nur eine Altersspanne angibt, ist es nicht sinnvoll, sie als Zahl und nicht als Ordinalvariable zu kodieren. Mit der Funktion `mutate` können wir deshalb die Altersgruppen als Ordinalvariablen (ordered factors) kodiert werden. Dazu wird die Variable `age_groups` in einen Faktor mit Stufen und Label gesetzt. Die Stufen geben die Reihenfolge der Werte an, und die Label können zur Umbenennung dieser Kategorien verwendet werden.

```

1 data_example <- data_example %>%
2   mutate(
3     age_groups = factor(
4       age_groups,
5       levels = c("<18", "18-25", "26-35", "36-45", "60+"),
6       ordered = TRUE,
7       labels = c("child","adult","adult","adult","senior")
8     )
9   )

```

1. Ähnlich wie im vorherigen Beispiel sollten wir angeben, dass wir den Datensatz `data_example` mit einer veränderten Version überschreiben.
2. In der zweiten Zeile wird dann die `mutate` Funktion aufgerufen um in der dritten Zeile die Variable `age_groups` zu überschreiben.
3. Durch das Setzen von `age_groups = factor(age_groups, ...)` wird die Variable `age_groups` in einen (bisher ungeordneten) Faktor umgewandelt, der das setzen von Stufen (`levels`) und Labels ermöglicht.
4. So wird durch das Setzen von `levels = c("<18", "18-25", ...)` eine Ordnung auf die Variable auferlegt.

5. `ordered=TRUE` gibt an, dass die Altersgruppen nach den im `levels`Vektor angegebenen Stufen aufsteigend sortiert sind.
6. Zu guter Letzt gibt `labels = c("child", "adult", ...)` die Bezeichnungen an, welche die numerischen Altersgruppen ersetzen. Zum Beispiel wird < 18 als "child" bezeichnet, die Bereiche 18-25, 26-35 und 36-45 als "adult", und > 60 als "senior".

Analog kann auch die Variable `eye_color` in eine nominale Faktorvariable umgewandelt werden:

```
data_example <- data_example %>%  
  mutate(  
    eye_color = factor(eye_color)  
  )
```

1.4 Datenvisualisierung mit {ggplot2}

Ein weiterer wichtiger Aspekt der statistischen Programmierung ist die grafische Darstellung von Datensätzen. Die Beschreibung eines Algorithmus mit mathematischer Notation oder die Untersuchung eines Datensatzes mit Hilfe von deskriptiven und induktiven Statistiken allein kann eine Herausforderung für die Vermittlung von Botschaften darstellen. Obwohl R einige Basisfunktionen zur Erstellung von Grafiken bietet, wird in diesem Kurs hauptsächlich die Bibliothek `{ggplot2}` verwendet. Eine umfassende Einführung in `{ggplot2}` findet sich in Hadley Wickhams Buch [Elegant Graphics for Data Analysis](#).

Für das folgende Beispiel wird der Datensatz `data_ramen_ratings` verwendet:

```
data_ramen_ratings <- data_ramen %>%  
  group_by(stars) %>%  
  summarise(count=n())  
  
data_ramen_ratings %>% head()
```

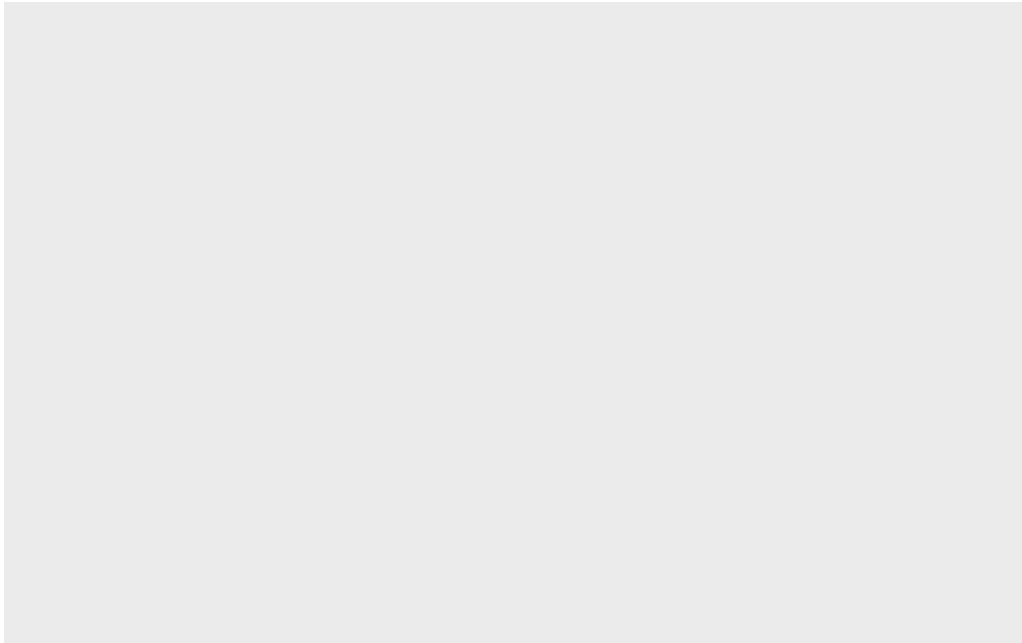
```
# A tibble: 6 x 2  
  stars count  
  <dbl> <int>  
1 0      26  
2 0.1     1  
3 0.25    11  
4 0.5     14  
5 0.75     1  
6 0.9     1
```

Hierbei wird ein neuer Datensatz erzeugt, welcher zwei Spalten enthält. Die erste Spalte enthält die verschiedenen Ausprägungen der Variable **stars** und die zweite Spalte **count** enthält die Anzahl der Ramen, welche die entsprechende Bewertung in **stars** haben. Der **head** Befehl gibt hierbei die ersten 6 Einträge des neu erstellten Datensatzes aus.

Eine mit **{ggplot2}** erstellte Grafik besteht aus den folgenden drei Grundkomponenten:

1. Die Daten selbst.

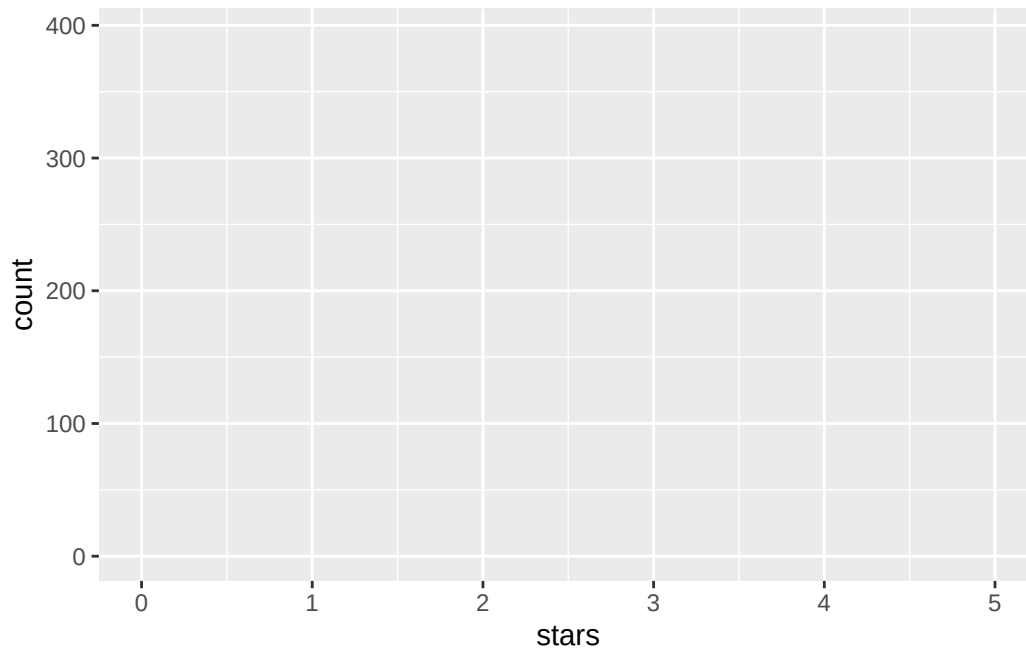
```
ggplot(data = data_ramen_ratings)
```



Beachte, dass das Diagramm so noch keine Achsen, Ticks und Variablen anzeigt.

2. Eine Reihe von *Ästhetik-Zuordnungen* (*aesthetics mappings*), die beschreiben, wie Variablen in den Daten auf visuelle Eigenschaften abgebildet werden.

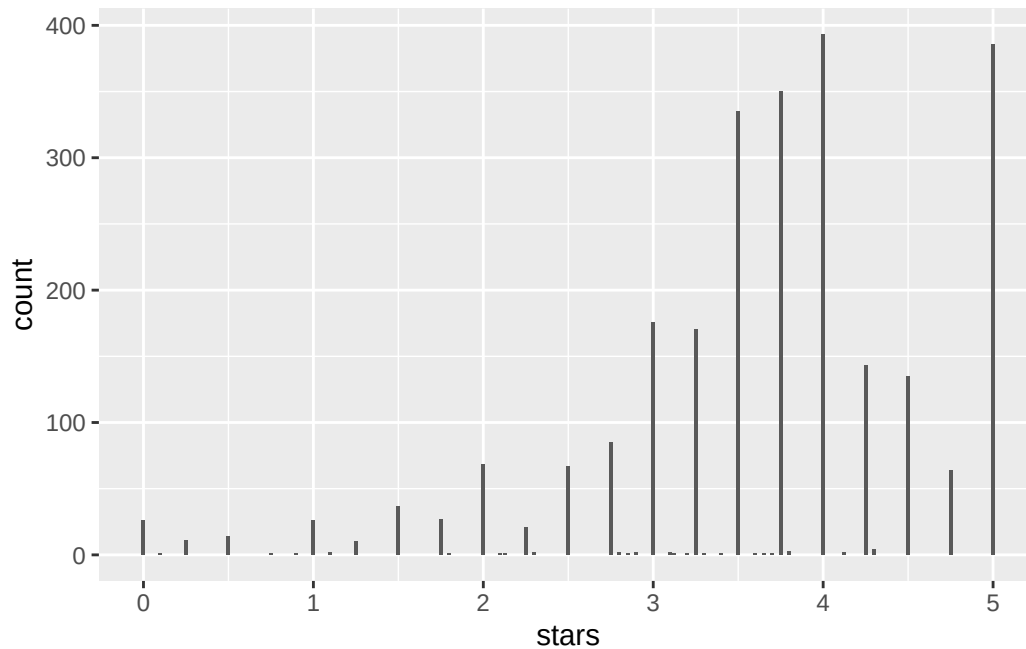
```
ggplot(aes(x=stars,y=count), data = data_ramen_ratings)
```



Die `aes` Funktion gibt hierbei die Ästhetik-Zuordnung an. Im obigen Beispiel wird spezifiziert, dass die x -Achse die Variable `stars` abbildet und die y -Achse die Anzahl der Bewertungen.

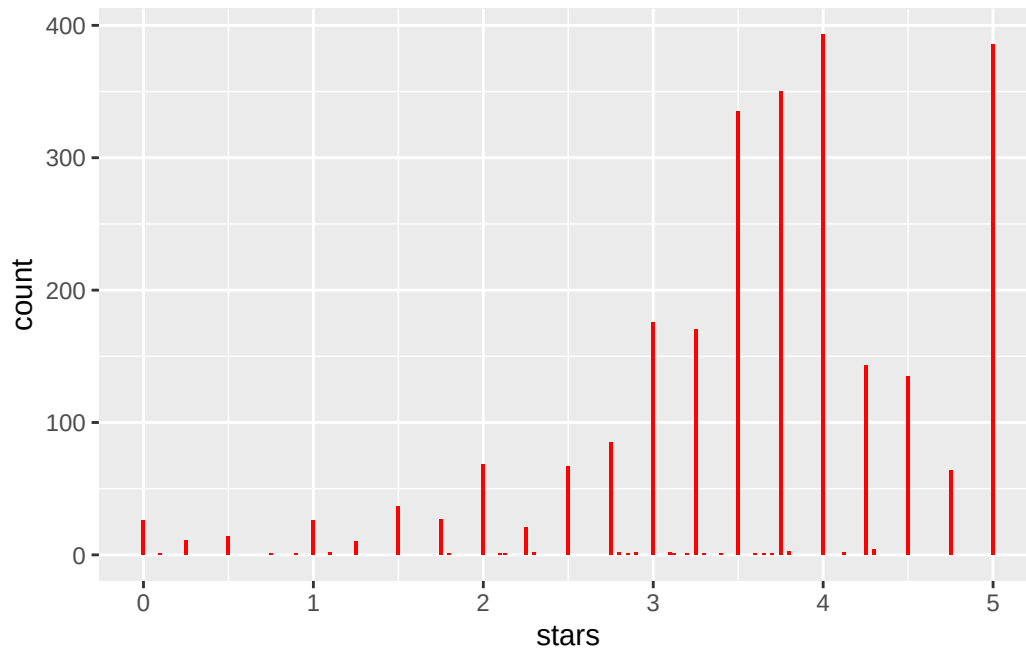
3. Als nächstes werden durch die `geom`-Layer die grafischen Komponenten hinzugefügt, welche beschreiben wie die Daten in der Grafik repräsentiert werden.

```
ggplot(aes(x=stars, y=count), data = data_ramen_ratings)+  
  geom_col()
```



Neben der `ggplot` Funktion wurde in der Obigen Zelle nun die Geometrie Column (Säule) hinzugefügt. Um eine neue Geometrie hinzuzufügen wird nicht wie bei den vorherigen Beispielen der Pipe Operator, sondern ein `+` verwendet. Der Funktion `geom_col` (Geometry Column) wurde kein weiteres Argument hinzugefügt. Man kann durch das Hinzufügen weiterer Argumente wie zum Beispiel `fill` (Füllung) die Farbe der Säulen verändern. So wird durch das Setzen von `fill="red"` die Farbe der Säulen auf rot gesetzt.

```
ggplot(aes(x=stars, y=count), data = data_ramen_ratings)+  
  geom_col(fill="red")
```



Weitere Möglichkeiten die Grafiken anzupassen und die Daten auf verschiedene Arten darzustellen werden in den folgenden Übungen eingeführt.

1.5 Übungsaufgaben

1.5.1 R als Taschenrechner

Aufgabe 1.1. Geben Sie folgende Ausdrücke in die R-Console ein und erklären Sie jeweils das Ergebnis.

```
2 + 3 * 4
(2 + 3) * 4
0.2 * 3 - 1.1
0,2 * 3 - 1.1
2^3^2
(2^3)^2
exp(1)
log(exp(1))
sqrt(16)
16^(1/2)
Sqrt(16)
# Das ist ein Kommentar.
```


Aufgabe 1.2. Weise einer neuen Variable `x` den Wert 4 zu. Weise dann einer neuen Variable `x2` den folgenden Wert zu:

$$\sqrt{3x^2 + \log\left(\frac{1}{e^x}\right) + 5}$$

Überlege was folgende Zeilen ausgeben und führe diese dann aus, um das Ergebnis zu überprüfen.

```
x
x2
x + x2
x.Produkt <- x * x2
x.Produkt
x.Produkt <- x.Produkt * x
ls()
rm(x)
x
```

Aufgabe 1.3. Kopiere den folgenden Code. Welche Ausgabe bewirkt der Code?

```
Zahl <- 10
keine.Zahl <- "10"
Zahl + 1
keine.Zahl + 1
```

1.5.2 Listen und Vektoren

Aufgabe 1.4. Erzeuge folgende Vektoren:

```
[1] 5 6 7 8 9
```

```
[1] 10 9 8 7 6 5 4 3 2 1
```

```
[1] -0.10 -0.08 -0.06 -0.04 -0.02 0.00
```

```
[1] 10000 12500 15000 17500 20000
```

```
[1] -3 -2 -1 0 1 2 -3 -2 -1 0 1 2 -3 -2 -1 0 1 2
```

```
[1] 5.0 6.0 7.0 8.0 9.0 10.0 10.1 10.2 10.3 10.4 10.5
```

Aufgabe 1.5. Gegeben sind die Vektoren:

```
x <- 4:2  
y <- seq(from=0.1, to=0.5, by=0.1)
```

Erkläre, was folgende Ausdrücke ergeben und überprüfe das Ergebnis:

```
x + y  
x * y  
x^3 + 1  
2*x - 3*y  
n <- length(x + y)  
sum(x+y)/n
```

Aufgabe 1.6. Gegeben seien die beiden folgenden Vektoren:

```
x <- seq(from=0, to=100, by=2)  
y <- 100:1
```

Schreibe die Ergebnisse folgender Ausdrücke auf und überprüfe anschließend das Ergebnis in R:

```
x[3]  
y[c(1,3,10)]  
x[1:4]  
x[x > 91]  
x[x > 20 & x <= 30]  
y[y==5 | y>95 | y<3]
```

1.5.3 Datensätze

Für die folgenden Übungen verwenden wir wieder den `data_ramen` Datensatz aus Sektion Abschnitt 1.3.2.

Wende hierfür die zuvor durchgeführten Datenmanipulationen auf den Datensatz an, so dass die Spalte `top_ten` entfernt wurde und die Spalte `stars` in den Datentyp `<dbl>` umgewandelt wurden.

Aufgabe 1.7. Finde heraus, aus welchen fünf Länder die meisten Ramen stammen. *Hinweis: Verwende die `arrange` Funktion in Kombination mit der `desc` Funktion um die Ergebnisse absteigend nach der Größe zu sortieren.*

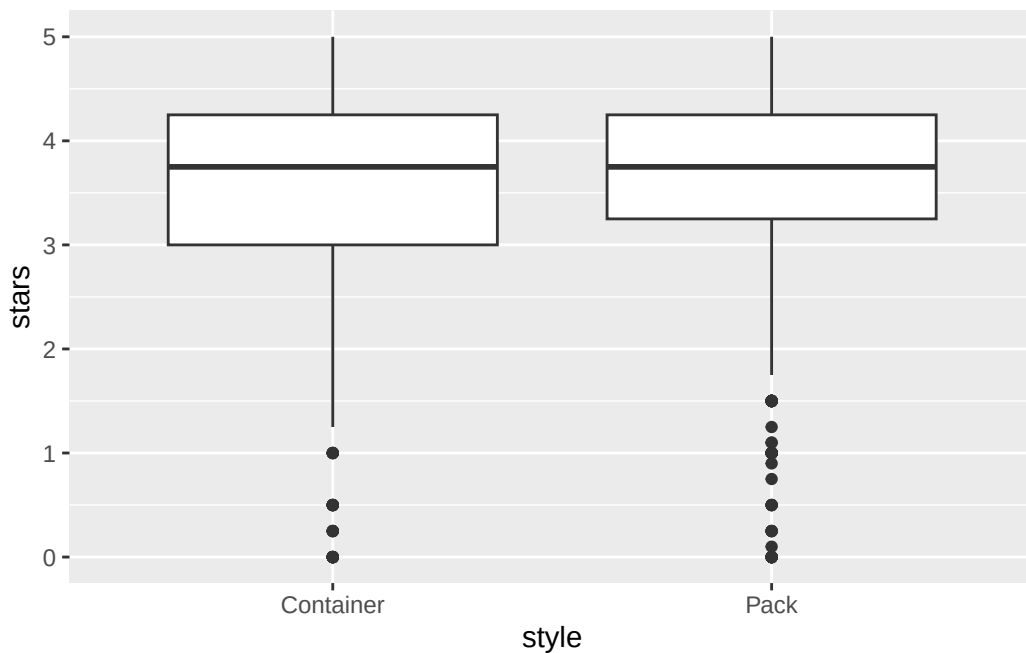
Aufgabe 1.8. Finde heraus, wie viele verschiedene Ramen Sorten in Deutschland (`country == "Germany"`) verfügbar sind.

1.5.4 Grafiken

Aufgabe 1.9. Erkläre die Funktionen und Argumente des folgenden Code Snippets und erkläre was sich im Vergleich zum unbearbeiteten Datensatz inhaltlich geändert hat.

```
1 data_ramen_filtered <- data_ramen %>%
2   filter(style %in% c("Bowl", "Cup", "Pack")) %>%
3   mutate(style = if_else(
4     style=="Pack", "Pack", "Container"
5   ),
6   style= factor(style)
7 )
```

Aufgabe 1.10. Beschreibe die folgende Grafik. Vergleiche dabei auch die beiden Gruppen untereinander.



Aufgabe 1.11. Verwende den Datensatz `data_ramen_filtered` und die `geom_density()`-Funktion, um die Dichte der Variable `stars` mit `{ggplot2}` darzustellen.

2 Lineare Regression

In diesem Kapitel setzen wir uns mit der einfachen und multiplen linearen Regression auseinander. Lineare Regressionsmodelle sind ein Grundbaustein im Kontext der Regression und des statistischen Arbeitens, da die Koeffizienten leicht interpretierbar sind und viele statistische Werkzeuge existieren, um die Sinnhaftigkeit der Modelle und deren Koeffizienten zu überprüfen.

2.1 Der Palmer-Penguin-Datensatz

Der Datensatz für diese und teilweise auch kommende Übungen ist der sogenannte *Palmer-Penguins-Datensatz. Als Teil der `{tidyverse}`-Library ist der Datensatz bereits vorinstalliert und wir können diesen nach dem Hinzufügen der `{tidyverse}`-Library über den Befehl `palmerpenguins::penguins` abrufen. Es ist wie immer ratsam, den Datensatz als neue Variable zu speichern:

```
library(tidyverse)
data_penguin<-palmerpenguins::penguins
```

Das Aufrufen der `glimpse`-Funktion gibt wieder einen kompakten Überblick über die verschiedenen Variablen und Werte, welche diese annehmen.

```
data_penguin %>% glimpse()
```

```
Rows: 344
Columns: 8
$ species      <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
$ island       <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
$ bill_length_mm <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, ~
$ bill_depth_mm <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, ~
$ flipper_length_mm <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186~
$ body_mass_g  <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, ~
$ sex          <fct> male, female, female, NA, female, male, female, male~
$ year         <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

Wie der Name des Datensatzes andeutet, enthält er Informationen über verschiedene Pinguinarten. Konkret umfasst er 344 Beobachtungen von drei unterschiedlichen Pinguinarten, die auf verschiedenen Inseln des Palmer-Archipels in der Nähe der Antarktis leben. Die Daten wurden von Dr. Kristen Gorman erhoben und dienen inzwischen als Standardbeispiel für vielfältige Anwendungen in der Datenanalyse.

Neben der Pinguinart (`species`) und der Beobachtungsinsel (`island`) sind folgende Messgrößen enthalten: die Schnabellänge in Millimetern (`bill_length_mm`), die Schnabeltiefe in Millimetern (`bill_depth_mm`), die Flügelänge in Millimetern (`flipper_length_mm`), das Körpergewicht in Gramm (`body_mass_g`) sowie das Geschlecht (`sex`). Zusätzlich ist das Beobachtungsjahr (`year`) im Datensatz vorhanden, spielt aber für diese Übung keine wesentliche Rolle.

2.1.1 Deskriptive Statistiken

Die Funktion `summary()` in R erzeugt beim Aufruf eine deskriptive Zusammenfassung des übergebenen Datensatzes. Für quantitative Variablen werden dabei deskriptive Statistiken wie Minimum, Maximum, Median, arithmetisches Mittel und die Quartile ausgegeben. Bei nominalen oder kategorialen Variablen werden die einzelnen Ausprägungen und deren absolute Häufigkeiten dargestellt. Zusätzlich wird für jede Variable die Anzahl der fehlenden Werte (kodiert als NA) aufgeführt.

```
summary(data_penguin)
```

species	island	bill_length_mm	bill_depth_mm
Adelie :152	Biscoe :168	Min. :32.10	Min. :13.10
Chinstrap: 68	Dream :124	1st Qu.:39.23	1st Qu.:15.60
Gentoo :124	Torgersen: 52	Median :44.45	Median :17.30
		Mean :43.92	Mean :17.15
		3rd Qu.:48.50	3rd Qu.:18.70
		Max. :59.60	Max. :21.50
		NA's :2	NA's :2

flipper_length_mm	body_mass_g	sex	year
Min. :172.0	Min. :2700	female:165	Min. :2007
1st Qu.:190.0	1st Qu.:3550	male :168	1st Qu.:2007
Median :197.0	Median :4050	NA's : 11	Median :2008
Mean :200.9	Mean :4202		Mean :2008
3rd Qu.:213.0	3rd Qu.:4750		3rd Qu.:2009
Max. :231.0	Max. :6300		Max. :2009
NA's :2	NA's :2		

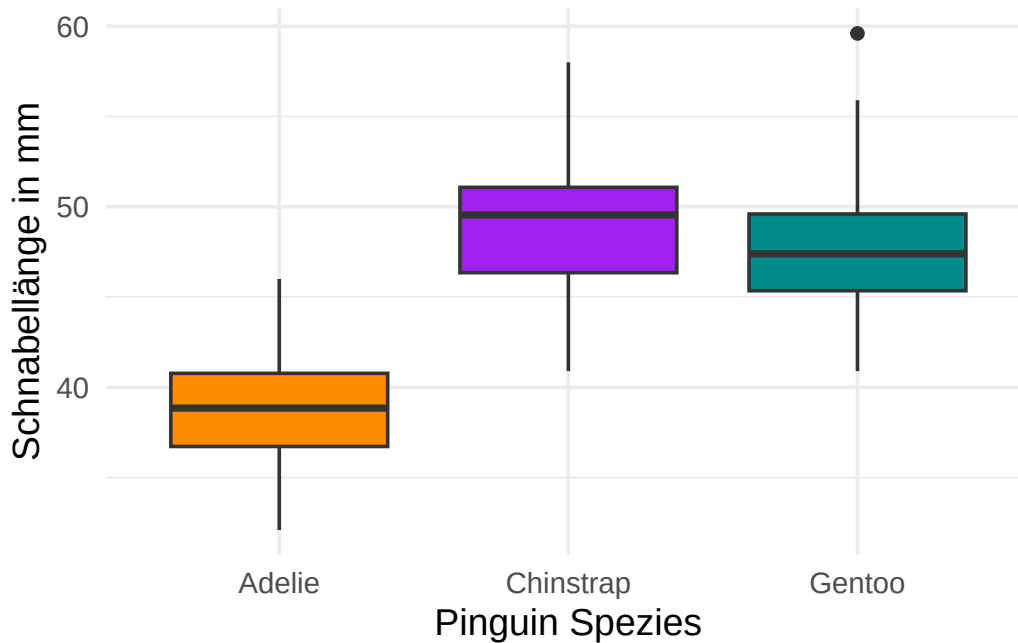
species	n	1st Qu.	Median	Mean	3rd Qu.
Adelie	146	36.725	38.85	38.82397	36.725
Chinstrap	68	46.350	49.55	48.83382	46.350
Gentoo	119	45.350	47.40	47.56807	45.350

Um deskriptive Statistiken für verschiedene Subgruppen metrischer Variablen zu berechnen, bieten sich die in der vorherigen Übung eingeführten Funktionen `summarise()` und `group_by()` aus der `dplyr`-Library an:

```
data_penguin %>%
  na.omit() %>%
  group_by(species) %>%
  summarise(n = n(),
            "1st Qu." = quantile(bill_length_mm, 0.25),
            "Median" = median(bill_length_mm),
            "Mean" = mean(bill_length_mm),
            "3rd Qu." = quantile(bill_length_mm, 0.75),
            )
```

Wir können diese Statistiken auch grafisch mit einem Boxplot darstellen:

```
data_penguin %>%
  na.omit() %>%
  ggplot(aes(x=species,y=bill_length_mm, fill = species))+
  geom_boxplot()+
  scale_fill_manual(values = c("darkorange","purple","cyan4"))+
  labs(
    x = "Pinguin Spezies",
    y = "Schnabellänge in mm"
  )+
  theme_minimal(base_size=14)+
  theme(
    legend.position = "none"
  )
```



Im Vergleich zur Visualisierung in Aufgabe 1.10 implementierte die oben dargestellte Grafik folgende Anpassungen:

1. Der Parameter `fill` in der Funktion `aes()` weist den grafischen Elementen Füllfarben zu.
2. Die Funktion `scale_fill_manual()` definiert die Füllfarben manuell. Hier setzen wir die Farbwerte (`values`) auf `c("darkorange", "purple", "cyan4")`.
3. Mit der Funktion `labs()` und den Argumenten `x` und `y` passen wir die Beschriftungen der x- und y-Achse an.
4. Die Funktion `theme_minimal(base_size = 14)` legt unter anderem einen weißen Hintergrund für die Grafik fest und setzt die Basis-Schriftgröße auf **14pt**.
5. Die Anweisung `theme(legend.position = "none")` unterdrückt die Legende der Grafik, was die Übersichtlichkeit erhöht.

Der bewusste Einsatz von Farben, Formen und Texten in Grafiken vereinfacht das Lesen und Verarbeiten der enthaltenen Informationen maßgeblich. Die effektive visuelle Kommunikation von Daten ist eine essenzielle Kompetenz, die man in jedem Berufsfeld, welches Daten involviert, beherrschen muss.

2.2 Die einfache lineare Regression

Die lineare Regression erweist sich als ein vergleichsweise einfaches, aber äußerst leistungsstarkes Werkzeug in der Statistik. Ihre Vorteile umfassen die hervorragende Interpretierbarkeit der

Koeffizienten – sowohl hinsichtlich ihrer Werte als auch ihrer statistischen Signifikanz.

In der einfachen linearen Regression besteht das Ziel darin, eine abhängige Variable (auch Zielvariable, Outcome oder Target genannt) $Y \in \mathbb{R}^K$ durch die Gleichung

$$Y = b_0 + b_1 X + \varepsilon \quad (2.1)$$

zu approximieren. Hierbei ist $X \in \mathbb{R}^K$ die unabhängige Variable, $b_0, b_1 \in \mathbb{R}$ sind die Regressionskoeffizienten (Achsenabschnitt und Steigung) und $\varepsilon \sim \mathcal{N}(0, 1) \in \mathbb{R}^K$ ist ein Fehlerterm, der als normalverteilt angenommen wird.

Gleichung Gleichung 2.1 bezeichnen wir auch als einfache lineare Regressionsgleichung oder Modellgleichung. Dabei sind X und Y Vektoren der Dimension $K \in \mathbb{N}$, also $X = (x_1, \dots, x_K)$ und $Y = (y_1, \dots, y_K)$ mit $x_i \in \mathbb{R}$ und $y_i \in \mathbb{R}$ für alle $k = 1, \dots, K$.

Wir können die Regressionsgleichung Gleichung 2.1 komponentenweise betrachten:

$$y_k = b_0 + b_1 x_k + \varepsilon_k, \quad k = 1, \dots, K$$

K entspricht hierbei wieder der Anzahl der Observationen.

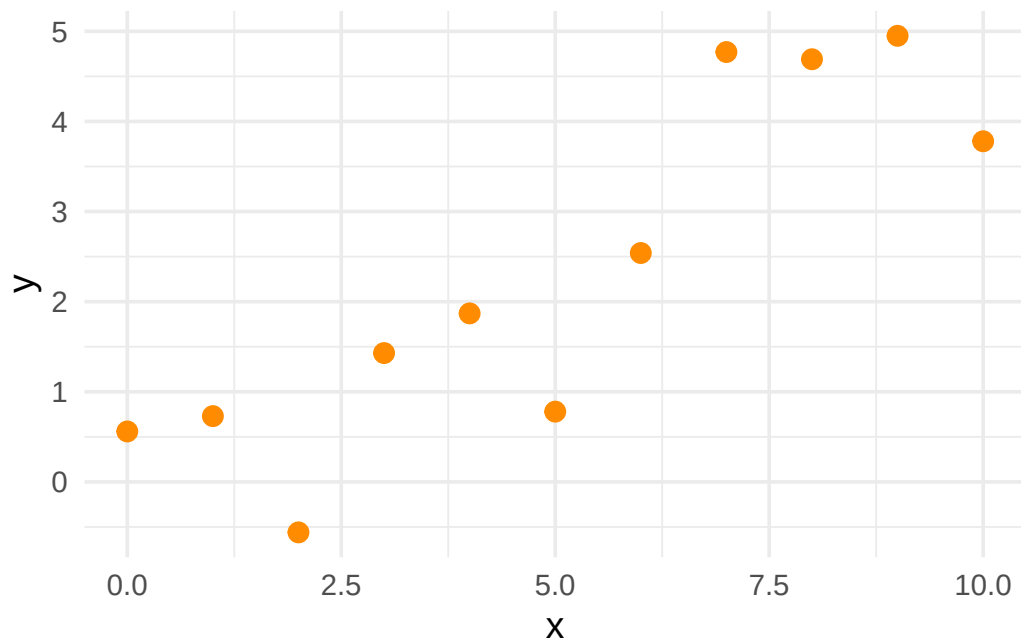
Beispiel 2.1. Betrachte folgenden synthetischen Datensatz:

```
set.seed(123)

data_example <- tibble(
  x = seq(0, 10, length.out = 11),
  y = round(0.5 * x - rnorm(11, 0, 1), 2)
)
```

Die folgende Grafik zeigt die Daten, visualisiert als Punktwolke:

x	y
0	0.56
1	0.73
2	-0.56
3	1.43
4	1.87
5	0.78
6	2.54
7	4.77
8	4.69
9	4.95
10	3.78



Um nun eine Regressionsgerade mit den Parametern $\hat{b}_0$ und $\hat{b}_1$ zu bestimmen, welche die abhängige Variable Y bestmöglich approximiert, verwenden wir die folgenden Formeln zur Schätzung der Koeffizienten:

$$\hat{b}_1 = \frac{\sum_{k=1}^K x_k y_k - K \bar{x} \bar{y}}{\sum_{k=1}^K x_k^2 - K \bar{x}^2} \quad \text{und} \quad \hat{b}_0 = \bar{y} - \hat{b}_1 \bar{x}$$

Dabei repräsentieren $\bar{x} = \frac{1}{K} \sum_{k=1}^K x_k$ und $\bar{y} = \frac{1}{K} \sum_{k=1}^K y_k$ den Stichprobenmittelwert der

unabhängigen Variable X bzw

Für das vorherige Beispiel ergeben sich die geschätzten Koeffizienten zu $\hat{b}_1 = 0.4908$ und $\hat{b}_0 = 0.2253$. Daraus resultiert die geschätzte Regressionsgleichung mit eingesetzten Werten:

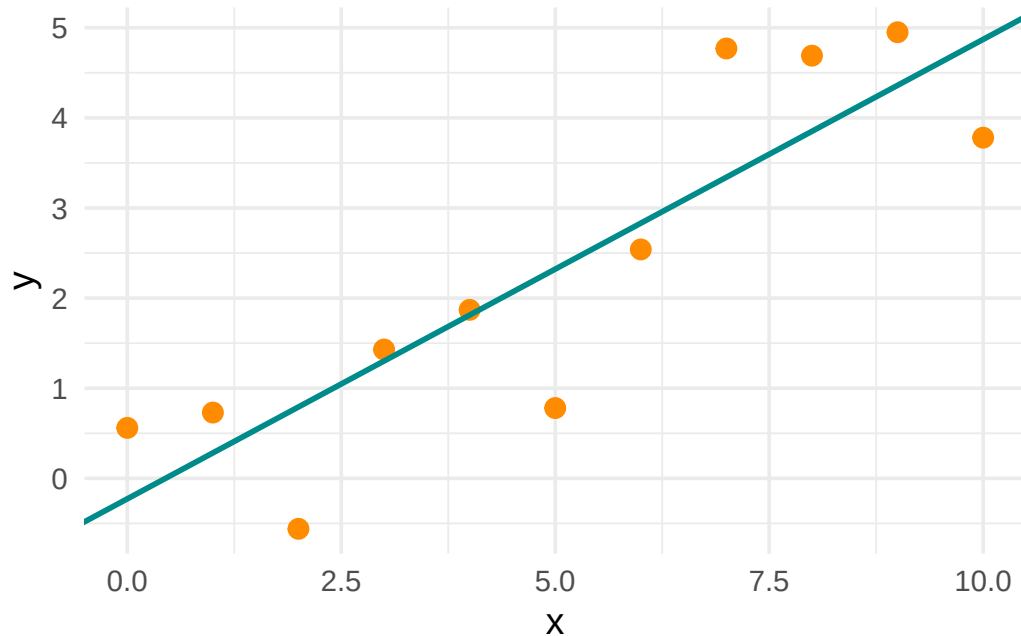
$$\hat{y} = \hat{b}_1 x + \hat{b}_0 = 0.5098x - 0.2273$$

Wir können die Regressionsgerade dann mithilfe der `geom_abline`-Funktion visualisieren:

```

1 data_example %>% ggplot(aes(x=x,y=y))+
2   geom_point(color="darkorange",
3             size = 3)+
4   geom_abline(slope = 0.5098,
5               intercept = -0.2273,
6                 linewidth = 1,
7                 color = "cyan4")+
8   theme_minimal(base_size=14)

```



In Zeile 4 spezifizieren wir mit dem `slope`-Argument die Steigung, bzw. den Parameter $\hat{b}_1$ und in Zeile 5 durch das Argument `intercept` den Schnittpunkt mit der y -Achse, bzw. den Parameter $\hat{b}_0$. Die Argumente `linewidth` und `color` steuern die Dicke und Farbe der Linie.

! Wichtig

Die Notation der Schätzgleichung unterscheidet sich von der Modellgleichung. Bei geschätzten Parametern oder Werten verwendet man die Notation $\hat{y}$, $\hat{b}_1$, und $\hat{b}_0$, um geschätzte Werte darzustellen.

2.2.1 Schätzung der Modellparameter in R

Um eine lineare Regression mit R durchzuführen, nutzen wir die `{tidymodels}`-Library. `{tidymodels}` erweitert das Konzept der `{tidyverse}`-Bibliothek um statistische Modellierung. Der Prozess zur Spezifikation eines linearen Modells gliedert sich hierbei, unabhängig vom verwendeten Datensatz, in zwei separate Phasen:

1. Modellspezifikation
2. Schätzung der Modellparameter

Diese Phasen werden durch die Funktionen `linear_reg()` und `fit()` spezifiziert. Die Spezifikation des Modells erfolgt stets unabhängig von den zugrundeliegenden Daten:

```
library(tidymodels)

lm_spec <- linear_reg()
```

Um nun die Modellparameter für die Daten aus Beispiel 2.1 zu schätzen, kombinieren wir die Modellspezifikation mit der `fit`-Funktion:

```
1 lm_fit <- lm_spec %>% fit(
2   data = data_example,
3   formula = y ~ x
4 )
```

1. In der ersten Zeile deklarieren wir ein neues Objekt namens `lm_fit`. Indem wir die zuvor definierte Modellspezifikation `lm_spec` an die Funktion `fit()` übergeben, weisen wir `fit()` an, die Parameter für ein lineares Regressionsmodell zu schätzen.
2. In der zweiten Zeile legen wir mithilfe des Arguments `data` den Datensatz fest, anhand dessen die Modellparameter b_0 und b_1 geschätzt werden sollen.
3. Nach der Datenspezifikation müssen wir noch die abhängige und die unabhängige Variable definieren. Dies geschieht mithilfe des Operators `~`. Die Variable links des `~`-Operators (`y`) ist die abhängige Variable, und die Variable rechts des `~`-Operators (`x`) ist die unabhängige Variable.

Das Objekt `lm_fit` enthält nun unter anderem die geschätzten Modellparameter b_0 und b_1 . Es existieren verschiedene Wege, um auf diese zuzugreifen:

Durch das Ausgeben des Objekts `lm_fit` erhalten wir neben den Koeffizienten b_0 (**Intercept**) und b_1 (`x`) beispielsweise auch die Formel, die zur Schätzung der Modellparameter verwendet wurde:

```
lm_fit
```

parsnip model object

Call:

```
stats::lm(formula = y ~ x, data = data)
```

Coefficients:

```
(Intercept)          x  
    -0.2273      0.5098
```

Eine detaillierte Übersicht der geschätzten Modellparameter und deren statistischer Signifikanz erhalten wir durch folgenden Code-Abschnitt:

```
lm_fit %>%  
  extract_fit_engine() %>%  
  summary()
```

Call:

```
stats::lm(formula = y ~ x, data = data)
```

Residuals:

```
      Min       1Q   Median       3Q      Max  
-1.5418 -0.6913  0.1278  0.6881  1.4285
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)  
(Intercept)  -0.2273     0.5768  -0.394 0.702757  
x              0.5098     0.0975   5.229 0.000543 ***
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.023 on 9 degrees of freedom

Multiple R-squared: 0.7523, Adjusted R-squared: 0.7248

F-statistic: 27.34 on 1 and 9 DF, p-value: 0.0005429

Die Funktion `extract_fit_engine()` ruft das Attribut `fit` aus dem Objekt `lm_fit` ab. Das Attribut `fit` eines linearen Modells beinhaltet neben den geschätzten Parametern zahlreiche weitere Informationen, die die Funktion `summary()` übersichtlich zusammenfasst. Diese werden wir in den folgenden Abschnitten genauer betrachten.

2.2.1.1 Residuen

Der Abschnitt **Residuals** der Zusammenfassung beschreibt deskriptive Merkmale der Residuen, welche sich durch die Schätzung der Modellparameter ergeben haben.

💡 Wiederholung

Zur Erinnerung: Für eine Observation (x_i, y_i) mit geschätztem Wert $\hat{y}_i = \hat{b}_1 x_i + \hat{b}_0$ ist das Residuum $\hat{\varepsilon}_i$ als $\hat{\varepsilon}_i = y_i - \hat{y}_i = y_i - (\hat{b}_1 x_i + \hat{b}_0)$ definiert.

...

Residuals:

Min	1Q	Median	3Q	Max
-1.5418	-0.6913	0.1278	0.6881	1.4285

Residual standard error: 1.023 on 9 degrees of freedom

...

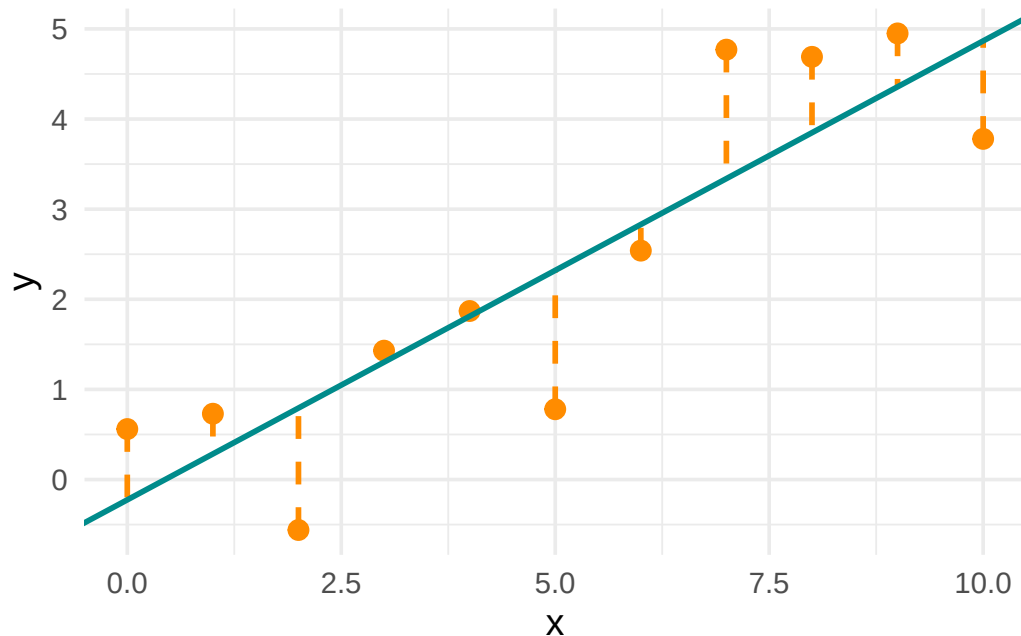
Konkret heißt das für das obige lineare Modell, dass etwa das kleinste Residuum den Wert -1.5418 annimmt und 75% der Residuen maximal den Wert 0.6881 besitzen.

Neben den deskriptiven Merkmalen wird im unteren Teil der Ausgabe auch noch der Standardfehler der Residuen angegeben.

Die geschätzten Residuen für das geschätzte Modell können auch grafisch dargestellt werden:

```
estimates <- lm_fit %>%
  predict(data_example) %>%
  pluck(1)

data_example %>% ggplot(aes(x=x,y=y))+
  geom_segment(aes(x=x,xend=x,y=y,yend=estimates),
    linewidth = 1,
    linetype = 2,
    color = "darkorange")+
  geom_point(color="darkorange",
    size = 3)+
  geom_abline(slope = 0.5092,
    intercept = -0.2253,
    linewidth = 1,
    color = "cyan4")+
  theme_minimal(base_size=14)
```



Zuerst haben wir mithilfe der Funktion `predict()` die vorhergesagten Werte $\hat{y} = \hat{b}_0 + x\hat{b}_1$ berechnet. Da `predict()` ein Dataframe oder Tibble zurückgibt, können wir die relevante Spalte mit `purrr::pluck()` als Vektor extrahieren.

Im Vergleich zur vorherigen Visualisierung stellen wir die Residuen nun mit der Funktion `geom_segment()` dar. `geom_segment()` zeichnet gerade Linien zwischen den Punkten `(x, y)` und `(xend, yend)`, wobei die Werte im Datensatz und die vorhergesagten Werte in der Liste `estimates` diese Endpunkte definieren. Um die Residuallinien hervorzuheben, haben wir ihre Breite (`linewidth`) angepasst, den Linientyp (`linetype`) auf „gestrichelt“ gesetzt und ihre Farbe (`color`) an die entsprechenden Datenpunkte angeglichen.

2.2.1.2 Koeffizienten

Der `Coefficients` Abschnitt beschreibt die geschätzten Parameter und deren statistische Signifikanz.

...

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.2273	0.5768	-0.394	0.702757
x	0.5098	0.0975	5.229	0.000543 ***

...

- Die Spalte `Estimate` gibt hierbei den geschätzten Wert für b_0 und b_1 an.
- Die Spalte `Std. Error` gibt für jeden geschätzten Parameter den Standardfehler an.
- Die Spalte `t value` gibt den Wert der t -Statistik für den entsprechenden Parameter an.
- Die Spalte `Pr(>|t|)` beschreibt den p -Wert des zugrundeliegenden t -Tests.

Zusätzlich werden rechts von der letzten Spalte noch Signifikanzcodes wiedergegeben:

...

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

...

Diese beschreiben, zu welchem Signifikanzniveau α ein Parameter signifikant ist. Falls also zum Beispiel bei der Variable `x` der Signifikanzcode `'***'` gegeben ist, dann bedeutet das, dass $0 < p < 0.001$.

Wiederholung

Der t -Test für den Koeffizienten b einer Variable x einer linearen Regression lautet:

$$H_0 : b = 0 \quad \text{vs.} \quad H_1 : b \neq 0$$

Der Wert der t -Statistik ist durch $\frac{\hat{b}}{\text{sd}_{\hat{b}}} \sqrt{K}$ gegeben, wobei $\text{sd}_{\hat{b}}$ den Standardfehler von $\hat{b}$ und K die Anzahl der Observationen beschreibt. Falls der Wert der t -Statistik im Bereich $B = (-\infty, -x_{1-\frac{\alpha}{2}}) \cup (x_{1-\frac{\alpha}{2}}, \infty)$ liegt, dann können wir H_0 verwerfen. Hierbei verwenden wir die Approximation $x_{1-\frac{\alpha}{2}} \approx z_{1-\frac{\alpha}{2}}$, wobei $z_{1-\frac{\alpha}{2}}$ das $1 - \frac{\alpha}{2}$ Quantil der Standardnormalverteilung ist.

Falls wir H_0 nicht verwerfen (z. B. zum Niveau $\alpha = 0.001$), können wir nicht mit ausreichender Sicherheit sagen, dass $\hat{b} \neq 0$ und der Parameter somit einen Einfluss auf die abhängige Variable y hat.

2.2.2 Das Gütemaß R^2

Neben den Parametern, deren Signifikanz und den Residuen sind wir ebenso an dem Gütemaß R^2 interessiert. Das Gütemaß R^2 beschreibt in Worten gefasst, wie viel Prozent der Varianz der Daten wir durch unser Modell erklären können.

...

Multiple R-squared: 0.7523, Adjusted R-squared: 0.7248

...

Falls also, wie im obigen Output zu sehen ist, der R^2 des Modells bei 0.7526 liegt, kann unser Modell ca. 75% der Varianz in den Daten erklären.

Um das Gütemaß zu berechnen, wird folgende Formel herangezogen:

$$R^2 = \frac{\sum_{k=1}^K (\hat{y}_k - \bar{y})^2}{\sum_{k=1}^K (y_k - \bar{y})^2} \quad (2.2)$$

2.2.2.1 Der F -Test

Die letzte Zeile der Zusammenfassung des Modells zeigt die F -Statistik mit korrespondierendem p -Wert.

...

F-statistic: 27.34 on 1 and 9 DF, p-value: 0.0005429

...

Der F -Test beschreibt, ob das Modell als gesamtes signifikant ist. Die zugrunde liegenden Hypothesen des F -Tests sind:

$$H_0 : b_i = 0 \text{ für alle } i, \quad \text{vs} \quad H_1 : b_i \neq 0 \text{ für mindestens ein } i$$

Falls wir H_0 nicht verwerfen, können wir nicht mit hoher Sicherheit sagen, dass **alle** Regressoren **keinen** Einfluss auf die abhängige Variable haben. Im Kontext der einfachen linearen Regression sind der F -Test und t -Test insofern äquivalent, dass

$$\text{F-statistic} = (\text{t value})^2$$

2.3 Die multiple lineare Regression

Die multiple lineare Regression erweitert die Idee der einfachen linearen Regression durch das Hinzufügen von weiteren erklärenden Variablen.

Die Modellgleichung lautet somit

$$Y = b_0 + b_1 X_1 + \dots + b_J X_J + \varepsilon,$$

falls die Daten aus J verschiedenen Variablen bestehen. Hierbei gilt wieder, dass für jede Variable X_j , $j = 1, \dots, J$ K Observationen vorhanden sind. Man geht hierbei davon aus, dass die K Observationen zufällig von einer Gesamtpopulation gezogen wurden.

Somit kann die Modellgleichung auch wieder komponentenweise beschrieben werden:

$$y_k = b_0 + b_1 x_{k1} + \dots + b_J x_{kJ} + \varepsilon_k$$

Zusätzlich kann die Modellgleichung auch in Matrixnotation dargestellt werden:

$$Y = Xb + \varepsilon \tag{2.3}$$

Wobei $Y = (y_1, \dots, y_K)' \in \mathbb{R}^{K \times 1}$, $X = (1, x_1, \dots, x_J) \in \mathbb{R}^{K \times (J+1)}$, $b = (b_0, b_1, \dots, b_J)' \in \mathbb{R}^{(J+1) \times 1}$ und $\varepsilon \sim \mathcal{N}(0, 1) \in \mathbb{R}^K$.

Das Schätzen der Modellparameter $b_0, \dots, b_J$ auf Basis gegebener Observationen wird bei der multiplen linearen Regression üblicherweise nicht mehr von Hand durchgeführt. Der einfachste Weg, die Modellparameter von Hand zu bestimmen, ist allerdings durch folgende Formel gegeben:

$$b = (X'X)^{-1}X'Y \tag{2.4}$$

Wobei X' die transponierte Matrix von X darstellt und $(X'X)^{-1}$ die inverse von $(X'X)$. Die Komplexität in der Berechnung von b anhand Gleichung 2.4 liegt in der Berechnung der inversen Matrix.

2.3.1 Multiple lineare Regression in R

Das Schätzen der Modellparameter im multidimensionalen Fall in R funktioniert fast analog wie im einfachen Fall.

Als Beispiel verwenden wir hierfür nun keinen synthetischen Datensatz mehr, sondern den `data_penguin`-Datensatz aus Abschnitt 2.1.

Angenommen, die abhängige Variable ist das Gewicht der Pinguine (`body_mass_g`). Wie im einfachen linearen Regressionsmodell wird zuerst das Modell durch die `linear_reg`-Funktion spezifiziert. Nach der Spezifikation werden dann die Parameter mithilfe der `fit`-Funktion geschätzt. Das Argument `formula` hat rechts des `~`-Operators einen Punkt stehen. Der `.`-Operator signalisiert, dass alle verbleibenden Variablen als unabhängige Variablen verwendet werden sollen.

```
mlr_spec <- linear_reg()
mlr_fit <- mlr_spec %>%
  fit(data = data_penguin,
       formula = body_mass_g ~.)
```

Alternativ kann man statt des `.` auch alle Variablen wie folgt ausschreiben, wobei das in den meisten Fällen aufgrund schlechter Lesbarkeit nicht zu empfehlen ist:

```
mlr_fit <- mlr_spec %>%
  fit(data = data_penguin,
       formula = body_mass_g ~ species + island + bill_length_mm +
         bill_depth_mm + flipper_length_mm + sex + year)
```

Analog zum eindimensionalen Fall kann mit den `extract_fit_engine`- und `summary`-Funktion eine Zusammenfassung erzeugt werden:

```
mlr_fit_summary <- mlr_fit %>%
  extract_fit_engine() %>%
  summary()

mlr_fit_summary
```

Call:

```
stats::lm(formula = body_mass_g ~ species + island + bill_length_mm +
  bill_depth_mm + flipper_length_mm + sex + year, data = data)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-809.70	-180.87	-6.25	176.76	864.22

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	84087.945	41912.019	2.006	0.04566	*
speciesChinstrap	-282.539	88.790	-3.182	0.00160	**
speciesGentoo	890.958	144.563	6.163	2.12e-09	***
islandDream	-21.180	58.390	-0.363	0.71704	
islandTorgersen	-58.777	60.852	-0.966	0.33482	
bill_length_mm	18.964	7.112	2.667	0.00805	**
bill_depth_mm	60.798	20.002	3.040	0.00256	**
flipper_length_mm	18.504	3.128	5.915	8.46e-09	***

```

sexmale          378.977      48.074    7.883 4.95e-14 ***
year             -42.785      20.949   -2.042  0.04194 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Residual standard error: 286.5 on 323 degrees of freedom

(11 observations deleted due to missingness)

Multiple R-squared: 0.8768, Adjusted R-squared: 0.8734

F-statistic: 255.4 on 9 and 323 DF, p-value: < 2.2e-16

Die Modellgleichung für das multiple lineare Regressionsmodell mit eingesetzten Zahlen lautet also

$$\begin{aligned}
 \text{body_mass_g}_i = & 84087.945 + \text{speciesChinstrap}_i \cdot -282.53 + \\
 & \text{speciesGentoo}_i \cdot 890.958 + \text{islandDream}_i \cdot -21.180 + \\
 & \text{islandTorgersen}_i \cdot -58.777 + \text{bill_length_mm}_i \cdot 18.964 + \quad (2.5) \\
 & \text{bill_depth_mm}_i \cdot 60.798 + \text{flipper_length_mm}_i \cdot 18.504 + \\
 & \text{sexmale}_i \cdot 378.977 + \text{year}_i \cdot -42.785 + \varepsilon_i
 \end{aligned}$$

2.3.2 Interpretation und Vergleich der Koeffizienten

Die geschätzten Koeffizienten im MLR-Modell lassen sich ähnlich wie bei der einfachen linearen Regression interpretieren, da der Effekt der Koeffizienten auch linear ist.

So erwarten wir zum Beispiel im obigen Beispiel, dass das Gewicht eines Pinguins im Schnitt um 18.504g steigt, falls sich die Länge der Flossen um 1 mm erhöht ($b_{\text{flipper_length_mm}} = 18.504$) und alle anderen Einflussfaktoren konstant bleiben.

Neben dem direkten Einfluss der geschätzten Koeffizienten können wir zwar die statistische Signifikanz der verschiedenen Parameter vergleichen, aber es ist ohne Weiteres nicht klar, welcher Koeffizient den größten Einfluss auf das Gewicht der Pinguine hat.

Um herauszufinden, welche Variable den größten Einfluss besitzt, müssen wir entweder die Variablen oder die Koeffizienten standardisieren. Ohne eine Standardisierung wäre ein Vergleich der geschätzten Koeffizienten nicht sinnvoll, da die Variablen teilweise in anderen Einheiten gemessen werden oder gar nicht metrisch sind. So macht etwa ein Vergleich der Parameter für die Schnabellänge und das Geschlecht keinen Sinn, da die Schnabellänge in *mm* gemessen wird und das Geschlecht nur die Ausprägungen "male" und "female" besitzt. Den standardisierten Koeffizienten für eine Variable x können wir mithilfe der Formel

$$\hat{b}_{x,\text{std}} = \hat{b}_x \frac{\text{sd}_x}{\text{sd}_y}$$

berechnen, wobei sd_x für die Standardabweichung von x und sd_y für die Standardabweichung von y steht.

Durch die Standardisierung erreichen wir, dass die standardisierten Koeffizienten vergleichbar sind. Der geschätzte Koeffizient mit dem größten standardisierten Wert hat dadurch den größten Einfluss auf die abhängige Variable, während der Koeffizient mit dem kleinsten standardisierten Wert den kleinsten Einfluss hat.

Durch die `lm.beta::lm.beta()`-Funktion können wir die standardisierten Koeffizienten für ein lineares Modell in R bestimmen:

```
mlr_fit %>%
  extract_fit_engine() %>%
  lm.beta::lm.beta() %>%
  summary()
```

Call:

```
stats::lm(formula = body_mass_g ~ species + island + bill_length_mm +
  bill_depth_mm + flipper_length_mm + sex + year, data = data)
```

Residuals:

Min	1Q	Median	3Q	Max
-809.70	-180.87	-6.25	176.76	864.22

Coefficients:

	Estimate	Standardized	Std. Error	t value	Pr(> t)	
(Intercept)	8.409e+04	NA	4.191e+04	2.006	0.04566	*
speciesChinstrap	-2.825e+02	-1.417e-01	8.879e+01	-3.182	0.00160	**
speciesGentoo	8.910e+02	5.311e-01	1.446e+02	6.163	2.12e-09	***
islandDream	-2.118e+01	-1.271e-02	5.839e+01	-0.363	0.71704	
islandTorgersen	-5.878e+01	-2.545e-02	6.085e+01	-0.966	0.33482	
bill_length_mm	1.896e+01	1.288e-01	7.112e+00	2.667	0.00805	**
bill_depth_mm	6.080e+01	1.487e-01	2.000e+01	3.040	0.00256	**
flipper_length_mm	1.850e+01	3.221e-01	3.128e+00	5.915	8.46e-09	***
sexmale	3.790e+02	2.357e-01	4.807e+01	7.883	4.95e-14	***
year	-4.278e+01	-4.320e-02	2.095e+01	-2.042	0.04194	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
Residual standard error: 286.5 on 323 degrees of freedom
(11 observations deleted due to missingness)
Multiple R-squared: 0.8768, Adjusted R-squared: 0.8734
F-statistic: 255.4 on 9 and 323 DF, p-value: < 2.2e-16
```

Im obigen Code haben wir nach der Erzeugung der standardisierten Koeffizienten auch noch die `summary()`-Funktion aufgerufen, um neben den standardisierten Koeffizienten auch noch eine Übersicht über die restlichen Parameter und Statistiken zu bekommen.

2.3.2.1 Dummy-Encoding

Beim Vergleichen der unabhängigen Variablen mit dem R-Output werden zum Beispiel statt `species` die beiden neuen Variablen `speciesChinstrap` und `speciesGentoo` verwendet. Das liegt daran, dass diese Variable nominal ist (Ausprägungen "Chinstrap", "Gentoo" und "Adelie") und die Ausprägungen deshalb nicht einfach mit geschätzten Koeffizienten multipliziert werden können, wie das im ordinalen oder metrischen Fall ist. Deshalb werden sogenannte *Dummy-Variablen* zur Berechnung der Parameter verwendet. Hierbei werden im Falle der Variable `species` $l - 1 = 3 - 1 = 2$ (mit l die Anzahl der verschiedenen Ausprägungen) neue Variablen angelegt. `speciesChinstrap` nimmt den Wert 1 an, falls ein Datenpunkt im Datensatz der Spezies Chinstrap angehört, und ansonsten 0. Gleich wird auch mit `speciesGentoo` verfahren. Allerdings ist hier nirgends die Variable `speciesAdelie` spezifiziert! Das liegt daran, dass der Einfluss dieses Datenpunktes durch den *Intercept*, also b_0 , abgebildet wird, um Multikollinearität zu vermeiden (siehe Abschnitt 2.3.3.5).

Das gleiche Vorgehen wurde auch bei den Variablen `island` und `sex` verwendet.

2.3.3 Modellannahmen

Unter welchen Voraussetzungen macht es Sinn, ein lineares Modell zu verwenden, und wie wird geprüft, ob diese erfüllt sind?

Der Hintergrund dieser Voraussetzungen liegt im sogenannten Gauss-Markov-Theorem:

Theorem 2.1 (Gauss-Markov). *Gegeben sei ein lineares Modell wie in Gleichung 2.3 mit unbekannten Parametern $b = (b_0, \dots, b_J)$. Angenommen, es gilt $\varepsilon \sim \mathcal{N}(0, \Sigma)$ mit $\Sigma = \sigma^2 I_J \in \mathbb{R}^{J \times J}$, wobei I_J eine J -dimensionale Einheitsmatrix ist und $\sigma^2 > 0$. Wenn zusätzlich die Matrix $X'X$ invertierbar ist, dann gilt:*

Der Kleinste-Quadrate-Schätzer für den wahren Parametervektor b , gegeben durch Gleichung 2.4, ist ein minimalvarianter linearer erwartungstreuer Schätzer für den wahren Parameter b .

Einfach gesagt bedeutet das:

Falls die einzelnen Störgrößen ε_i , $i = 1, \dots, J$ normalverteilt sind mit Erwartungswert 0 und Varianz $\sigma^2 > 0$ und zusätzlich die Variablen $X_1, \dots, X_J$ linear unabhängig voneinander sind, dann erwarten wir, dass der Schätzer $\hat{b}$ im Mittel dem wahren Parameter entspricht und es keinen anderen erwartungstreuen Schätzer $\tilde{\hat{b}}$ gibt, welcher eine bessere Modellgüte liefert.

Aus diesen Aussagen lassen sich folgende Bedingungen herleiten:

1. Die Beziehung zwischen den unabhängigen und der abhängigen Variable muss tatsächlich linear sein.
2. Die Varianz der Residuen muss konstant sein (Homoskedastizität).
3. Die Residuen sind unabhängig (keine Autokorrelation).
4. Die Residuen sind normalverteilt.

2.3.3.1 Lineare Beziehung

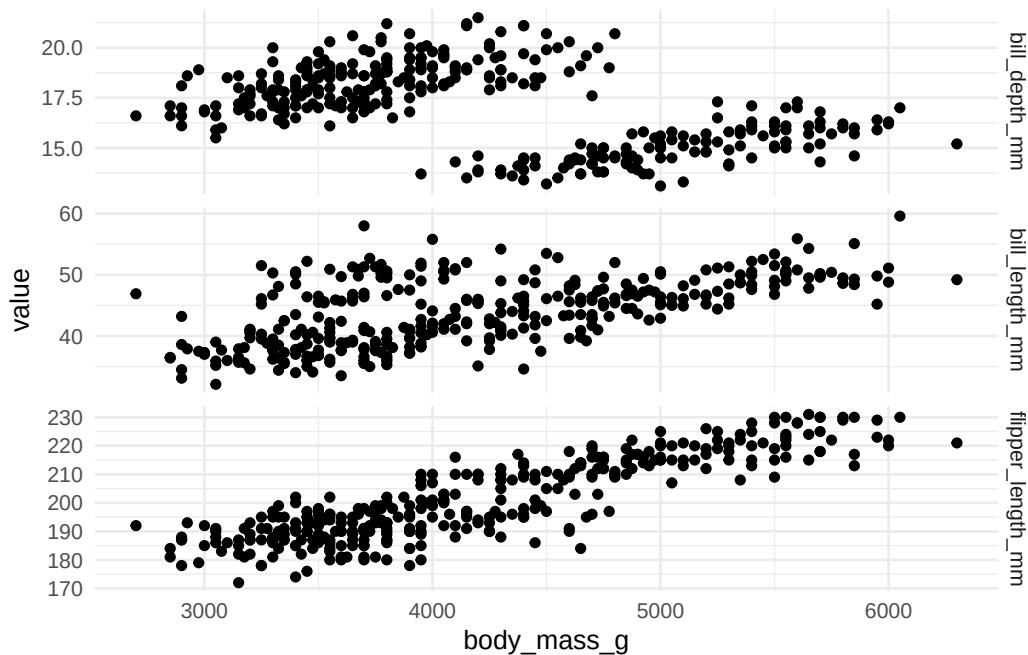
Eine grundlegende Bedingung ist, dass die Beziehung zwischen der abhängigen Variable und den (metrischen) erklärenden Variablen tatsächlich linear ist. Falls diese Bedingung nicht erfüllt ist, können die Effekte der verschiedenen unabhängigen Variablen nicht richtig interpretiert werden. Um zu prüfen, ob diese erfüllt ist, können die paarweise Punktwolken zwischen der abhängigen und den unabhängigen Variablen betrachtet werden.

Das folgende Code-Snippet erzeugt drei Punktwolken, welche jeweils auf der x-Achse die abhängige Variable `body_mass_g` und auf der y-Achse die metrischen unabhängigen Variablen abbilden.

```

1 data_penguin %>%
2   select(ends_with("mm"), body_mass_g) %>%
3   na.omit() %>%
4   pivot_longer(-body_mass_g) %>%
5   ggplot(aes(y=value, x=body_mass_g))+
6   geom_point()+
7   facet_grid(rows = vars(name), scales = "free")+
8   theme_minimal(base_size = 10)

```



In den ersten vier Zeilen wird der Datensatz für die Darstellung aufbereitet:

1. In Zeile 2 werden mithilfe der `select`- und `ends_with`-Funktion alle Variablen des Datensatzes `data_penguin` ausgewählt, deren Name entweder mit `"mm"` endet oder `body_mass_g` ist. Durch diese Zeile werden alle metrischen Werte ausgewählt.
2. In Zeile 3 werden alle Einträge entfernt, welche fehlende Werte beinhalten.
3. Die `pivot_longer`-Funktion wandelt die übergebenen Daten in eine *verlängerte* Version um, welche weniger Spalten, aber mehr Zeilen enthält:

```
# A tibble: 6 x 3
  body_mass_g name      value
    <int> <chr>      <dbl>
1     3750 bill_length_mm  39.1
2     3750 bill_depth_mm  18.7
3     3750 flipper_length_mm 181
4     3800 bill_length_mm  39.5
5     3800 bill_depth_mm  17.4
6     3800 flipper_length_mm 186
```

Durch das Spezifizieren von `-body_mass_g` wird erreicht, dass alle Variablen **außer** `body_mass_g` in zwei Spalten `name` und `value` zusammengefasst werden. Der verlängerte Datensatz enthält also nur noch die drei Spalten `body_mass_g`, `name` und `value`. In der Variable `name` sind die ursprünglichen Variablennamen gespeichert und in `value` der

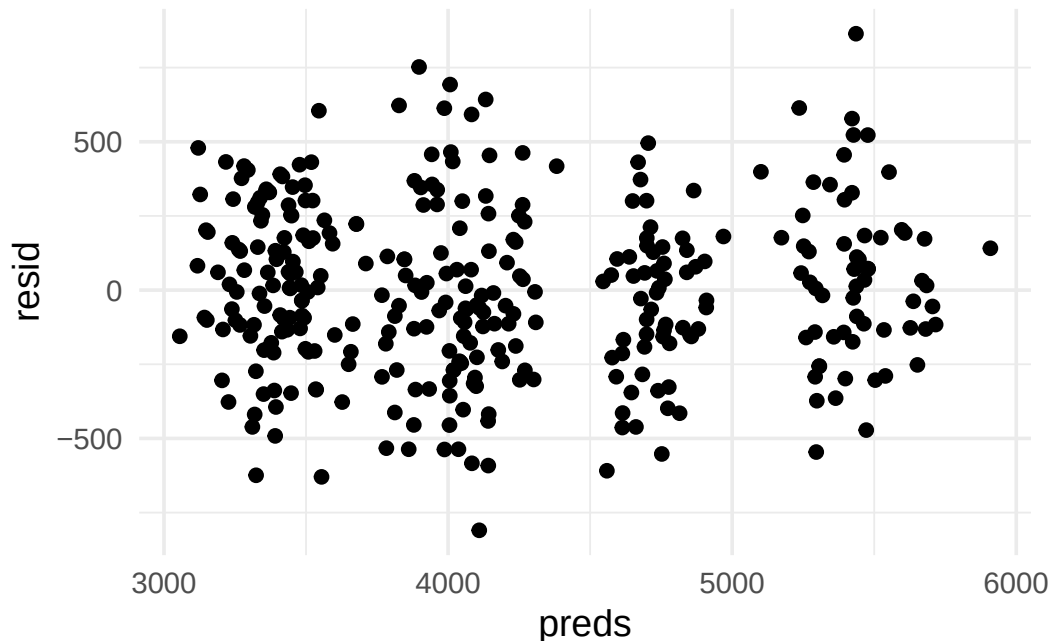
dazugehörige Wert. Dieses Vorgehen erlaubt im nächsten Schritt das Erzeugen von mehreren Plots in Kombination mit der `facet_grid`-Funktion.

4. Die `facet_grid`-Funktion erzeugt eine Matrix von Plots, wobei durch das Argument `rows = vars(name)` spezifiziert wird, dass die Ausprägungen der Variable `names` die Zeilen der Plotmatrix abbilden sollen.

An der obigen Grafik kann abgelesen werden, dass der Zusammenhang zwischen den unabhängigen Variablen und `body_mass_g` linear ist, da ein linearer Trend erkennbar ist. Selbst bei der Variable `bill_depth_mm`, in welcher zwei distinkte Gruppen erkennbar sind, lässt sich jeweils ein linearer Trend feststellen.

2.3.3.2 Konstante Varianz der Residuen

Um die Residuen auf konstante Varianz zu prüfen, verwenden wir ähnlich wie im vorherigen Abschnitt ein visuelles Hilfsmittel: Indem wir die Residuen in Abhängigkeit von den Vorhersagewerten als Punktwolke darstellen, können wir die Streuung um den Ursprung betrachten.



In der obigen Grafik ist die Varianz der Residuen vermutlich konstant.

Begründung: Obwohl unter den Vorhersagewerten `preds` in den Bereichen `[3750,4250]` und `[5250,5500]` Punkte zu erkennen sind, welche als Ausreißer interpretiert werden könnten, sind die restlichen Punkte eher gleichmäßig um den Ursprung verteilt. Gegeben dieser Observation können wir die Vermutung rechtfertigen, dass die Streuung der Residuen konstant ist.

Um eine Grafik wie die obige zu erzeugen, müssen wir zuerst die Residuen und dann die Vorhersagen erzeugen.

Die Residuen können wir ähnlich wie die Zusammenfassung der Modellparameter erzeugen, indem wir zuerst das `fit`-Attribut des Modells aus dem Objekt `mlr_fit` mithilfe der `extract_fit_engine`-Funktion extrahieren. Nach der Extraktion können wir dann mit der `residuals`-Funktion die Residuen auslesen.

```
resid<-mlr_fit %>%  
  extract_fit_engine() %>%  
  residuals()
```

Ein ähnliches Vorgehen können wir auch bei den Vorhersagen anwenden. Allerdings müssen wir die Vorhersagen erst mithilfe der `predict`-Funktion erzeugen. Um Vorhersagen für alle Observationen zu erhalten, reicht es, den gesamten Datensatz als Argument zu übergeben. Durch die `na.omit`-Funktion werden dann noch alle fehlenden Werte entfernt.

```
preds <- mlr_fit %>%  
  extract_fit_engine() %>%  
  predict(data_penguin) %>%  
  na.omit()
```

Nach dem Erzeugen der Residuen und der Vorhersagewerte können wir diese in einem neuen Datensatz speichern und direkt an die `ggplot`-Funktion übergeben:

```
tibble(  
  predictions = preds,  
  residuals = resid  
) %>%  
  ggplot(aes(x=preds,y=resid))+  
  geom_point()+  
  theme_minimal(base_size=14)
```

Neben den Residuen ist es manchmal auch sinnvoll, die *Quadratwurzel des Betrags der standardisierten Residuen* zu betrachten.



Figure 2.1: [Quelle](#)

Die geschätzten Residuen $\hat{\varepsilon} = \hat{y} - y$ werden standardisiert, indem man durch eine empirische Standardabweichung teilt.

So ergibt sich

$$\hat{\varepsilon}_{\text{std}} = \frac{\hat{\varepsilon}}{\text{sd}_{\hat{\varepsilon}}}$$

Die empirische Standardabweichung wird hierbei durch den Term

$$\text{sd}_{\hat{\varepsilon}} = \hat{\sigma} \sqrt{(K-1)/K}$$

mit

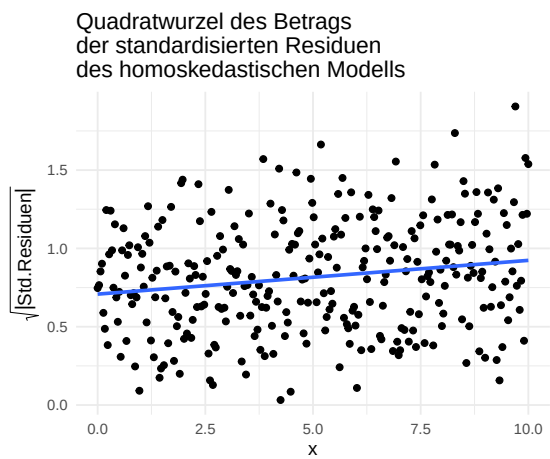
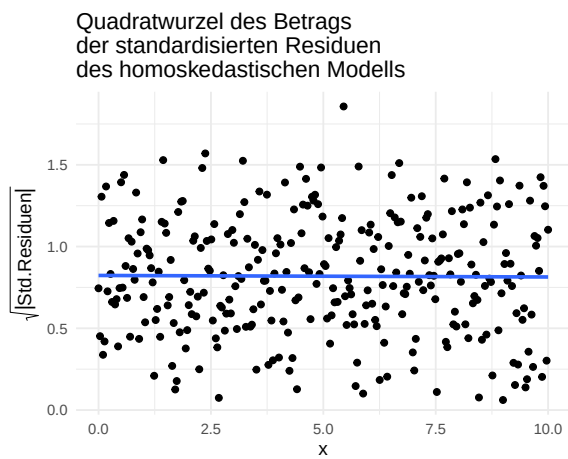
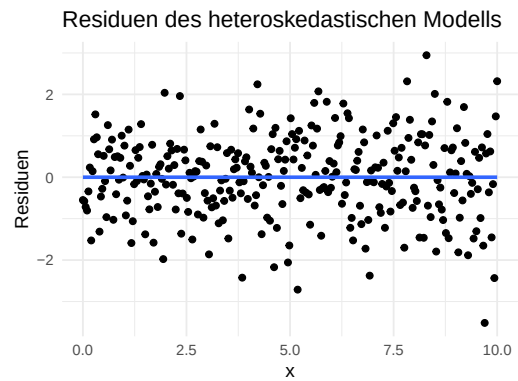
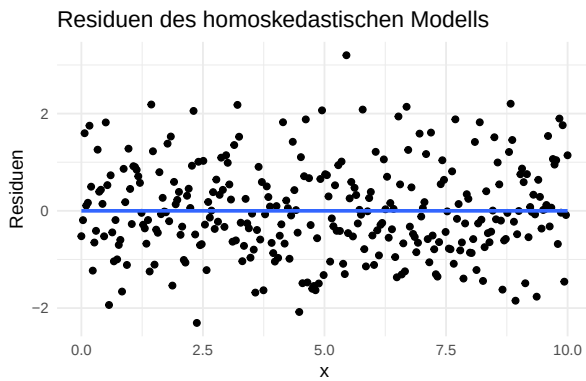
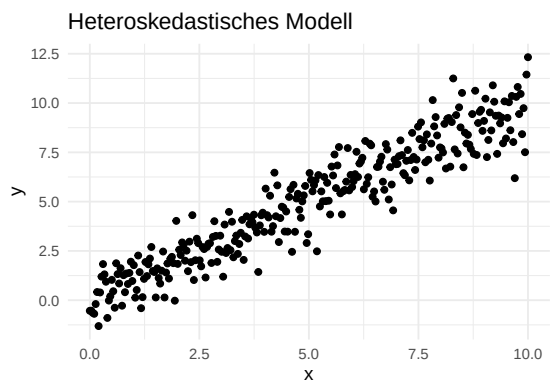
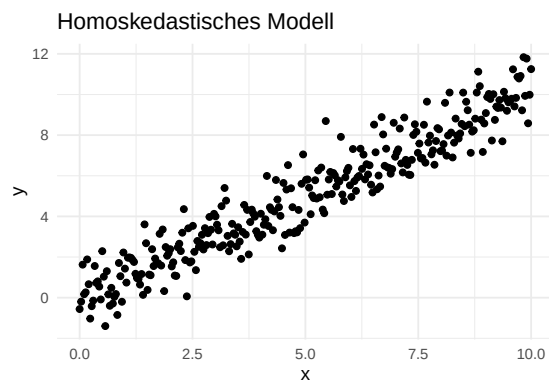
$$\hat{\sigma}^2 = \frac{1}{K-J} \sum_{k=1}^K \hat{\varepsilon}_k^2$$

geschätzt.

Die Motivation für standardisierte Residuen ist, dass, obwohl unser Modell Homoskedastizität mit einem i. i. d.-Fehlerterm mit fester Varianz $\varepsilon_k \sim \mathcal{N}(0, 1)$ annimmt, die Verteilung der Residuen ε_k nicht i. i. d. sein kann, da die Summe der Residuen immer genau null ist.

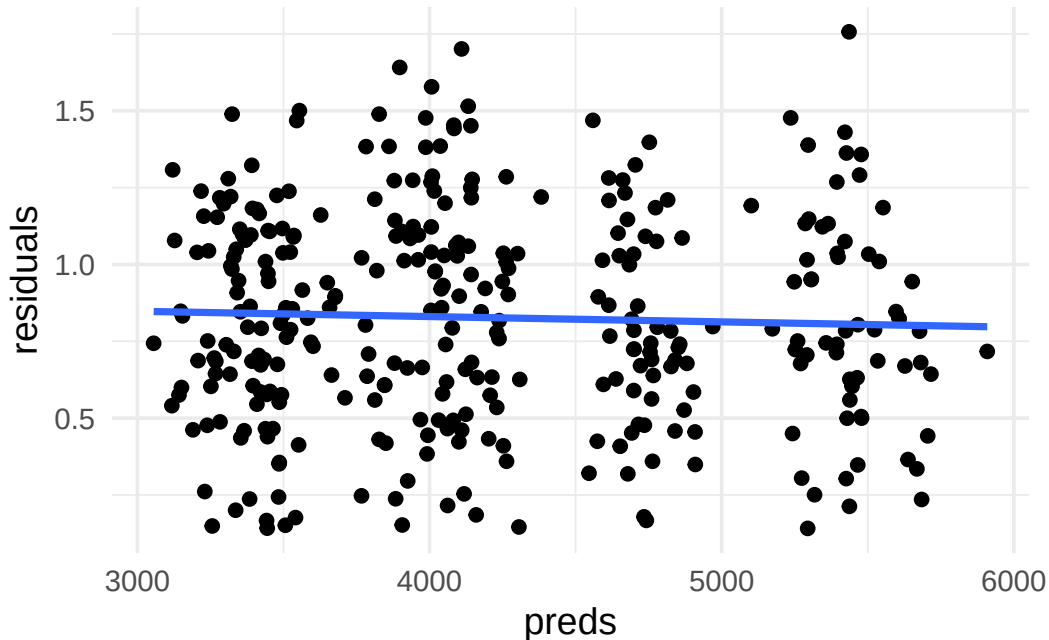
Einfach gesagt bedeutet das: Beim Darstellen der nicht standardisierten Residuen sind die Abweichungen schwieriger zu erkennen.

Folgendes Beispiel illustriert diesen Effekt:



Falls die Daten in der rechten Spalte ebenso homoskedastisch wären, müsste die Trendlinie in der letzten Grafik auch horizontal sein. Anhand der anderen beiden Plots lässt sich kaum erkennen, ob Heteroskedastie vorliegt.

Im Beispiel des Pinguin-Datensatzes sieht die Darstellung der Residuen wie folgt aus:



In den Residuen wird ein kleiner Abwärtstrend sichtbar, welcher allerdings so gering ist, dass für das gegebene Modell keine Heteroskedastie vermutet werden sollte.

Die obige Grafik wurde durch folgenden Code erzeugt:

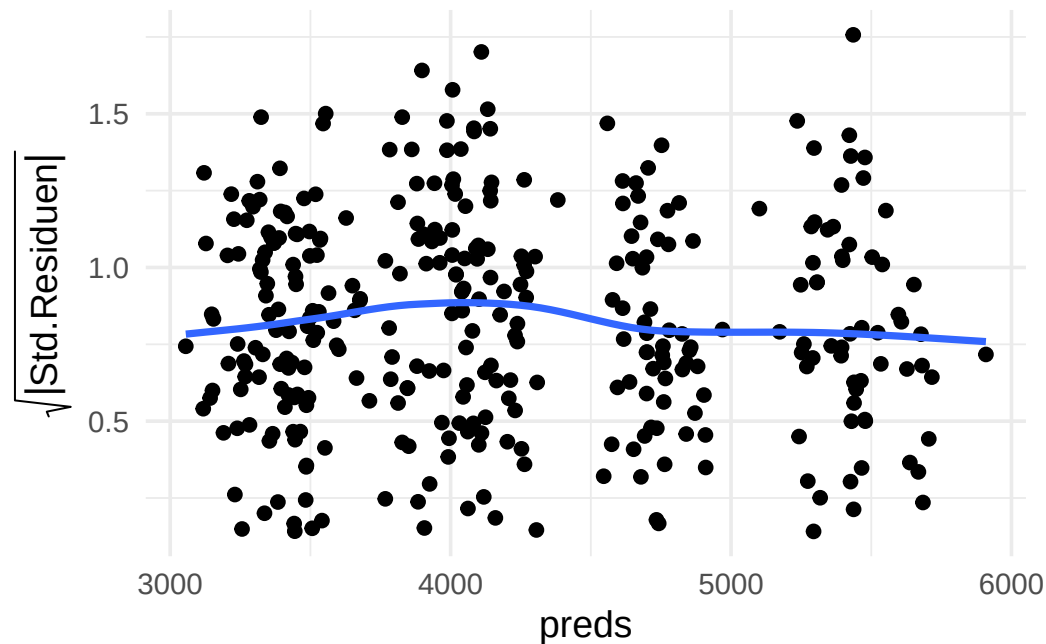
```
1 resid<-mlr_fit %>%
2   extract_fit_engine() %>%
3   rstudent()
4
5 tibble(
6   predictions = preds,
7   residuals = sqrt(abs(resid))
8 ) %>%
9   ggplot(aes(x=preds,y=residuals))+
10  geom_point()+
11  geom_smooth(method = "lm", formula = y~x, se = FALSE)+
12  theme_minimal(base_size=14)
```

Ähnlich wie bei der vorherigen Grafik wurde mithilfe von `ggplot` eine Punktwolke durch die Residuen und Vorhersagen erzeugt. Der Unterschied liegt hierbei in den Zeilen 3, 7 und 11.

- In Zeile 3 wurde nicht die `residuals()`-Funktion verwendet, sondern die `rstudent`. Diese erzeugt die standardisierten Residuen, wie sie im vorherigen Abschnitt beschrieben wurden.
- In Zeile 7 wird die Quadratwurzel des Betrags der standardisierten Residuen durch die Funktionen `sqrt()` und `abs()` berechnet.
- In Zeile 11 wird mithilfe der `geom_smooth`-Funktion eine Trendlinie hinzugefügt. Das Argument `method = "lm"` sorgt dafür, dass die Trendlinie linear ist. Durch `formula = y~x` wird die Formel der Trendlinie spezifiziert. Die Notation mit `y` und `x` bezieht sich hierbei auf die Variablen `y` und `x` welche in den Aesthetics in Zeile 9 übergeben wurden. Das Argument `se=FALSE` steuert, dass keine Konfidenzstreifen um die Trendlinien hinzugefügt werden.

2.3.3.3 Autokorrelation

Um zu überprüfen, ob die Residuen unabhängig voneinander sind, kann man ebenso als visuelles Hilfsmittel eine Punktwolke wie bei der Überprüfung der Homoskedastizitätsannahme erstellen.



In der obigen Grafik überprüfen wir allerdings nicht nur, ob die Trendlinie horizontal ist, sondern ob diese *linear* ist. Beim Erzeugen der Trendlinie wurde nämlich nicht wie zuvor `method =`

"lm" verwendet, sondern `method = "loess"`. "loess" steht in diesem Kontext für [locally estimated scatterplot smoothing](#). Wichtig hierbei ist lediglich zu wissen, dass ein LOESS-Modell auch nichtlineare Zusammenhänge zwischen der abhängigen und den unabhängigen Variablen abbilden kann.

Die Trendlinie in der obigen Grafik ist leicht nichtlinear, folgt allerdings keinem erkennbaren Trend, sodass wir hier eine leichte Korrelation der Residuen unterstellen können.

2.3.3.4 Normalität der Residuen

Die letzte Annahme des Gauss–Markov–Theorems lässt sich durch verschiedene visuelle Hilfsmittel überprüfen.

2.3.3.4.1 QQ-Plots

Häufig wird ein sogenannter *QQ-Plot* (Quantil-Quantil-Plot) verwendet, um zu überprüfen, ob die unterliegende Verteilung einer Normalverteilung entspricht. Hierbei werden den empirischen Quantilen (der unterliegenden Residuen) die theoretischen Quantile gegenübergestellt. Falls die Punkte hierbei sehr nah an der erzeugten Geraden liegen, so geht man davon aus, dass die Quantile der empirischen Verteilung gleich der Normalverteilung sind und somit die Residuen ebenso normalverteilt sind.

```
resid <- mlr_fit %>%  
  extract_fit_engine() %>%  
  residuals() %>%  
  tibble(residuals = .)  
  
resid %>% ggplot(aes(sample=residuals))+  
  geom_qq()+  
  geom_qq_line()+  
  theme_minimal(base_size=14)
```



Da die empirischen Quantile der Residuen relativ nah an der erzeugten Geraden liegen, kann man davon ausgehen, dass diese normalverteilt sind.

Um die obige Grafik zu erzeugen, werden wieder zuerst die Residuen in den Codezeilen 1-4 erzeugt. Hierbei werden die Residuen allerdings in einem Tibble bzw. Data Frame gespeichert, da der Rückgabewert der `residuals()`-Funktion den Datentyp `<dbl>` besitzt. Die `ggplot`-Funktion setzt allerdings voraus, dass die Daten als Data Frame oder Tibble übergeben werden. Das Argument `residuals = .` sorgt dafür, dass im Tibble eine neue Spalte `residuals` erzeugt wird, welche als Werte den Output der `residuals()`-Funktion übergeben bekommt.

Um dann den Q-Q-Plot zu erzeugen, muss man in den Aesthetics der `ggplot`-Funktion das Argument `sample` setzen (anstatt `x` und `y` wie gewohnt). Nachdem das Sample auf `residuals` gesetzt wurde, können dann mithilfe der `geom_qq()` und der `geom_qq_line()`-Funktion die Punkte und die Gerade erzeugt werden.

💡 Wiederholung

Um für eine gegebene Stichprobe $x_1, \dots, x_K$ die empirischen Quantile zu berechnen, wird die Stichprobe zuerst sortiert (Ordnungsstatistik). Die sortierte Stichprobe wird durch die Notation $x_{(1)}, \dots, x_{(K)}$ mit $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(K)}$ dargestellt. Für $p \in (0,1)$ ist dann das empirische p -Quantil definiert als

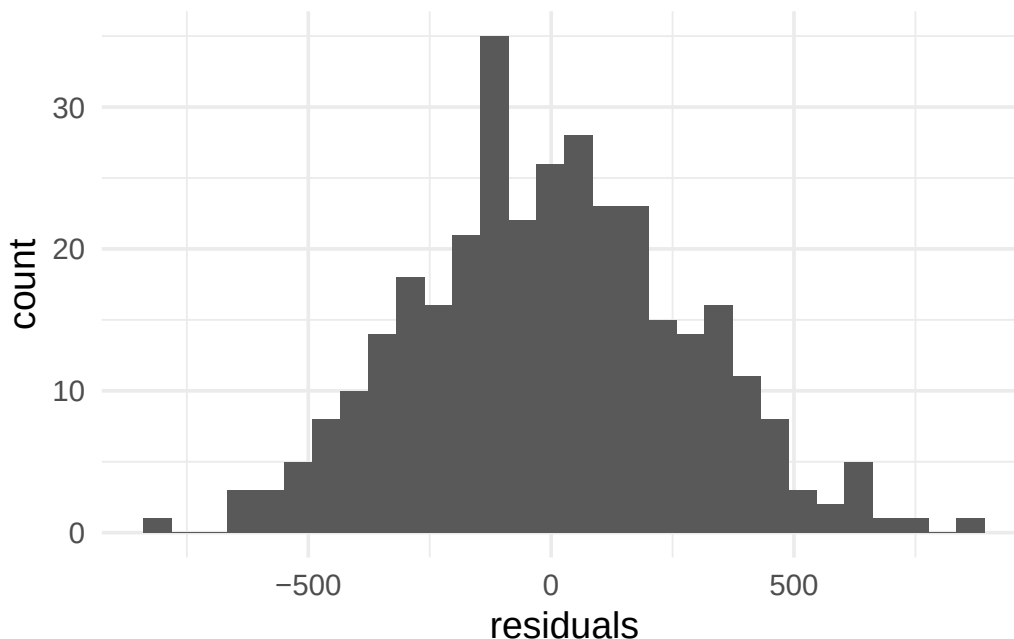
$$\tilde{x}_p = \begin{cases} x_{(\lfloor K \cdot p \rfloor + 1)} & \text{für } K \cdot p \notin \mathbb{Z} \\ \frac{x_{(K \cdot p)} + x_{(K \cdot p + 1)}}{2} & \text{für } K \cdot p \in \mathbb{Z} \end{cases}$$

2.3.3.4.2 Vergleich der Histogramme mit der Dichte einer Normalverteilung

Neben den Quantilen bietet sich auch die Betrachtung des Histogramms der Residuen an:

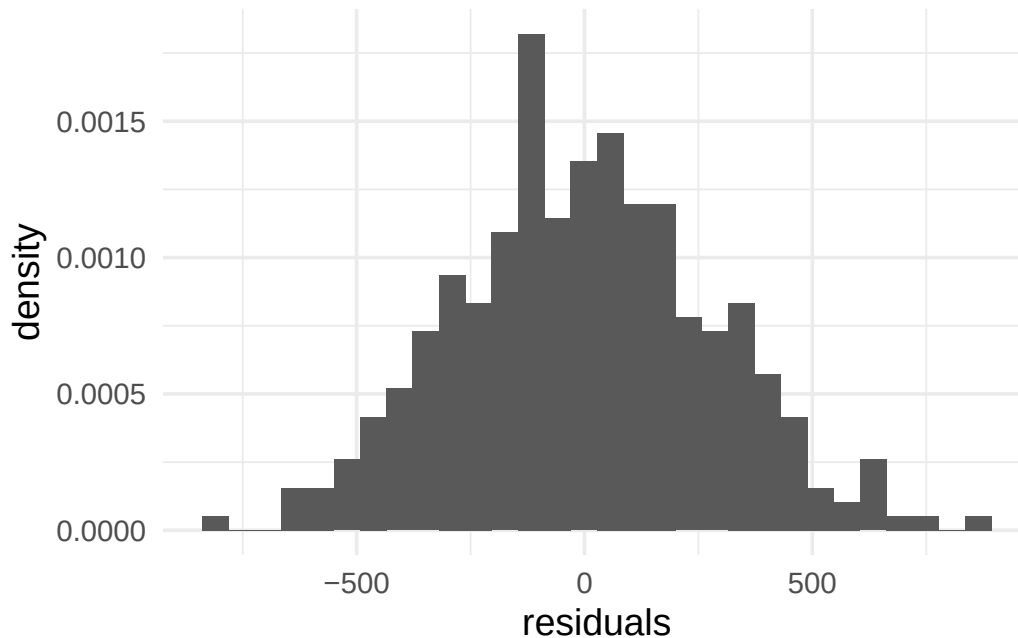
```
resid %>%
  ggplot(aes(x=residuals))+
  geom_histogram()+
  theme_minimal(base_size = 14)
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



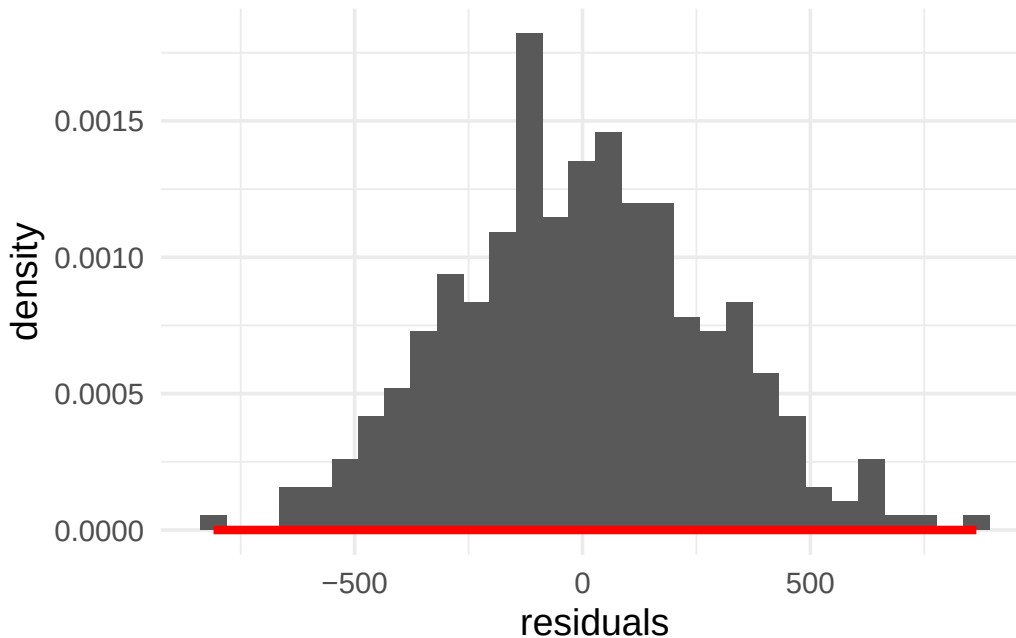
Um das Histogramm, welches durch die `geom_histogram`-Funktion erzeugt wurde, mit der Dichte einer Normalverteilung vergleichen zu können, müssen wir zuerst die y-Achse so anpassen, dass diese nicht mehr die absoluten Häufigkeiten, sondern die relativen Häufigkeiten darstellt. Durch das Setzen von `y=after_stat(density)` in den Aesthetics der `geom_histogram`-Ebene wird dies erreicht.

```
resid %>%
  ggplot(aes(x=residuals))+
  geom_histogram(aes(y= after_stat(density)))+
  theme_minimal(base_size = 14)
```



Nach dem Festlegen der Darstellung der Häufigkeit kann als Nächstes die Dichte einer Normalverteilung hinzugefügt werden. Die `stat_function()`-Funktion erleichtert das Hinzufügen der Dichte einer Normalverteilung um ein vielfaches, da hier vorerst nur das Argument `fun=dnorm` übergeben werden muss.

```
resid %>%
  ggplot(aes(x=residuals))+
  geom_histogram(aes(y= after_stat(density)))+
  stat_function(fun = dnorm, color = "red", linewidth = 1.5)+
  theme_minimal(base_size = 14)
```



Problematisch ist allerdings, dass die Dichte einer Standardnormalverteilung ($\mathcal{N}(0, 1)$) abgebildet wird, für welche gilt $f(x) \approx 0$, $|x| > 4$. Die rote Linie in der obigen Grafik stellt die Dichte der Standardnormalverteilung dar. Für das Histogramm gilt allerdings, dass selbst für $|\text{residuals}| > 500$ noch Observationen erkennbar sind. Es müssen also entweder die Werte der Residuen skaliert werden, oder die Parameter der Normalverteilung angepasst werden. Um die Parameter der Normalverteilung anzupassen, verwendet man das `args` Argument. Die Funktion `dnorm` besitzt die Argumente `mean` und `sd`, was für den Mittelwert und die Standardabweichung der Normalverteilung steht. Indem man das Argument `mean` auf den Mittelwert der Residuen und `sd` auf die Standardabweichung der Residuen setzt, sollte sich also die Dichte einer Normalverteilung erzeugen lassen, welche wir mit dem Histogramm vergleichen können!

Um die beiden Argumente `mean` und `sd` zu übergeben, kann man diese zuerst separat berechnen und dann als Liste `list(mean = ..., sd = ...)` übergeben:

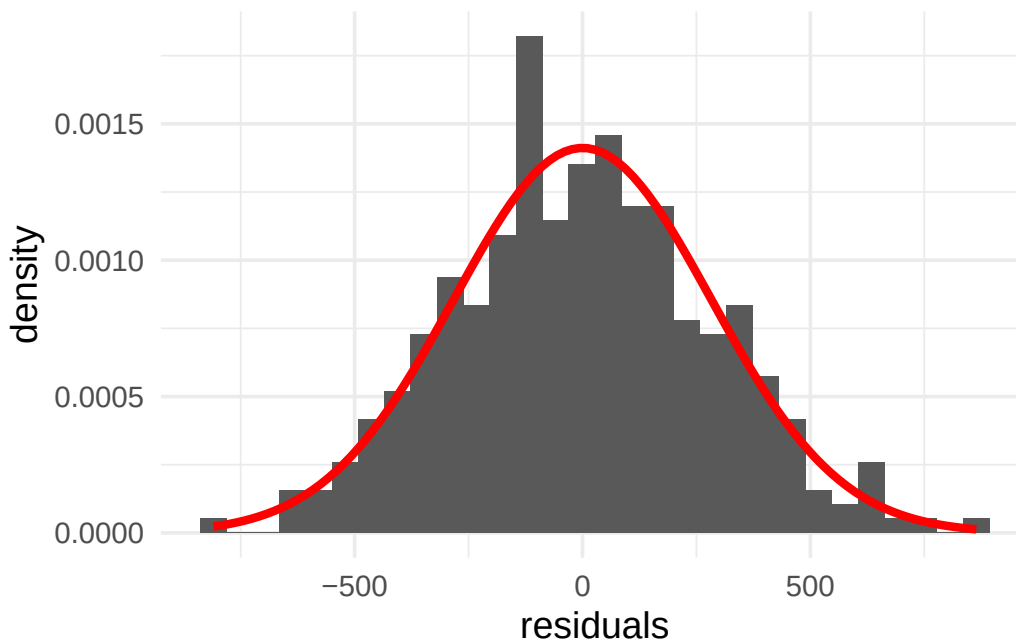
```
mean_res <- mlr_fit %>%
  extract_fit_engine() %>%
  residuals() %>%
  mean()

sd_res <- mlr_fit %>%
  extract_fit_engine() %>%
  residuals() %>%
  sd()
```

```

resid %>%
  ggplot(aes(x=residuals))+
  geom_histogram(aes(y= after_stat(density)))+
  stat_function(fun = dnorm,
               color = "red",
               linewidth = 1.5,
               args = list(mean = mean_res,
                           sd = sd_res))+
  theme_minimal(base_size = 14)

```



Wie an der obigen Grafik zu erkennen ist, passt die dargestellte Dichte gut zu dem erzeugtem Histogramm. Wir können also auch daraus schließen, dass die Residuen vermutlich normalverteilt sind.

2.3.3.5 Multikollinearität

Eine weitere Bedingung, welche zwar nicht durch das Gauss–Markov Theorem vorausgesetzt wird, aber trotzdem wichtig ist, ist die der nichtvorhandenen *Multikollinearität* der erklärenden Variablen.

Multikollinearität ist wie folgt definiert:

Falls für einen Datensatz mit erklärenden Variablen $X_1, \dots, X_J \in \mathbb{R}^K$ eine Teilmenge $X_i, X_{i+1}, \dots, X_j$ mit $1 \leq i < j+1 \leq J$ existiert, sodass

$$X_i = \sum_{n=i}^j \alpha_n X_{n+1}, \quad \alpha_1, \dots, \alpha_n \in \mathbb{R} \setminus \{0\} \quad (2.6)$$

dann sind die Variablen $X_i, X_{i+1}, \dots, X_j$ multikollinear. Die rechte Seite der Gleichung 2.6 wird auch Linearkombination der Vektoren $X_{i+1}, \dots, X_{j+1}$ genannt. Die Vektoren X_i und $X_{i+1}, \dots, X_{j+1}$ sind somit linear abhängig. Falls keine Konstanten $\alpha_1, \dots, \alpha_n \in \mathbb{R} \setminus \{0\}$ existieren, sodass in Gleichung 2.6 Gleichheit gilt, linearer Unabhängigkeit der Vektoren.

Beispiel 2.2. Seien

$$X_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \quad X_2 = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}.$$

1. Die Spalten von X_1 sind paarweise linear unabhängig und linear unabhängig.
2. Die Spalten von X_2 sind paarweise linear unabhängig, aber nicht linear unabhängig, da

$$x_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} = -1 \cdot \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

Der erste Spaltenvektor lässt sich also als Linearkombination der anderen beiden Spaltenvektoren darstellen.

In den meisten Fällen sind die Variablen in einem Datensatz allerdings nicht multikollinear wie in Gleichung 2.6, sondern nur approximativ kollinear, d.h.

$$X_i \approx \sum_{n=i}^j \alpha_n X_{n+1}, \quad \alpha_1, \dots, \alpha_n \in \mathbb{R} \setminus \{0\}.$$

Um zu überprüfen, ob die unabhängigen Variablen eines Datensatzes linear abhängig sind, wird nicht wie im obigen Beispiel versucht eine passende Linearkombination zu finden, sondern es wird der *Spaltenrang* der Feature-Matrix bestimmt.

Der Spaltenrang einer Matrix X ist definiert als die maximale Anzahl an linear unabhängigen Spaltenvektoren in X .

Beispiel 2.3.

1. Die Matrix X_1 aus Beispiel 2.2 hat Spaltenrang 3, da die drei Spalten linear unabhängig sind.
2. Die Matrix X_2 aus Beispiel 2.2 hat Spaltenrang 2, da nur 2 der drei Spalten linear unabhängig sind.

! Wichtig

Eine Quadratische Matrix $X \in \mathbb{R}^{n \times n}$ ist invertierbar, falls sie vollen Rang hat, i.e., alle Spalten sind linear unabhängig. Falls X vollen Rang besitzt, dann existiert eine Matrix $X^{-1} \in \mathbb{R}^{n \times n}$ (die Inverse (Matrix) von X) sodass $X \cdot X^{-1} = I_n$ mit

$$I_n = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix} \quad (2.7)$$

gilt.

2.3.3.6 Das Problem mit Multikollinearität

Warum ist Multikollinearität nun ein Problem? Kurz zusammengefasst:

Multikollinearität führt dazu, dass die Regressionsparameter nicht mehr korrekt interpretiert werden können und im schlimmsten Fall, dass das MLR Modell als ganzes nicht mehr interpretierbar ist.

Da in Gleichung 2.4 die Matrix $(X'X)$ invertiert wird um die optimalen Parameter $b_0, \dots, b_J$ schätzen zu können, muss diese die Voraussetzung der Invertierbarkeit erfüllen. Wie im vorherigen Abschnitt beschrieben ist die Invertierbarkeit einer quadratischen Matrix allerdings nur dann möglich, falls diese vollen Spaltenrang hat, also die Spalten alle unabhängig von einander sind. Falls die Spalten abhängig voneinander und somit (multi)kollinear sind, können die Parameter nicht mehr geschätzt werden. Auch falls die Spalten approximativ linear abhängig sind, führt das ebenfalls zu Problem bei der Berechnung.

Das folgende Beispiel illustriert diese Effekte.

Beispiel 2.4. Der Datensatz `data_kantine` bildet die Anzahl der Besucher in einem Klub (`num_vis`) an 20 Abenden, den durchschnittlichen Preis für ein Getränk (`avg_drink_price`), die Wochentage sowie die durchschnittlichen Ausgaben pro Gast an den entsprechenden Abenden ab.

num_vis	avg_drink_price	avg_expense	week_day
289	4.84	22.7	Mittwoch
295	5.00	23.8	Samstag
331	4.43	20.5	Sonntag
301	4.81	25.4	Samstag
303	4.81	24.4	Samstag
334	4.24	19.8	Sonntag
309	5.12	24.1	Sonntag
275	5.29	24.7	Mittwoch
286	4.86	24.2	Mittwoch
291	5.40	25.5	Mittwoch
324	4.87	23.7	Sonntag
307	4.86	23.4	Samstag
308	5.14	24.5	Samstag
302	5.20	26.2	Samstag
289	5.32	25.1	Mittwoch
336	4.81	24.8	Sonntag
310	5.04	22.6	Sonntag
261	5.37	26.1	Mittwoch
314	4.78	23.2	Sonntag
291	4.99	24.2	Mittwoch

Die metrischen Spalten des Datensatzes sind linear unabhängig. Die Korrelationsmatrix auf basis des Pearson Korrelationskoeffizient ergibt allerdings

```
# A tibble: 3 x 4
  term          num_vis avg_drink_price avg_expense
  <chr>          <dbl>          <dbl>          <dbl>
1 num_vis        NA          -0.717          -0.596
2 avg_drink_price -0.717          NA             0.817
3 avg_expense    -0.596          0.817          NA
```

D.h., die beiden Spalten `avg_expense` und `avg_drink_price` korrelieren stark ($\rho > 0.9$)!

Verwendet man nun die numerischen Variablen, um die Anzahl der Clubbesucher anhand eines MLR Modells zu modellieren, so erhält man

```
...

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   535.4580    55.1249   9.714 2.37e-08 ***
avg_drink_price -45.1042    19.1417  -2.356  0.0307 *
avg_expense    -0.3753     3.4631  -0.108  0.9150
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
...
```

Die beiden Variablen `avg_drink_price` und `avg_expense` sind im obigen Modell beide nicht hochsignifikant ($p > 0.01$).

Falls aber im Gegensatz dazu jeweils entweder nur die Variable `avg_drink_price` oder `avg_expense` verwendet um die Besucherzahl zu modellieren, ergeben sich folgende Statistiken:

1. Modell 1:

```
...

              Estimate Std. Error t value Pr(>|t|)
(Intercept)   534.87     53.33  10.029 8.54e-09 ***
avg_drink_price -46.80     10.74  -4.359 0.000378 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
...
```


2. Modell 2:

...

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  471.381      53.673   8.782 6.33e-08 ***
avg_expense  -7.040       2.237  -3.148 0.00556 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

...

Da die beiden Variablen stark korrelieren, sind ihre Effekte im Modell, welches beide Variablen enthält, weder signifikant noch interpretierbar. Allerdings sind die Parameter getrennt betrachtet in Modell 1 und Modell 2 (hoch)signifikant, was ebenso dafür spricht, dass die Feature stark korrelieren.

Noch extremer ist der Effekt, wenn man die Variable `weekday` in eine Dummy Variable umwandelt, welche jede Ausprägungsmöglichkeit abbildet:

```
data_kantine_dummy <- data_kantine %>%
  mutate( week_day_wednesday = if_else(week_day == "Mittwoch", 1,0),
           week_day_saturday = if_else(week_day == "Samstag", 1,0),
           week_day_sunday = if_else(week_day == "Sonntag", 1,0),
           week_day = NULL
  )
```

Im obigen Datensatz wurde die Variable `week_day` durch Dummy Variablen ersetzt, welche jeden Wochentag im Datensatz abbilden. Wenn dann die Parameter für das MLR Modell geschätzt werden, welches den Einfluss der Wochentage auf die Besucherzahl modelliert, dann ergibt sich der Auszug aus folgender Zusammenfassung:

...

```
Coefficients: (1 not defined because of singularities)
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    322.571      3.742  86.213 < 2e-16 ***
week_day_wednesday -39.429      5.291  -7.452 9.47e-07 ***
week_day_saturday -19.905      5.507  -3.614 0.00214 **
week_day_sunday      NA          NA      NA      NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

...

Da die Dummy Variablen perfekt mit dem Intercept korrelieren, kann der Koeffizient für `week_day_sunday` nicht berechnet werden.

2.3.3.6.1 Variance Inflation Factor

Neben einer Korrelationsmatrix kann auch der Variance Inflation Factor definiert als

$$\text{VIF}_j = \frac{1}{1 - R_j^2} \quad (2.8)$$

verwendet werden um zu prüfen, ob Multikolarität in einem Modell vorhanden ist. Hierbei steht R_j^2 für das Bestimmtheitsmaß der Regression des Regressors X_j auf alle anderen Regressoren. Als Daumenregel wird häufig genannt, dass falls $\text{VIF}_j \geq 5$, der Regressor X_j aus dem Modell entfernt werden sollte.

Um den VIF für alle unabhängigen Variablen zu berechnen, kann man die `vif()`-Funktion der `{car}` Library verwenden. Ähnlich wie beim Erzeugen der Modellzusammenfassung muss man zuerst die `extract_fit_engine()`-Funktion anwenden, um das Modell an die `vif()`-Funktion übergeben zu können.

```
mlr_fit %>%
  extract_fit_engine() %>%
  car::vif()
```

	GVIF	Df	GVIF ^{1/(2*Df)}
species	71.200285	2	2.904828
island	3.762055	2	1.392696
bill_length_mm	6.117069	1	2.473271
bill_depth_mm	6.274341	1	2.504864
flipper_length_mm	7.775036	1	2.788375
sex	2.343434	1	1.530828
year	1.172956	1	1.083031

Im obigen Output wird neben dem Generalized Variance Inflation Factor (GVIF) auch eine standardisierte Version berechnet. Die generalisierte Version des VIF wird für alle metrischen Variablen durch die Formel in Gleichung 2.8 berechnet. Lediglich die Berechnung des VIF für die nominalen Feature unterscheidet sich. Die Berechnung für die nominalen Feature würde allerdings den Rahmen der Vorlesung sprengen, weshalb diese hier nicht weiter erläutert wird.

Wichtig ist allerdings die Interpretation der Werte (unabhängig vom Datentyp): Die standardisierte Version des VIF sollte für nominale Features betrachtet werden, da dieser die Anzahl der Dummyspalten einbezieht. Falls $\text{GVIF}^{(1/(2 \cdot \text{Df}))} \geq \sqrt{5} \approx 2.236$, dann liegt eventuell bei einem nominalen Feature Multikolarität vor.

Für das obige MLR Modell bedeutet das, dass für die Variablen `species`, `bill_length_mm`, `bill_depth_mm` und `flipper_length_mm` Multikollinearität vorliegen könnte. Um den Effekt im obigen Modell zu mitigieren, wäre eine Möglichkeit, neue Modelle zu definieren, welche die einzelnen Variablen ausschließen und für diese neuen Modelle die VIF zu berechnen.

2.3.3.7 Weitere wichtige Prüfkriterien

Neben den bisher erläuterten Prüfkriterien sollten wir uns im Zuge unserer Modellanalyse auch mit der Datenqualität auseinandersetzen. Hierbei sind vor allem folgenden Konzepte nicht zu unterschätzen:

- Ausreißeranalyse
- Einflussreiche Observationen

2.3.3.7.1 Ausreißeranalyse

Ob eine Observation ein Ausreißer ist, hängt stark von der Definition eines Ausreißers ab. In der Literatur werden verschiedene Definitionen verwendet, wobei eine gängige die Definition von [John Tukey](#) stammt:

Definition

Sei $q_{(0.25)}$ das 25% Quantil und $q_{(0.75)}$ das 75% Quantil einer Variable, dann ist eine gegebene Observation ein Ausreißer, falls sie außerhalb des Intervalls

$$[q_{(0.25)} - k(q_{(0.75)} - q_{(0.25)}), q_{(0.75)} + k(q_{(0.75)} - q_{(0.25)})], \quad k \geq 0$$

liegt.

k ist hierbei frei wählbar, allerdings wird häufig $k = 1.5$ verwendet.

Eine weitere Möglichkeit der Ausreißerbestimmung verwendet die *Cook's Distance*, die wir im folgenden Abschnitt betrachten.

2.3.3.7.2 Einflussreiche Observationen

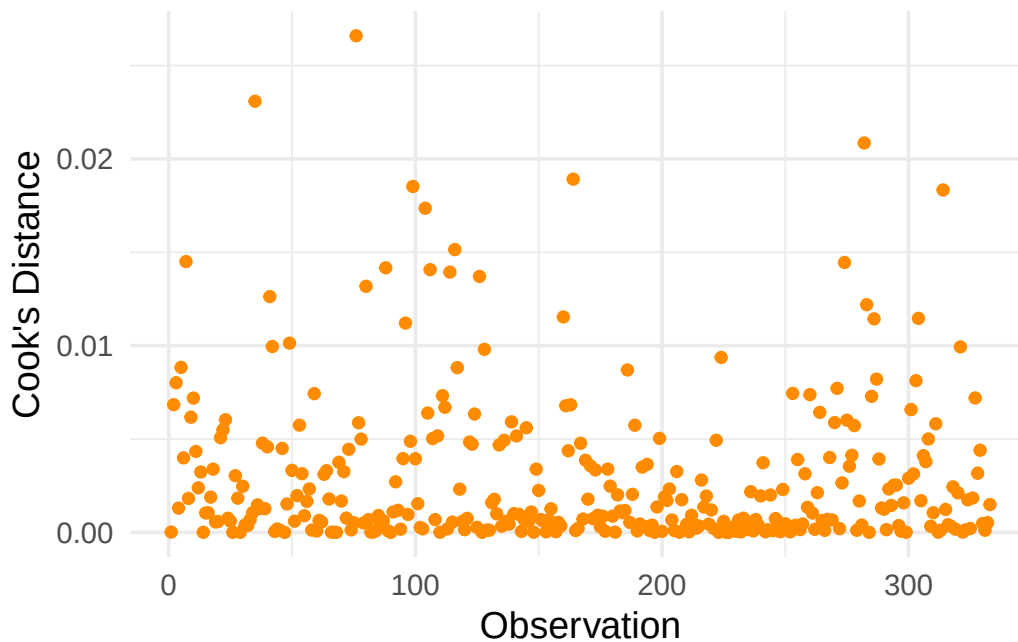
Observationen, welche Ergebnisse der Parameterschätzung stark beeinflussen, werden einflussreiche Observationen genannt. Um zu bestimmen, ob eine Observation einflussreich ist, verwenden wir die Cook's Distance, definiert als

$$D_i = \frac{(\hat{b} - \hat{b}_{(i)})'(X'X)(\hat{b} - \hat{b}_{(i)})}{(J+1)\hat{\sigma}^2},$$

wobei $\hat{b}_{(i)}$ die geschätzten Parameter ohne die i -te Beobachtung sind und $\hat{\sigma}$ der RMSE des Modells (siehe Abschnitt 2.3.4.3). Falls $D_i > F_{J;K-J-1;0.5}$, dann ist die Observation i ein Ausreißer.

Um den Cook's Distance in R zu berechnen, verwenden wir die `stats::cooks.distance()`-Funktion.

```
cooks_d <- mlr_fit %>%  
  extract_fit_engine() %>%  
  cooks.distance()  
  
tibble(  
  cooks_d = cooks_d,  
  obs = 1:length(cooks_d)  
) %>%  
  ggplot(aes(x=obs,y=cooks_d))+  
  geom_point(size = 1.5, color = "darkorange")+  
  labs(  
    x = "Observation",  
    y = "Cook's Distance"  
  )+  
  theme_minimal(base_size = 14)
```



2.3.4 Gütemaße

Bei der multiplen linearen Regression können wir analog zur einfachen linearen Regression anhand verschiedener Gütemaße quantifizieren, wie gut das Modell den Zusammenhang zwischen den unabhängigen und der abhängigen Variable beschreibt.

2.3.4.1 Korrigierter R^2

Neben dem Gütemaß R^2 (vgl. Gleichung 2.2) wird bei MLR-Modellen auch häufig die korrigierte Version

$$R_{\text{kor}}^2 = R^2 - \frac{J \cdot (1 - R^2)}{K - J - 1} \quad (2.9)$$

verwendet. Hierbei ist allerdings zu beachten, dass R_{kor}^2 nicht mehr direkt interpretiert werden kann und das Gütemaß auch keine untere Grenze (kann auch negative Werte annehmen) besitzt.

2.3.4.2 Akaike Information Criteria

Das Akaike-Informationskriterium (AIC) wird häufig verwendet, um Modelle untereinander zu vergleichen, da der Bildbereich dieses Gütemaßes unbeschränkt ist (kann jede Zahl $x \in \mathbb{R}$ annehmen).

$$\text{AIC} = \log \left(\frac{\sum_{k=1}^K (y_k - \hat{y}_k)^2}{K} \right) + \frac{2J}{K} \quad (2.10)$$

In R können wir den AIC mithilfe der `stats::AIC`-Funktion berechnen:

```
mlr_fit %>%  
  extract_fit_engine() %>%  
  stats::AIC()
```

```
[1] 4724.97
```

2.3.4.3 Loss-Funktionen

Neben den bisher eingeführten Gütemaßen werden bei Regressionsmodellen auch oft sogenannte *Loss*-Funktionen berechnet. Diese messen, wie weit die tatsächlichen Werte der abhängigen Variable von der Schätzung entfernt sind. Eine gängige Loss-Funktion im Regressionskontext ist der Root Mean Squared Error (RMSE). Dieser ist definiert als

$$\text{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{K} \sum_{k=1}^K (y_k - \hat{y}_k)^2}. \quad (2.11)$$

Der RMSE wird häufig verwendet, da dieser die euklidische Distanz zwischen zwei Punkten berechnet und deshalb sehr intuitiv interpretierbar ist.

Wir können den RMSE mithilfe der `yardstick::mse()`-Funktion berechnen. Hierfür müssen wir zuerst die Vorhersagen generieren und diese dann zusammen mit den tatsächlichen Werten übergeben:

```
1 mlr_fit %>%  
2   parsnip::augment(data_penguin) %>%  
3   yardstick::rmse(truth = .pred, estimate = body_mass_g)  
  
# A tibble: 1 x 3  
  .metric .estimator .estimate  
  <chr>   <chr>      <dbl>  
1 rmse    standard      282.
```

In der zweiten Zeile berechnen wir mit der `parsnip::augment()`-Funktion die durch das MLR-Modell geschätzten Werte. Der Rückgabewert dieser Funktion ist ein neuer Datensatz, welcher neben dem Datensatz `data_penguin` noch die Spalten `.pred` und `.resid` enthält. In der Spalte `.pred` sind die durch das Modell geschätzten Vorhersagen enthalten und in der Spalte `.resid` die Residuen. Diesen neuen Datensatz übergeben wir dann der `yardstick::rmse`-Funktion mit den Argumenten `truth = .pred` und `estimate = body_mass_g`.

Den Wert `.estimate = 282` können wir wie folgt interpretieren:

Im Schnitt erwarten wir bei der Schätzung des Gewichts eines Pinguins eine Abweichung von 282 g vom wahren Wert.

2.4 Übungsaufgaben

2.4.1 Explorative Datenanalyse

Aufgabe 2.1. Finde heraus, wie groß die relativen Anteile der jeweiligen Pinguinspezies an der Gesamtanzahl der Pinguine im Datensatz sind.

Aufgabe 2.2. In Abschnitt 2.1.1 haben wir einen Boxplot für die Schnabellänge der verschiedenen Pinguinspezies erstellt. Erstelle jetzt einen Dichteplot, in welchem die empirische Dichte der Schnabellängen der jeweiligen Pinguinspezies dargestellt wird.

2.4.2 Einfache lineare Regression

Aufgabe 2.3. Entferne alle NA-Werte und die Variable `year` im Datensatz. Erstelle anschließend einen neuen Teildatensatz `data_penguin_adelie`, welcher nur Pinguine der `Adelie` Spezies enthält.

Aufgabe 2.4.

1. Schätze für den neu erstellten Datensatz aus Aufgabe 2.3 ein einfaches lineares Modell in R, wobei die abhängige Variable durch `flipper_length_mm` und die unabhängige Variable durch `body_mass_g` gegeben ist.
2. Erzeuge für das in Aufgabe 1. erstellte lineare Modell eine Zusammenfassung in R. Ist die unabhängige Variable zum Signifikanzniveau $\alpha = 0.01$ statistisch signifikant?
3. Angenommen, der Wert der t -Statistik für die Variable `body_mass_g` beträgt nicht 6.30, sondern 2.88. Bestimme das kleinste Signifikanzniveau, zu welchem die Variable signifikant ist. Verwende hierfür folgende Tabelle, welche Quantile der Standardnormalverteilung enthält:
4. Angenommen, ein Pinguin wiegt 16000 g, wie groß ist die erwartete Flossenlänge nach dem in Teilaufgabe 1. geschätzten einfachen linearen Modell?

2.4.3 Kennzahlen

Aufgabe 2.5. In Abschnitt 2.3.4.2 wurde das AIC eingeführt. Beurteile anhand Gleichung 2.10, ob ein hoher oder niedriger AIC-Wert für ein Modell spricht, welches den Zusammenhang zwischen den Variablen gut erklärt.

Aufgabe 2.6. Beurteile den Wahrheitsgehalt der folgenden Aussage und begründe deine Entscheidung:

	0	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
2.0	0.9772	0.9778	0.9783	0.9788	0.9793	0.9798	0.9803	0.9808	0.9812	0.9817
2.1	0.9821	0.9826	0.9830	0.9834	0.9838	0.9842	0.9846	0.9850	0.9854	0.9857
2.2	0.9861	0.9864	0.9868	0.9871	0.9875	0.9878	0.9881	0.9884	0.9887	0.9890
2.3	0.9893	0.9896	0.9898	0.9901	0.9904	0.9906	0.9909	0.9911	0.9913	0.9916
2.4	0.9918	0.9920	0.9922	0.9925	0.9927	0.9929	0.9931	0.9932	0.9934	0.9936
2.5	0.9938	0.9940	0.9941	0.9943	0.9945	0.9946	0.9948	0.9949	0.9951	0.9952
2.6	0.9953	0.9955	0.9956	0.9957	0.9959	0.9960	0.9961	0.9962	0.9963	0.9964
2.7	0.9965	0.9966	0.9967	0.9968	0.9969	0.9970	0.9971	0.9972	0.9973	0.9974
2.8	0.9974	0.9975	0.9976	0.9977	0.9977	0.9978	0.9979	0.9979	0.9980	0.9981
2.9	0.9981	0.9982	0.9982	0.9983	0.9984	0.9984	0.9985	0.9985	0.9986	0.9986

„Jeder Ausreißer ist eine Einflussreiche Observation.“

Aufgabe 2.7. In Abschnitt 2.3.3.7.2 haben wir Cook’s Distance als Methode zur Erkennung von einflussreichen Observationen eingeführt.

Berechne für das Beispiel in Abschnitt 2.3.3.7.2 den Wert $F_{J;K-J-1;0.5}$ und entscheide, ob im Datensatz, welcher für die Berechnungen verwendet wurde, einflussreiche Variablen sind.

Hinweis: Um ein Quantil der F-Verteilung zu berechnen, können wir die `qf()`-Funktion verwenden.

2.4.4 Multiple Lineare Regression

Für die folgenden Aufgaben verwenden wir den `hotel_rates`-Datensatz der `{modeldata}`-Library¹. Die in `{modeldata}` enthaltene Version des Datensatzes enthält die durchschnittlichen Zimmerpreise für Hotels in Lissabon, Portugal, aus den Jahren 2016–2017. Das Ziel dieser Aufgabe ist es, die Variable `avg_price_per_room` zu modellieren, die den durchschnittlichen Preis pro Nacht in einem Hotel beschreibt.

Aufgabe 2.8. Beschreibe, wie im folgenden Code-Snippet die Daten aufbereitet werden. Gehe dabei auf die einzelnen Funktionen und deren Rückgabewert ein.

```

1 hotel_rates_filtered <- hotel_rates %>%
2   mutate(
3     arrival_month = factor(month(arrival_date)),
4     num_guests = adults+children,
5     is_repeated_guest = factor(is_repeated_guest),
6   ) %>%

```

¹Ggf. muss mithilfe des `install.packages("modeldata")`-Befehls die Library erst noch installiert werden.


```
7   select(num_guests,total_of_special_requests,  
8         required_car_parking_spaces,arrival_month,  
9         is_repeated_guest,lead_time,stays_in_week_nights,  
10        avg_price_per_room)
```

Aufgabe 2.9. Erstelle mit R ein MLR-Modell, welches als abhängige Variable `avg_price_per_room` enthält und als unabhängige Variablen alle anderen Variablen im Datensatz `hotel_rates_filtered`.

Aufgabe 2.10.

1. Erzeuge für das Modell aus Aufgabe 2.9 eine Modellzusammenfassung. Gibt es Variablen im Modell, welche **nicht** statistisch signifikant sind?
2. Erweitere die Modellzusammenfassung durch eine Version, welche die standardisierten Koeffizienten enthält. Welche Variable hat den größten Einfluss auf die abhängige Variable `avg_price_per_room`?
3. Führe mithilfe der Modellzusammenfassung einen F -Test durch und evaluiere auf Basis dieses Tests die Modellgüte zum Signifikanzniveau $\alpha = 0.05$.

Aufgabe 2.11. Überprüfe das Modell aus Aufgabe 2.9 im Hinblick auf

1. Homoskedastizität.
2. Autokorrelation.
3. Die Annahme der Normalverteilung der Residuen.

3 Weiterführende Konzepte des Statistischen Lernens

In diesem Kapitel werden wir uns mit weiterführenden Konzepten des statistischen Lernens auseinandersetzen. Diese umfassen das Aufteilen der Daten in Trainings- und Testdaten, Cross-Validation (Kreuzvalidierung), die Imputation von fehlenden Werten sowie die Merkmalsselektion.

3.1 Training und Test Split

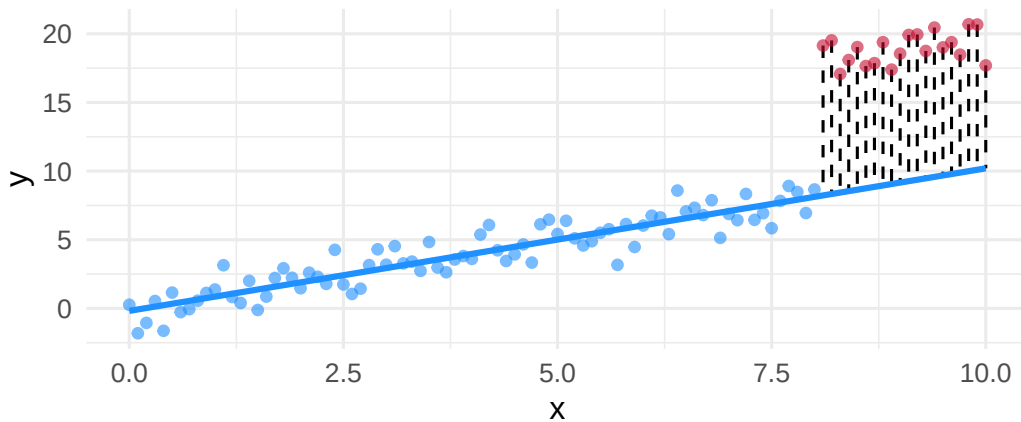
Die Parameter der linearen Regressionsmodelle aus Kapitel 2 haben wir unter Verwendung aller Daten geschätzt. Falls wir allerdings erwarten, dass neue Daten zu den Datensätzen hinzugefügt werden, ist das nicht die beste Idee, da wir nicht wissen, wie gut unser Modell auf zuvor ungesehenen Daten funktioniert. Eine gängige Praxis ist deshalb, die Daten in ein Trainings- und ein Testset aufzuteilen. Mit den Trainingsdaten werden die Parameter des Modells geschätzt und mit den Testdaten wird die sogenannte Out-of-Sample (OOS) Performance gemessen.

Beim Aufteilen der Daten sollte beachtet werden, dass das Verhältnis zwischen den beiden Teilen sinnvoll und die Aufteilung zufällig gewählt wird.

Beispiel 3.1. In einem gegebenen Datensatz sind 101 Observationen $(x, y) \in \mathbb{R}$. Nun werden die letzten 20% dieser Daten als Testdaten festgelegt und die ersten 80% der Daten als Trainingsdaten. Mit den Trainingsdaten schätzen wir die Parameter eines linearen Regressionsmodells und generieren Vorhersagen auf Basis der Testdaten. Wir können dann die Vorhersagegüte des Modells auf den Testdaten visuell evaluieren, indem wir die Abstände der tatsächlichen Werte der Testdaten zur Regressionsgeraden inspizieren.

Ein schlechter Split

Aufteilung eines Datensatzes in unvorteilhafte Trainings- und Testdaten

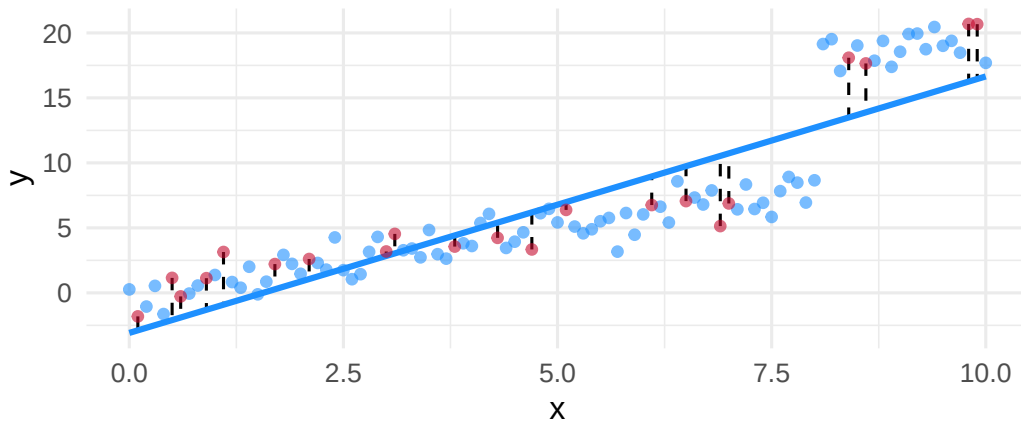


Im obigen Beispiel ist klar zu erkennen, dass der Split (also das Aufteilen der Daten) nicht optimal war. Zwar erzeugen die geschätzten Parameter gute Vorhersagen auf den Trainingsdaten, allerdings sind die Vorhersagefehler in den Testdaten sehr hoch.

Wenn wir stattdessen die Daten in zufällige Trainings- und Testdaten aufteilen, ist für das gegebene Modell der In-Sample-Fehler (IS-Fehler) zwar höher, allerdings der OOS-Fehler niedriger:

Ein guter Split

Aufteilung eines Datensatzes in gute Trainings- und Testdaten



Im obigen Beispiel haben wir gesehen, dass ein zufälliger Split zu besseren Ergebnissen führen kann. Die Frage ist nun, wie wir in R einen zufälligen Split erzeugen können!

3.1.1 Training- und Testsplit in R

Die `rsample::initial_split()`-, `rsample::training()`- und `rsample::testing()`-Funktionen erlauben ein einfaches und effizientes Aufteilen der Daten in Trainings- und Testdaten. Das `{rsample}`-Paket ist Teil des `{tidymodels}`-Ökosystems, weshalb wir auch einfach mit dem Befehl `library(tidymodels)` dieses hinzufügen können. In Abschnitt 2.3.1 haben wir den `penguins`-Datensatz verwendet, um die Parameter eines multiplen linearen Regressionsmodells zu schätzen. Des Weiteren haben wir in Abschnitt 2.3.4.3 das MLR-Modell auf dem gesamten Datensatz anhand der RMSE-Metrik evaluiert. Hierbei hat sich ein RMSE-Wert von 282 ergeben. Im Folgenden wollen wir das gleiche Modell erstellen, allerdings die Parameter nur mit den Trainingsdaten schätzen und die Performance dann auf den Testdaten evaluieren.

```
1 data_penguin <- palmerpenguins::penguins
2
3 set.seed(123)
4
5 split_penguin <- initial_split(data = data_penguin, prop = 0.8)
6 data_train <- training(split_penguin)
7 data_test <- testing(split_penguin)
```

In der dritten Zeile wenden wir den `set.seed()`-Befehl an. Diese Funktion ist wichtig, um die Reproduzierbarkeit der Resultate zu gewährleisten, wenn zufällige Prozesse im Programmablauf involviert sind. Dadurch, dass wir im Folgenden die Daten zufällig aufteilen, wird unter der Verwendung der `set.seed()`-Funktion gewährleistet, dass bei jedem Durchführen der darauffolgenden Zeilen der gleiche zufällige Split durchgeführt wird.

In Zeile 5 verwenden wir die `initial_split()`-Funktion, um die Daten aufzuteilen. Das Argument `data = data_penguin` beschreibt, welche Daten aufgeteilt werden sollen, und das Argument `prop=0.8` steuert die Proportionalität des Splits. Der Wert `prop = 0.8` teilt die Daten somit zufällig in 80% Trainings- und 20% Testdaten ein.

In Zeile 6 und 7 verwenden wir dann die `training()`- und `testing()`-Funktionen, um aus dem `split_penguin`-Objekt die Trainings- und Testdaten zu extrahieren. Die Trainings- und Testdaten sind daraufhin in einem Data-Frame bzw. Tibble gespeichert und können wie gewohnt verwendet werden.

Wir können nun die Parameter des MLR-Modells schätzen, wie wir es in `#sec-MLRR` getan haben:

```
1 mlr_spec <- linear_reg()
2 mlr_fit <- mlr_spec %>%
3   fit(data = data_train,
4       formula = body_mass_g ~.)
```

Lediglich in Zeile 3 haben wir anstatt des `data_penguin`-Datensatzes nun den `data_train`-Datensatz verwendet. Da wir nur an der Modellgüte auf Basis der Testdaten interessiert sind, evaluieren wir an dieser Stelle nicht weiter das Modell auf Basis der Trainingsdaten.

Mit der `augment()`-Funktion können wir nun wieder Vorhersagen und die dazugehörigen Residuen und mithilfe der `rmse()`-Funktion den RMSE berechnen:

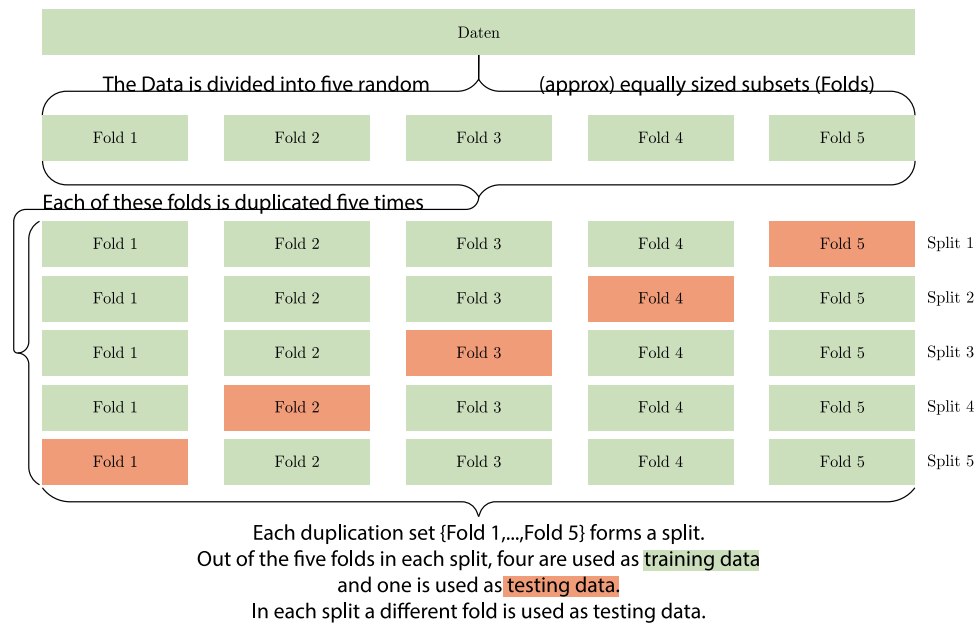
```
mlr_fit %>%  
  augment(data_test) %>%  
  rmse(truth = .pred, estimate = body_mass_g)
```

```
# A tibble: 1 x 3  
  .metric .estimator .estimate  
  <chr>   <chr>       <dbl>  
1 rmse    standard      289.
```

Der Testfehler mit dem Wert 289 ist also nur knapp größer als der Fehler, welchen wir in Abschnitt 2.3.4.3 beobachten konnten (282). Wir können deshalb für zuvor ungesehene Pinguine erwarten, dass das Gewicht im Schnitt 289 g höher oder niedriger als der tatsächliche Wert geschätzt wird.

3.2 Cross-Validation

Das Konzept der Cross-Validation (Kreuzvalidierung) knüpft direkt an das des Aufteilens der Trainings- und Testdaten an. Die Idee hierbei ist, dass wir die Daten nicht nur einmal in Trainings- und Testdaten aufteilen, sondern mehrmals auf verschiedene Art und Weise. Die folgende Grafik illustriert die Idee der five-fold Cross-Validation:



In einem ersten Schritt teilen wir die Daten in fünf gleichgroße, zufällige Teildatensätze (Folds) auf. Vereinigt ergeben diese Teildatensätze also wieder den gesamten Datensatz. Diese fünf Folds werden dann fünfmal dupliziert, sodass wir jeden Fold fünfmal auffinden. Die einzelnen Duplikate werden hierbei Splits genannt. In jedem dieser 5 Splits wird dann ein anderer Fold als Testdaten definiert und die verbleibenden vier Folds als Trainingsdaten. Wir können dann vier verschiedene Trainings- und Testdatensätze verwenden, um etwa die Parameter eines linearen Modells zu schätzen.

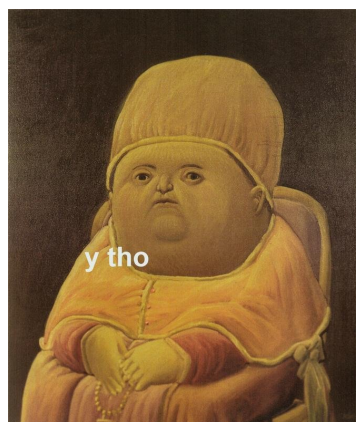


Figure 3.1: [Quelle](#)

In Beispiel 3.1 haben wir gesehen, dass ein unvorteilhafter Training- und Testsplit zu verzerrten Ergebnissen führen kann. Selbst bei einem zufällig ausgewählten Trainings- und Testsplit besteht das Risiko, einen unvorteilhaften Split zu erstellen. Indem wir die Daten also systematisch aufteilen und den Testfehler mitteln, können wir das Risiko mitigieren. Wir können den Cross-Validation-Testfehler durch den Term

$$\text{CV-Loss} = \frac{1}{5} \sum_{i=1}^5 \text{Loss}_{\text{OOS}_i}(y, \hat{y})$$

berechnen. $\text{Loss}_{\text{OOS}_i}$ steht hierbei für die OOS-Performance in Split i bezüglich einer beliebigen Loss-Funktion wie RMSE.

Wir können die Anzahl der Splits und Folds auch beliebig höher oder niedriger wählen. Falls wir zum Beispiel auf einen Datensatz mit n Datenpunkten n -Fold-Cross-Validation anwenden, d. h., die Testsets aus einem Split entsprechen einem einzelnen Sample, dann spricht man auch von Leave-One-Out-Cross-Validation. Dieses Prinzip garantiert zwar einen sehr robusten CV-Loss, kann aber unter Umständen auch zu sehr langen Berechnungszeiten führen, da wir schließlich für jeden Split ein Modell trainieren.

3.2.1 Cross-Validation in R

Die `rsample::vfold_cv`-Funktion ermöglicht es uns, ein Cross-Validation-Objekt zu erstellen. Dieses ist nichts anderes als ein verschachteltes Tibble bzw. Data Frame. D. h., jeder Eintrag ist wieder ein Tibble oder Data Frame selbst.

```
folds_penguin <- vfold_cv(data = data_penguin, v = 5)
```

Neben dem Argument `data`, welches die Daten spezifiziert, müssen wir noch die Anzahl der Folds übergeben. Diese können wir über das Argument `v` steuern. Falls wir also `v=5` setzen, dann bedeutet das, dass wir 5-Fold-Cross-Validation anwenden.

Wir können nun die Parameter eines MLR-Modells für jeden Split schätzen und den CV-Loss berechnen:

```
1 mlr_fit_cv <- mlr_spec %>%  
2   fit_resamples(  
3     resamples = folds_penguin,  
4     preprocessor = body_mass_g ~.  
5   )
```

1. In der ersten Zeile verwenden wir das bereits spezifizierte MLR-Modell `mlr_spec`, um für dieses auf Basis jedes Splits die Parameter zu schätzen.

.metric	.estimator	mean	n	std_err	.config
rmse	standard	287.459	5.000	9.545	pre0_mod0_post0
rsq	standard	0.873	5.000	0.007	pre0_mod0_post0

2. In Zeile 2. verwenden wir statt der `fit`-Funktion die `fit_resamples`-Funktion. Wie der Name bereits vermuten lässt, können wir mit dieser Funktion die Parameter auf Basis der verschiedenen Splits berechnen.
3. Wir müssen deshalb in Zeile 3 statt dem Argument `data` das Argument `resamples = folds_penguin` übergeben.
4. In der vierten Zeile spezifizieren wir dann mit dem `preprocessor`-Argument noch die Formel, bezüglich welcher die Parameter geschätzt werden sollen. Äquivalent dazu wird in der `fit`-Funktion das Argument `formula` verwendet. Hintergrund für die unterschiedlichen Argumentnamen ist, dass wir die `fit_resamples`-Funktion noch viel umfangreicher nutzen könnten, das aber den Rahmen dieser Vorlesung sprengt (siehe bei Interesse [Machine-Learning-Übungsskript](#)).

Um nun die Performance der Cross-Validation zu überprüfen, können wir die `tune::collect_metrics`-Funktion verwenden:

```
mlr_fit_cv %>% collect_metrics()
```

Der Rückgabewert der `collect_metrics`-Funktion ist ein Tibble, welches verschiedene Informationen der Evaluation des CV-Objekts enthält:

Da wir keine weiteren Metriken spezifiziert haben, werden automatisch der RMSE sowie R^2_{adj} berechnet. Die Namen der Metriken sind in der Spalte `.metric` gespeichert. In der Spalte `mean` können wir den CV-Loss einsehen. Die Spalte `n` gibt die Anzahl der Folds an und `std_error` den entsprechenden Standardfehler der Metrik auf Basis der Folds. Die `.config`-Spalte ist für diesen Kurs ebenso nicht relevant.

3.3 Umgang mit fehlenden Daten

In Abschnitt 1.3.3.2 haben wir bereits die `dplyr::na.omit()`-Funktion kennengelernt, welche ganze Einträge entfernt, wenn mindestens eine Variable in des Eintrags nicht vorhanden ist. Diese Art des Umgangs mit fehlenden Daten ist zwar einfach, birgt allerdings Risiken:

- Falls wir durch diese Methode eine oder mehrere einflussreiche Variablen entfernen, können die Ergebnisse bei einer anschließenden Regression stark verzerrt sein.
- Falls wir vorher nicht prüfen, wie viele Einträge durch das Anwenden der Funktion entfernt werden, kann unter Umständen ein Großteil der Daten entfernt werden.

- Falls wir in den Daten ein Muster nicht erkennen, welches durch das Entfernen der Einträge mit fehlenden Daten entfernt wird, kann das ebenso zu verzerrten Ergebnissen bei einer anschließenden Regression führen.

Aufgrund der obigen Aspekte sollten wir uns also auch mit einer anderen Methode auseinandersetzen, welche sich mit dem Umgang mit fehlenden Werten beschäftigt.

Als Beispieldatensatz verwenden wir wieder den Pinguin-Datensatz. Zuerst sollten wir herausfinden, wie viele Einträge im Datensatz nicht vorhanden sind:

```
data_penguin %>%
  is.na() %>%
  colSums() %>%
  bind_rows() %>%
  t()
```

	[,1]
species	0
island	0
bill_length_mm	2
bill_depth_mm	2
flipper_length_mm	2
body_mass_g	2
sex	11
year	0

Im Datensatz sind also sowohl numerische als auch nominale Werte teilweise nicht vorhanden.

3.3.1 Imputation

Eine Methode für den Umgang mit fehlenden Daten ist die *Imputation*. Die grundlegende Idee hierbei ist simpel: Wir ergänzen die fehlenden Werte. Das sinnvolle Ergänzen der Werte ist hierbei die größte Schwierigkeit, weshalb wir im Folgenden einige Ansätze besprechen.

3.3.1.1 Imputation durch Lagemaße (cold imputation)

Das wohl einfachste Imputationsverfahren ergänzt fehlende Werte durch Lagemaße wie den Mittelwert, den Median oder den Modus. Hierbei gilt allerdings zu beachten, dass der Mittelwert und Median ausschließlich für numerische Variablen verwendet werden können. Das folgende Code-Snippet illustriert, wie wir im Falle des Pinguin-Datensatzes Mean- bzw. Modusimputation durchführen können:

```

1 data_penguin_imputed <- data_penguin %>%
2   mutate(
3     across(.cols = where(is.numeric),
4       .fns = ~if_else(
5         is.na(.),
6         mean(.,na.rm = TRUE),
7         .)
8     ),
9     across(.cols = where(is.factor),
10      .fns = ~if_else(
11        is.na(.),
12        (
13          data_penguin %>%
14            select(sex) %>%
15            table() %>%
16            as_tibble() %>%
17            filter(n==max(n)) %>%
18            select(sex) %>%
19            pluck(1) %>%
20            factor()
21        ),
22      .)
23   )
24 )

```

1. Im ersten Schritt erstellen wir einen neuen Datensatz `data_penguin_imputed`.
2. Danach übergeben wir den `data_penguin`-Datensatz an die `mutate`-Funktion.
3. In den Zeilen 3–7 überschreiben wir in den numerischen Spalten die fehlenden Werte mit dem entsprechenden Durchschnitt der Spalte.
 1. Hierbei verwenden wir die `across`- und `where`-Funktionen. Sinngemäß kann die Kombination dieser beiden Funktionen als „wähle alle Spalten aus, bei welcher die Bedingung ... gilt“ interpretiert werden. ... ist hierbei ein Platzhalter für eine Bedingung oder Funktion. In Zeile 3 ist die Bedingung durch die `is.numeric()`-Funktion gegeben, welche prüft, ob eine Spalte einen numerischen Datentyp hat.
 2. Falls eine der `data_penguin`-Spalten einen numerischen Datentyp besitzt, wird die `if_else`-Funktion auf diese angewendet. Die `if_else()`-Funktion überprüft ebenso eine Kondition, welche als erstes Argument übergeben wird. Falls die Kondition erfüllt ist, so wird das zweite Argument angewendet und falls nicht, entsprechend das dritte.
 3. Wir prüfen also mithilfe der `if_else()`- und `is.na()`-Funktionen, ob ein Eintrag in einer Spalte den Wert NA oder NaN besitzt. Falls das der Fall ist, wird dieser Wert mit dem durchschnittlichen Wert dieser Spalte befüllt (berechnet mithilfe der

`mean`-Funktion). Falls der Wert nicht `NA` oder `NaN` ist, so wird der ursprüngliche Wert beibehalten.

Die Bedeutung der `.` und `~` Operatoren sind hierbei die folgenden:

- Der `.` Operator wird als Platzhalter für alle Spalten verwendet, welche aus der vorherigen Operation zurückgegeben wurden. So wird zum Beispiel beim Funktionsaufruf `is.na(.)` in Zeile 5 mithilfe des `.` symbolisiert, dass jene Spalten ausgewertet werden, welche zuvor durch den `where(is.numeric)` selektiert wurden.
 - Der `~` Operator wird verwendet um eine [anonyme Funktion](#) zu erstellen und anzuwenden. Da das `.fn` Argument der `across()`-Funktion eine Funktion selbst erwartet, übergeben wir nicht den ausgewerteten Funktionsaufruf `is.na(.)` direkt, sondern symbolisieren mithilfe des `~` Operators, dass diese Funktion auf Basis der vorherigen Argumente ausgewertet werden soll.
4. In den Zeilen 9–22 wird das Prozedere für die Faktorvariablen wiederholt, mit dem entscheidenden Unterschied, dass hier nicht der Durchschnitt, sondern der Modus in den Zeilen 13–20 berechnet wird. Als zweites Argument der `if_else`-Funktion wird der Modus der Spalte `sex` berechnet.

Vorteile der Lagemaßimputation umfassen die Einfachheit der Implementierung, schnelle Berechenbarkeit und Stabilität.

3.3.1.2 Imputation durch Regression

Neben der Imputation durch Lagemaße werden heutzutage auch oft komplexere Algorithmen verwendet, welche die fehlenden Daten genauer approximieren können. Eine relativ einfache, aber dennoch fortgeschrittene Methode ist die Imputation mithilfe eines linearen Modells. Falls in einem Datensatz mit unabhängigen Variablen $X = (X_1, X_2, \dots, X_J)$ etwa Werte in der Spalte X_j , $1 \leq j \leq J$ vorhanden sind, so kann mithilfe eines MLR-Modells mit der Modellgleichung

$$X_j = \sum_{i=1, i \neq j}^J \beta_i X_i + \beta_0 + \varepsilon$$

geschätzt werden. Zum Schätzen der Parameter können wir hierbei alle Einträge verwenden, in welchen die Variable X_j vorhanden ist. Dieser Ansatz funktioniert besonders gut, wenn die unterliegenden Daten alle Annahmen des MLR-Modells erfüllen.

Mithilfe der `{mice}`-Library¹, können wir nicht nur eine Imputation wie in Abschnitt 3.3.1.1 durchführen, sondern auch eine Regression, wie sie oben beschrieben wurde. Hierfür benötigen wir die Funktion `mice` und `complete`:

¹Diese muss eventuell über den Befehl `install.packages("mice")` installiert werden

```

1 library(mice)
2
3 methods <- c(bill_length_mm = "norm.predict",
4             bill_depth_mm = "norm.predict",
5             flipper_length_mm = "norm.predict",
6             body_mass_g = "norm.predict",
7             sex = "logreg"
8             )
9
10 data_penguin_imputed <- data_penguin %>%
11   mice(method = methods,
12        blocks = names(methods),
13        maxit = 5,
14        m = 1) %>%
15   complete()

```

1. In der ersten Zeile fügen wir die `{mice}`-Library unserer Session hinzu.
2. In den Zeilen 3–7 spezifizieren wir für jede Variable mit fehlenden Werten eine Methode, welche für die Imputation verwendet werden soll. Die Methode `"norm.predict"` spezifiziert hierbei ein MLR-Modell für die numerischen Variablen und `"logreg"` ein logistisches Modell für die nominale Variable `"sex"`.
3. In Zeile 10 generieren wir einen neuen Datensatz `data_penguin_imputed`, welcher später den vervollständigten Datensatz enthält.
4. Anschließend wenden wir auf den alten Datensatz in den Zeilen 10–14 die `mice`-Funktion an. Das Argument `method` spezifiziert, wie wir die fehlenden Werte ersetzen wollen (als Argument geben wir hierbei den gleichnamigen Vektor aus Zeile 3 an). Das Argument `blocks` spezifiziert, auf welche Variablen die jeweiligen Imputationsalgorithmen angewendet werden sollen. Durch den Wert `names(methods)` können wir direkt die Imputationsalgorithmen mit den entsprechenden Variablen matchen.
5. Das Argument `maxit=1` in Zeile 9 spezifiziert die Anzahl der Iterationen für das Generieren der synthetischen Daten. Die Details hierfür würden den Rahmen der Vorlesung sprengen und können unter [diesem Link](#) bei Interesse nachgeschlagen werden. Wichtig ist nur das Verständnis, dass eine höhere Anzahl an Iterationen stabilere Werte liefern kann.
6. Das Argument `m` beschreibt, wie oft ein Datensatz mithilfe der Imputationstechniken erzeugt werden soll. Das Argument `m=1` spezifiziert somit, dass wir nur einen solchen Datensatz erzeugen wollen.
7. Da der Rückgabewert der `mice`-Funktion ein Objekt ist, welches mehr als nur den vervollständigten Datensatz enthält, verwenden wir zum Schluss die `complete`-Funktion, welche uns einen vervollständigten Datensatz zurückgibt.

Wir können nun nochmals überprüfen, ob wirklich alle Werte befüllt wurden:

```
data_penguin_imputed %>%
  is.na() %>%
  colSums() %>%
  bind_rows() %>%
  t()
```

```

              [,1]
species      0
island       0
bill_length_mm 0
bill_depth_mm 0
flipper_length_mm 0
body_mass_g   0
sex          0
year         0

```

3.4 Merkmalsselektion mithilfe von Lasso-Regression

Ein weiteres wichtiges Thema ist die Variablenselektion. Motivierende Faktoren für die Variablenselektion sind:

- In umfangreichen Datensätzen haben nicht alle unabhängigen Variablen aufgrund von Multikollinearität einen signifikanten Einfluss auf die Zielvariable.
- Falls Interpretierbarkeit eine große Rolle spielt, ist es manchmal vorteilhaft sich auf einige wenige Variablen festzulegen, welche die geschätzte Variable hinreichend erklären.

In der Vorlesung wurden bereits Methoden wie *Forward*, *Backward* und *Stepwise* Selektion behandelt. Wir wollen deshalb in diesem Abschnitt eine weitere, essenzielle Methodik betrachten, welche auf der Idee der linearen Regression aufbaut.

Angenommen, wir befinden uns in dem bekannten Fall, dass ein Datensatz $X = (X_1, \dots, X_J)$ mit $X_j \in \mathbb{R}^K$ und K Realisierungen der Zielvariable $Y = (y_1, \dots, y_K)$ vorliegt und wir die Parameter $\hat{b}_0, \dots, \hat{b}_J$ schätzen. Die Schätzung der Parameter kann zum Beispiel durch die Minimierung des MSE (siehe vorheriges Kapitel) durchgeführt werden:

$$\min_{b_0, \dots, b_J} \frac{1}{K} \sum_{k=1}^K (y_k - (b_0 + b_1 x_{1,k} + \dots + b_J x_{J,k}))^2$$

Durch das Hinzufügen eines sogenannten *Penalty*-Terms können wir steuern, dass Parameter b_j , welche keinen großen Einfluss auf die Zielvariable haben, verkleinert oder sogar ganz eliminiert werden:

$$\min_{b_0, \dots, b_J} \frac{1}{K} \sum_{k=1}^K (y_k - (b_0 + b_1 x_{1,k} + \dots + b_J x_{J,k}))^2 + \lambda \sum_{j=0}^J |b_j| \quad (3.1)$$

$\lambda \in [0, \infty)$ (λ) steht hier für den Penalty: Je größer dieser Wert ist, desto stärker werden die Parameter bestraft. Für die genaue Funktionsweise und Methodik hinter diesem Konzept siehe [Wikipedia Artikel](#). Wichtig an dieser Stelle ist lediglich zu verstehen, wie der Fehlerterm in Gleichung 3.1 definiert ist und welchen Einfluss der Parameter λ hat.

Die R-Syntax zum Trainieren und Evaluieren eines solchen Modells ist sehr nah an der eines MLR-Modells:

```
mlr_spec_lasso <- linear_reg(penalty = 20,
                             mixture = 1,
                             engine = "glmnet")

mlr_lasso_fit <- mlr_spec_lasso %>%
  fit(data = data_train,
       formula = body_mass_g~.)

mlr_lasso_fit %>%
  tidy()
```

```
# A tibble: 10 x 3
  term          estimate penalty
  <chr>          <dbl>    <dbl>
1 (Intercept)    63116.      20
2 speciesChinstrap -173.      20
3 speciesGentoo   676.      20
4 islandDream     -4.23     20
5 islandTorgersen 0          20
6 bill_length_mm  9.03      20
7 bill_depth_mm   0          20
8 flipper_length_mm 21.0      20
9 sexmale        454.      20
10 year          -31.8     20
```

Bevor wir den obigen Code analysieren, ist es sinnvoll, den Rückgabewert zu betrachten. Das Tibble enthält drei Spalten, wobei in der ersten der Variablenname, in der zweiten der geschätzte Lasso-Parameter b_j^{Lasso} und in der dritten der Term $\lambda = 20$ steht. Der Parameter $\lambda = 20$ hat dafür gesorgt, dass die geschätzten Parameter für die Variablen `bill_depth_mm` und `islandTorgersen` auf 0 reduziert wurden. Da `islandTorgersen` Teil eines Dummy-Konstrukts

ist, können wir diese Spalte nicht direkt entfernen. Allerdings könnten wir auf Basis unserer Analyse etwa die Variable `bill_depth_mm` entfernen.

! Achtung

Die geschätzten Parameter sind nicht mehr *unbiased*, weshalb eine statistische Analyse, zum Beispiel durch den *t*-Test, hier weniger sinnvoll ist als bei einem MLR-Modell.

Nun zurück zum Code:

```
1 mlr_spec_lasso <- linear_reg(penalty = 20,  
2                               mixture = 1,  
3                               engine = "glmnet")  
4  
5 mlr_lasso_fit <- mlr_spec_lasso %>%  
6   fit(data = data_train,  
7       formula = body_mass_g~.)  
8  
9 mlr_lasso_fit %>%  
10  tidy()
```

Im Vergleich zu einem MLR-Modell wie in Abschnitt 2.3.1 haben wir bei der Modellspezifikation durch die `linear_reg`-Funktion die Argumente `penalty` und `mixture` ergänzt. Wie bereits besprochen, steht `penalty` hier für den Parameter λ aus Gleichung 3.1. Der Parameter `mixture = 1` spezifiziert hierbei, dass es sich um ein Lasso-Modell handelt. Der Parameter `mixture` kann Werte in dem Intervall $[0, 1]$ annehmen, wobei wir ausschließlich an dem Fall `mixture=1` interessiert sind. Außerdem haben wir in Zeile 3 das Argument `engine = "glmnet"` spezifiziert. Dieses steuert, dass die Parameter nicht mehr wie bei einer linearen Regression berechnet werden (aufgrund des Penalty-Terms), sondern durch ein Optimierungsverfahren, welches Gleichung 3.1 löst. Im letzten Schritt rufen wir nicht wie gewohnt die `extract_fit_engine()`- und `summary()`-Funktionen auf, sondern wenden auf das trainierte Modell die `tidy()`-Funktion an, welche die geschätzten Parameter als Tibble zurückgibt.

3.5 Übungsaufgaben

In dieser Übung werden wir uns noch einmal mit dem `hotel_rates`-Datensatz aus den Übungen Aufgabe 2.8 – Aufgabe 2.11 auseinandersetzen.

Verwende die unten stehende Version des `hotel_rates`-Datensatzes für die folgenden Aufgaben.

```

hotel_rates_filtered <- hotel_rates %>%
  mutate(
    arrival_month = factor(month(arrival_date)),
    num_guests = adults+children,
    is_repeated_guest = factor(is_repeated_guest),
  ) %>%
  select(num_guests,total_of_special_requests,
         required_car_parking_spaces,arrival_month,
         is_repeated_guest,lead_time,stays_in_week_nights,
         avg_price_per_room,country)

```

Aufgabe 3.1.

1. In der Variable `country` wurden fehlende Werte mit dem String "null" encodiert. Finde heraus, wie viele Einträge der Variable `country` den Wert "null" besitzen.
2. Verwende die `dplyr::na_if`-Funktion, um die mit "null" encodierten Werte in den Wert NA umzuwandeln.
3. Wir wollen nun die fehlenden Werte ergänzen. Da die Variable `country` nominal ist, können wir keine lineare Regression anwenden. Wende stattdessen in der `mice()`-Funktion die Methode "cart" an, was für *Classification And Regression Trees* steht (diese werden im folgenden Kapitel genauer beleuchtet). Setze die Parameter `maxit` und `m` beide auf 1 und erzeuge dann mithilfe der `complete()`-Funktion einen neuen Datensatz `hotel_rate_imp`, welcher die imputierten Werte enthält.

Aufgabe 3.2.

1. Da in dem Datensatz etwa 100 verschiedene Nationalitäten vertreten sind, wir aber nur an jenen interessiert sind, welche zur Erklärung der Zielvariable beitragen, sollte die Spalte `country` gefiltert werden. Der folgende Vektor enthält 39 Country-Codes, von welchen wir vermuten, dass sie einen statistisch signifikanten Einfluss auf die Variable `avg_price_per_room` haben.

```

cntry_list<-c("and", "aut", "aze",
              "che", "chl", "cze",
              "esp", "geo", "hun", "ita",
              "lux", "mar", "mwi", "nga", "prt",
              "rus", "swe", "tur", "tw",
              "usa")

```

Passe unter Verwendung der `filter()`-Funktion und des `%in%`-Operators den Datensatz aus Aufgabe 3.1 so an, dass dieser nur noch Einträge enthält, welche in der Spalte `country` einen Wert des obigen Vektors enthalten.

2. Entferne mit der gleichen Methode alle Einträge, welche in der Spalte `arrival_month` die Werte `month2` und `month11` enthalten.

Aufgabe 3.3. Verwende den in Aufgabe 3.2 erstellten Datensatz, um einen Trainings- und Testdatensatz im Verhältnis 4:1 zu erstellen. Um die Reproduzierbarkeit sicherzustellen, verwende den `set.seed(123)`-Befehl.

Aufgabe 3.4.

1. Verwende die in der vorherigen Aufgabe erstellten Trainingsdaten, um ein MLR-Modell wie in Aufgabe Aufgabe 2.9 zu schätzen.
2. Welches der noch im Datensatz enthaltenen Herkunftsländer hat den größten Einfluss auf den durchschnittlichen Zimmerpreis?
3. Berechne anschließend für beide Teildatensätze den R^2 und $RMSE$ und vergleiche die beiden Werte.

Aufgabe 3.5. Verwende den Datensatz aus Aufgabe 3.2 um ein 5-Fold-Cross-Validation-(CV-)Objekt zu erstellen. Schätze anschließend auf diesem CV-Objekt ein MLR-Modell wie in Aufgabe 3.4 und vergleiche den CV-RMSE mit dem Test-RMSE aus Aufgabe Aufgabe 3.4. Achte auch hierbei auf Reproduzierbarkeit und verwende den Befehl `set.seed(123)`.

4 Regressionsbäume

In diesem Kapitel setzen wir uns mit Baummodellen in einem Regressionskontext auseinander. Baummodelle bilden die Grundlage für viele weitere Modelle, welche aufgrund ihrer Fähigkeit, Zusammenhänge sehr gut zu modellieren, oft genutzt werden. Mit einer dieser Erweiterungen setzen wir uns in Abschnitt 4.5 auseinander.

Der Hauptfokus dieses Kapitels liegt allerdings darauf, die Grundidee des Modells und des Algorithmus zu verstehen, der für das Schätzen von Regressionsbäumen verwendet wird.

4.1 Grundidee

Regressionsbäume sind Teil der *Classification and Regression Tree* (CART)-Familie. Regressions- und Klassifikationsbäume werden auch Entscheidungsbäume genannt, da die Vorhersagen durch (binär: ‘Ja’/‘Nein’) Entscheidungen berechnet werden.

4.1.1 Darstellung von Bäumen und Vorhersagen

In diesem Unterkapitel diskutieren wir zwei Möglichkeiten, um Entscheidungsbäume darzustellen.

4.1.1.1 Baumdiagramme

Die grafischen Bestandteile bei der Darstellung eines Entscheidungsbaums als Baumdiagramm sind *Knoten* (nodes) und *Kanten*. Bei den Knoten unterscheiden wir vor allem zwischen dem *Wurzelknoten* (root node) und *Blattknoten*. Der Wurzelknoten steht hierbei immer ganz oben im Diagramm und die Blattknoten am Ende der jeweiligen Entscheidungspfade. Knoten, welche zwischen dem Wurzel- und den Blattknoten stehen, nennen wir auch *Entscheidungsknoten*. (Beachte, dass auch der Wurzelknoten ein Entscheidungsknoten ist!) In einem Entscheidungsknoten prüfen wir eine aufgestellte Bedingung und bewegen uns auf Basis der Entscheidung entsprechend entlang der Kanten. Bei numerischen Features sind die Bedingungen in Form eines Größenvergleichs gegeben. Es wird also überprüft, ob das Feature eines Datenpunktes größer (gleich) oder kleiner als ein Schwellenwert ist. Ähnlich wie beim Dummy-Encoding bei einer linearen Regression wird bei nominalen Merkmalen lediglich geprüft, ob die

Ausprägung der Variable mit dem Entscheidungswert übereinstimmt oder nicht. Diesen Vorgang wiederholen wir, bis ein Blattknoten erreicht wird, in welchem die Vorhersagen stehen.

Folgende Grafik zeigt eine beispielhafte Darstellung eines solchen Entscheidungsbaums.

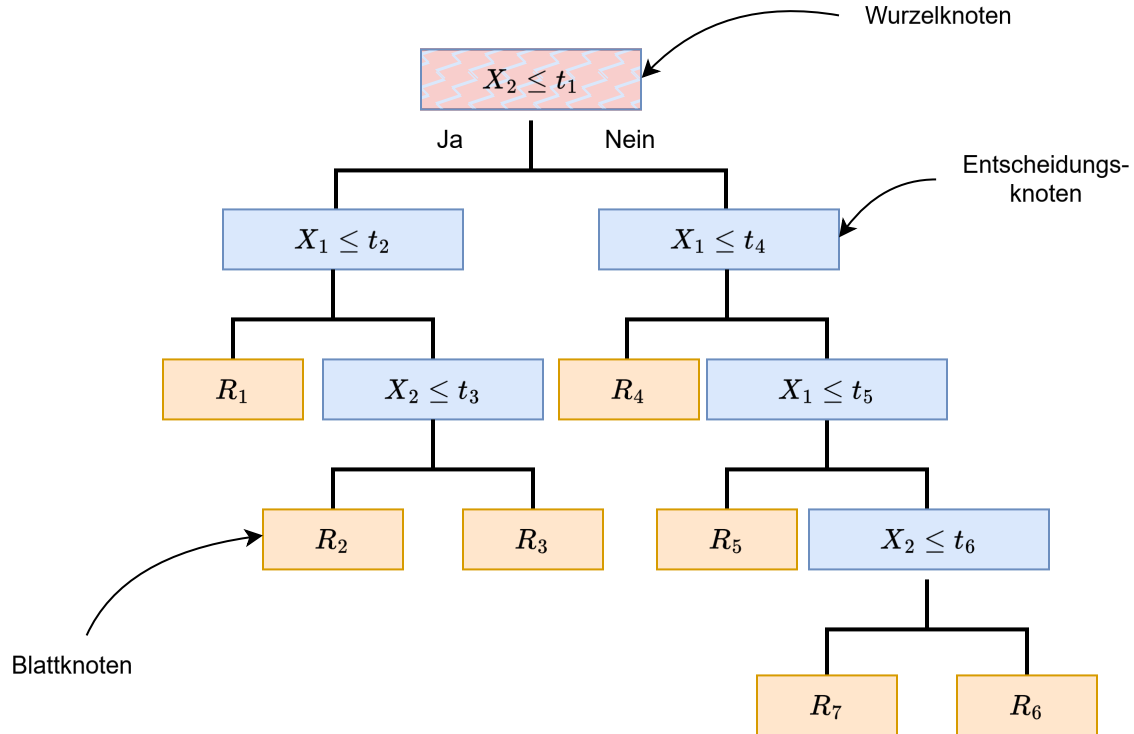


Figure 4.1

Das in Figure 4.1 dargestellte Baumdiagramm wurde auf Basis eines Datensatzes erzeugt, welcher die beiden Variablen X_1 und X_2 enthält. In jedem Entscheidungsknoten wird das Entscheidungskriterium durch einen Schwellenwert (Splittingpoint) $t_i, i = 1, \dots, 6$ ausgedrückt. Falls in einem binären Entscheidungsbaum also 6 Splittingpoints gegeben sind, dann existieren $6 + 1 = 7$ Blattknoten. Da der oberste Knoten (Wurzelknoten) auch ein Entscheidungsknoten ist, hat dieser teilweise die gleiche Farbe wie die anderen Entscheidungsknoten, ist aufgrund seiner Wurzeleigenschaft aber noch einmal gesondert zu betrachten.

Wir prüfen also im ersten Schritt, ob die Variable X_2 kleiner oder gleich t_1 ist. Falls das der Fall ist, bewegen wir uns entlang der linken Kante und ansonsten entlang der rechten Kante zum nächsten Entscheidungsknoten. Angenommen, $X_2 \leq t_1$, dann bewegen wir uns zum nächsten Knoten, in welchem wir die Bedingung $X_1 \leq t_2$ prüfen. Falls diese Bedingung ebenso erfüllt ist, landen wir im Blattknoten mit dem Wert R_1 . Der Wert R_1 ist dann die Vorhersage für den entsprechenden Datenpunkt. Wie die Werte $R_k, k = 1, \dots, 6$ berechnet werden, untersuchen wir in Abschnitt 4.2.

4.1.1.2 Aufteilung des Featurespace

Äquivalent zur Darstellung eines Diagramms können wir, zumindest bei ein- oder zweidimensionalen Datensätzen, einen Entscheidungsbaum auch im *Featurespace* darstellen. Mit Featurespace meinen wir hier den Raum, welcher durch die Ausprägungen aufgespannt wird. Existieren wie in Figure 4.1 zum Beispiel zwei Feature X_1 und X_2 , dann können wir die Splittingpoints und Blattknoten in einem Koordinatensystem darstellen:

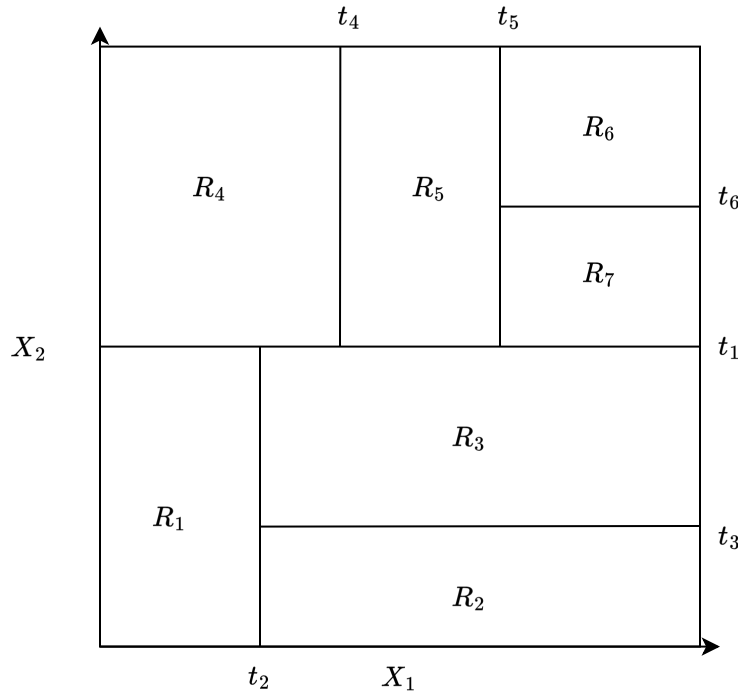


Figure 4.2

Beachte, dass die Darstellungen in Figure 4.1 und Figure 4.2 äquivalent sind. Wir können also jeweils die eine Darstellung in die andere überführen. Um etwa den Blattknoten R_6 aus Figure 4.1 zu erreichen, müssen wir in Figure 4.2 die gleichen Bedingungen prüfen:

1. $X_2 \leq t_1$ ist bei R_6 nicht erfüllt und deshalb $X_2 > t_1$. Wir befinden uns also oberhalb der horizontalen Linie durch den Punkt t_1 .
2. $X_1 \leq t_4$ ist bei R_6 ebenso nicht erfüllt, weshalb $X_1 > t_4$ gilt.
3. $X_1 \leq t_5$ ist bei R_6 ebenso nicht erfüllt, weshalb $X_1 > t_5$ gilt.
4. $X_2 \leq t_6$ ist bei R_6 ebensowenig erfüllt, weshalb $X_2 > t_6$ gilt.

Zusammengefasst muss für einen Datenpunkt (x_1, x_2) also $x_2 > t_6$ und $x_1 > t_5$ gelten, damit der Datenpunkt in den Blattknoten R_6 fällt.

4.1.2 Vorteile gegenüber linearen Modellen

Bäume gehören aufgrund ihrer Struktur zu nichtlinearen Modellen. Falls die unterliegenden Daten also nicht der Linearitätsannahme unterliegen, können wir bei einem *guten* Baum erwarten, dass dieser die Zusammenhänge in den Daten besser beschreibt.

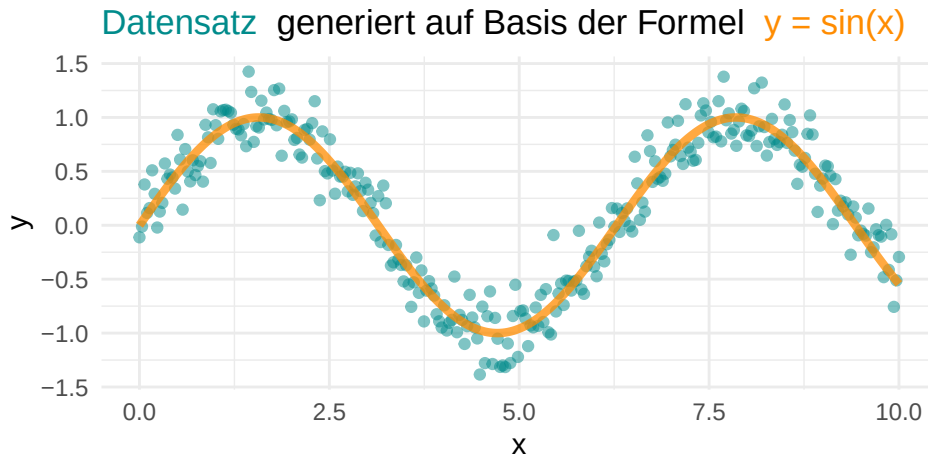
Betrachte hierfür folgendes synthetisches Beispiel, welches diesen Effekt illustriert:

Beispiel 4.1. Der Datensatz `data_exm` besteht aus zwei Spalten `x` und `y`, wobei `x` Werte zwischen 0 und 10 annimmt und `y` durch die Formel

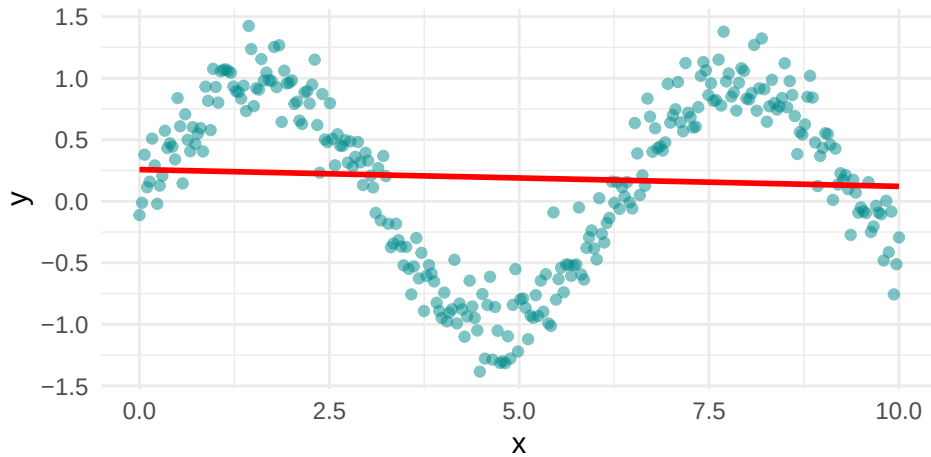
$$y = f(x) + \varepsilon = \sin(x) + \varepsilon$$

(mit $\varepsilon \sim \mathcal{N}(0, 1)$) berechnet wird. `y` soll hierbei die abhängige und `x` die unabhängige Variable darstellen und das Ziel ist deshalb, `y` durch `x` zu approximieren.

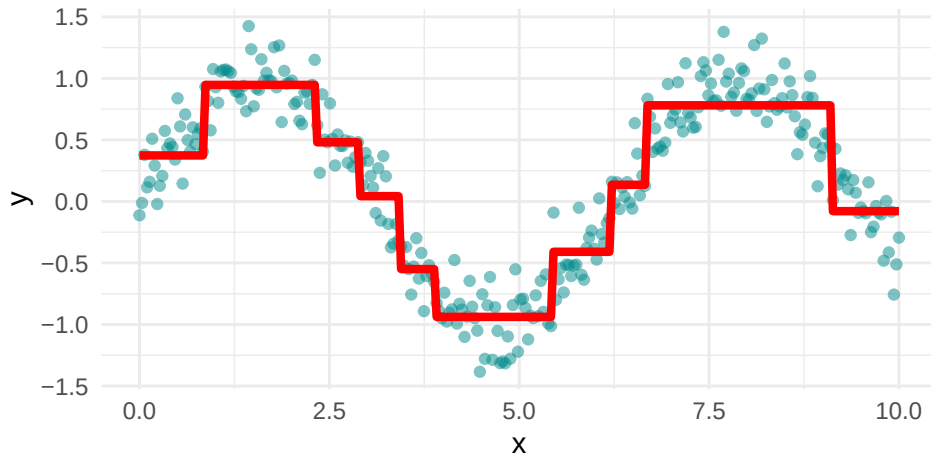
Die folgende Grafik zeigt eine Realisation dieser Simulation:



Falls wir nun versuchen, den Zusammenhang zwischen `y` und `x` mithilfe eines einfachen linearen Modells zu modellieren, so erhalten wir die Regressionsgerade, welche in der folgenden Grafik dargestellt ist.



Offensichtlich eignet sich die einfache lineare Regression nicht, um den Zusammenhang zwischen y und x zu modellieren. Wir können allerdings den Zusammenhang ebenso durch einen Regressionsbaum modellieren, welcher im Featurespace wie folgt dargestellt werden kann.



Beachte, dass unser Featurespace in diesem Beispiel nur eindimensional ist, weshalb das resultierende Baummodell keine Flächen (wie in Figure 4.2), sondern Geraden enthält. Die rote Linie beschreibt hierbei die durch das Baummodell geschätzten Werte $\hat{y}$ für ein gegebenes x . Das Baummodell modelliert den Einfluss von x auf y also vergleichsweise gut, obwohl die Daten keinen linearen Zusammenhang besitzen!

Zusammengefasst spiegelt sich der erste Vorteil von Baummodellen gegenüber linearen Modellen also in der Fähigkeit wider, auch nichtlineare Zusammenhänge zu modellieren.

Einen weiteren Vorteil können wir bei der Betrachtung von Figure 4.1 erkennen: Bei Baummodellen ist sehr leicht erkennbar, auf Basis welcher Variablen und Schwellenwerte die Vorhersagen

getroffen werden. Obwohl das Modell aus einer Vielzahl von Variablen besteht, kann man durch die Darstellung eines Baumdiagramms effizient den Wert berechnen, welcher durch das Modell vorhergesagt wird.

4.2 Top-Down-Greedy

Bisher haben wir zwar verschiedene Darstellungen der Regressionsbäume betrachtet und herausgefunden, wie wir bei den verschiedenen Darstellungen Vorhersagewerte ablesen können, allerdings nicht, wie die Schwellwerte und Splittingvariablen bestimmt werden.

Der Algorithmus, welcher hierfür verwendet wird, ist der sogenannte *Top-Down-Greedy*- oder *Recursive-Binary-Splitting*-Algorithmus. Um den Algorithmus zu erklären, orientieren wir uns primär an der Darstellung aus Figure 4.1.

Die Idee des Algorithmus ist, dass wir in jedem Schritt den Feature Space *bestmöglich* zweiteilen, um eine verbesserte Vorhersage im Vergleich zur ungeteilten Version zu erhalten. Formal lässt sich der Algorithmus wie folgt darstellen:

1. Initialisierung:

Beginne mit dem gesamten Trainingsdatensatz

$$D = \{(x_k, y_k) : x_k \in \mathbb{R}^J, y_k \in \mathbb{R}, k = 1, \dots, K\}$$

in der Wurzel des Baums.

2. Spaltensuche :

Für jedes Feature X_j und jeden potenziellen¹ Schwellenwert s :

i) Teile die Daten in zwei Regionen:

$$R_1(j, s) = \{x \in D \mid x_j \leq s\}$$

$$R_2(j, s) = \{x \in D \mid x_j > s\}$$

ii) Berechne den MSE, welcher durch diese Aufteilung entsteht:

$$\text{MSE}(j, s) = \frac{1}{|R_1|} \sum_{i: x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \frac{1}{|R_2|} \sum_{i: x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2$$

Wobei $\bar{y}_{R_1}$ der durchschnittliche Wert aller Datenpunkte y ist, welche in R_1 liegen, und $\bar{y}_{R_2}$ der durchschnittliche Wert aller Datenpunkte y , welche in R_2 liegen.

¹Diese sind durch die Mittelwerte aller benachbarten Punkte gegeben.

3. Wahl des besten Schwellenwerts :

Wähle das Paar (j^*, s^*) , welches den MSE minimiert.

4. Rekursives Aufteilen:

- i) Erzeuge zwei neue Knoten für $R_1(j^*, s^*)$ und $R_2(j^*, s^*)$.
- ii) Wiederhole Schritte 2–4 rekursiv für jeden dieser Knoten.

5. Stopkriterium prüfen:

Beende die Aufteilung, wenn eines der folgenden Kriterien erfüllt ist:

- i) Die Region enthält weniger als eine vorher festgelegte Mindestanzahl an Beobachtungen.
- ii) Eine vorher festgelegte maximale Baumtiefe wurde erreicht.
- iii) Die Reduktion des Fehlers liegt unter einem vorher festgelegten Schwellenwert.

6. Terminalknoten-Wert setzen:

Weise jedem Blattknoten den Mittelwert der Zielvariablen $\bar{y}_{R_i}$, $i = 1, \dots, I$ der enthaltenen Datenpunkte zu.

4.2.1 Rekursive und Greedy-Eigenschaft des Algorithmus

Der *rekursive* Charakter des Algorithmus ist hierbei in Schritt 4. zu erkennen, da nach jedem Aufteilen des Datensatzes in zwei Teildatensätze diese Teildatensätze nach dem gleichen Prinzip wieder aufgeteilt werden.

Der *greedy* Charakter des Algorithmus ist in Schritt 3. wiederzufinden. Da wir in jeder Iteration jenen Split wählen, welcher den MSE in diesem Schritt minimiert, ist gewährleistet, dass wir in jeder Iteration den bestmöglichen Split bilden. Dieses Vorgehen kann aber unter Umständen zu einem nicht optimalen Baum führen: Angenommen, wir wollen eine Region R_i in zwei Teilregionen $R_{i,1}$ und $R_{i,2}$ aufteilen und wählen hierfür das optimale Paar (j^*, s^*) . Dann existiert keine weitere Kombination $(\tilde{j}, \tilde{s})$, sodass $\text{MSE}(\tilde{j}, \tilde{s}) < \text{MSE}(j^*, s^*)$. Falls wir also R_i bezüglich des Paares (j^*, s^*) in $R_{i,1}$ und $R_{i,2}$ aufteilen und daraufhin $R_{i,1}$ optimal in $R_{i+1,1}$ und $R_{i+1,2}$ aufteilen, dann kann es sein, dass unter der Verwendung von $(\tilde{j}, \tilde{s})$ aus der letzten Iteration ein Split $R_{\tilde{i}+1,1}$ und $R_{\tilde{i}+1,2}$ hätte entstehen können, für den

$$\text{MSE}_{\tilde{i}+1}(j, s) < \text{MSE}_{i+1}(j, s)$$

gilt.

4.2.2 Stoppkriterien und Pruning

Im 5. Schritt des Top-Down-Greedy-Algorithmus haben wir verschiedene Kriterien genannt, welche zum Stoppen des Algorithmus führen können.

Wir wollen in diesem Abschnitt die Stopp-Kriterien ein wenig genauer definieren und untersuchen.

4.2.2.1 Mindestanzahl an Beobachtungen `min_n`

Das Kriterium zur Mindestanzahl an Beobachtungen prüft vor jedem weiteren Aufteilen des Featurespace, ob in den durch das Aufteilen resultierenden Teildatensätzen eine Mindestanzahl an Beobachtungen liegt. Ziel dieses Kriteriums ist es, zu vermeiden, dass sich das Baummodell zu stark an die Trainingsdaten anpasst. Die Vorhersagen $\hat{y}$ werden durch das Bilden des Durchschnitts in einem Blattknoten R_m generiert:

$$\frac{1}{|R_m|} \sum_{k: x_k \in R_m} y_k$$

Falls also in R_m nur eine Observation vorhanden ist, kann das zu einer starken Verzerrung bei einer Evaluation mithilfe von Testdaten führen.

Wir bezeichnen nun mit `min_n` die Anzahl an Datenpunkten, welche mindestens in einem Teilgebiet liegen müssen, sodass dieses geteilt wird. `min_n` muss also mindestens den Wert 2 besitzen. Jedoch stellt sich natürlich die Frage, welchen Wert wir für `min_n` beim Schätzen eines Regressionsbaumes festlegen sollen. Natürlich lässt sich diese Frage wie immer **nicht eindeutig** beantworten. Wir werden später sehen, dass R diesen Wert erst mal selbst bestimmt, wir diesen alternativ aber auch übergeben können. Im Rahmen dieser Veranstaltung geben wir uns aber damit zufrieden, welchen Wert R für optimal hält.

4.2.2.2 Baumtiefe `tree_depth`

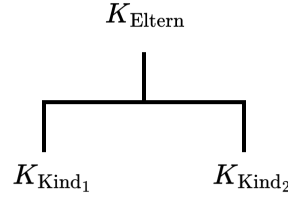
Der Parameter `tree_depth` kann ebenso zum Stoppen des Algorithmus führen: Damit der Baum nicht unkontrolliert wächst (genauer gesagt an Tiefe gewinnt), stoppen wir den Algorithmus, sobald der Baum eine bestimmte Tiefe besitzt. Jedoch müssen wir noch definieren, was mit *Tiefe* überhaupt gemeint ist.

1. Die Länge eines Pfades zwischen dem Wurzelknoten und einem beliebigen anderen Knoten R_m ist durch die Anzahl der Kanten definiert, an welchen wir entlanggehen müssen, um den Knoten R_m zu erreichen.
2. Die Tiefe eines Baums ist dann definiert durch den längsten Pfad vom Wurzelknoten zu einem anderen Knoten.

So hat der Baum in Figure 4.1 etwa die Tiefe 4, da wir entlang der rechten Seite des Baums vier Kanten entlanggehen müssen, um entweder beim Knoten mit dem Wert R_6 oder R_7 zu landen. *Beachte: Der längste Pfad muss also nicht eindeutig sein!*

4.2.2.3 Reduktion des Fehlers (`cost_complexity`)

Ein weiteres wichtiges Stoppkriterium ist das Stoppen durch unzureichende Reduzierung des Fehlers. Betrachte hierfür folgenden Ausschnitt aus einem beliebigen Regressionsbaum:



Wir können für jeden der drei Knoten den MSE durch die Formel

$$\text{MSE}_{R_m} = \frac{1}{|R_m|} \sum_{k: x_k \in R_m} (\bar{y}_{R_m} - y_k)^2$$

berechnen, wobei R_m für den entsprechenden Knoten $K_{\text{Eltern}}, K_{\text{Kind}_1}, K_{\text{Kind}_2}$ steht.

Wir sind nun an der relativen Reduzierung des Fehlers durch den Split des Elternknotens in zwei Kindknoten interessiert. Diese Änderung lässt sich mithilfe der MSE Terme durch folgende Formel beschreiben:

$$\begin{aligned} & \frac{|K_{\text{Eltern}}| \cdot \text{MSE}_{K_{\text{Eltern}}} - (|K_{\text{Kind}_1}| \cdot \text{MSE}_{K_{\text{Kind}_1}} + |K_{\text{Kind}_2}| \cdot \text{MSE}_{K_{\text{Kind}_2}})}{|K_{\text{Eltern}}| \cdot \text{MSE}_{K_{\text{Eltern}}}} \\ &= 1 - \frac{|K_{\text{Kind}_1}| \cdot \text{MSE}_{K_{\text{Kind}_1}} + |K_{\text{Kind}_2}| \cdot \text{MSE}_{K_{\text{Kind}_2}}}{|K_{\text{Eltern}}| \cdot \text{MSE}_{K_{\text{Eltern}}}} \end{aligned} \quad (4.1)$$

Wir nennen (4.1) auch den *Improvement*-Wert. Der Quotient beschreibt hierbei, wie groß der Anteil der Fehlerreduktion im Vergleich zum Eltern-Fehler ist. Da die Summe der Teilfehler $(|K_{\text{Kind}_1}| \cdot \text{MSE}_{K_{\text{Kind}_1}} + |K_{\text{Kind}_2}| \cdot \text{MSE}_{K_{\text{Kind}_2}})$ immer kleiner gleich dem Fehler des Elternknotens ist $(|K_{\text{Eltern}}| \cdot \text{MSE}_{K_{\text{Eltern}}})$ ist der Quotient somit immer nicht-negativ und kleiner oder gleich 1. Falls der Zähler in (4.1) einen (im Vergleich zum Nenner) kleinen Wert annimmt, dann erreichen wir durch das Aufsplitten des Elternknotens eine große Verbesserung der Modellperformanz.

Der Improvement-Wert kann somit Werte zwischen 0 und 1 annehmen, wobei ein Improvement-Wert von 1 impliziert, dass der Quotient den Wert 0 besitzt und damit auch die beiden MSE der Kindknoten den Wert 0 annehmen. Falls der Improvement-Wert sehr klein ist, bedeutet das im Umkehrschluss, dass durch das Splitten des Elternknotens keine große Verringerung des Fehlers erreicht wird.

Wir können nun für den Improvement-Wert, ähnlich wie er in (4.1) definiert ist, einen Schwellenwert `cost_complexity` festlegen. Falls ein Improvement-Wert für einen Knoten kleiner ist als der Schwellenwert `cost_complexity` wird dieser Knoten nicht weiter geteilt.

Der Improvement-Wert bezieht sich hierbei aber im Nenner nicht auf den Elternknoten, sondern auf den Wurzelknoten (vgl. Abschnitt 4.4.4).

4.2.2.4 Pruning

Zwar können alle genannten Kriterien verwendet werden, um das Wachsen des Baums frühzeitig zu stoppen (sog. *Prepruning*), allerdings ist das vorzeitige Stoppen durch eine zu kleine Veränderung des Improvement-Wertes zu kurz gegriffen:

Falls auf einen vermeintlich schlechten Split (Improvement-Wert $< \text{cost_complexity}$) ein besserer folgt (Improvement-Wert $> \text{cost_complexity}$), dann wird der bessere Split durch eine vorzeitige Beendigung nicht berücksichtigt. Deshalb wird für gewöhnlich das Aufspannen des Baums **lediglich** durch `min_n` und `tree_depth` vorzeitig gestoppt.

Nachdem ein Baum also auf Basis der anderen Kriterien aufgespannt wurde, wird das Cost-Complexity-Pruning (sog. *Postpruning*) angewendet, um diesen weit aufgespannten Baum zu trimmen:

Wir suchen unter der Verwendung des zuvor festgelegten Parameters `cost_complexity` = α denjenigen Teilbaum, welcher den Term

$$\sum_{m=1}^{|T|} \sum_{k: x_k \in R_m} (y_k - y_{R_m}^-)^2 + \alpha |T| \quad (4.2)$$

minimiert, wobei $|T|$ für die Anzahl der Blattknoten steht.

Falls wir für α einen **sehr kleinen** Wert wählen, sorgt das also dafür, dass der Term $\alpha |T|$ ebenso relativ wenig Einfluss auf den gesamten Term hat und somit ein Baum mit vielen Blattknoten aufgespannt wird. Falls wir für α auf der anderen Seite einen **sehr großen** Wert wählen, dann dominiert der Term $\alpha |T|$ die beiden Summanden, was dazu führen kann, dass ein Baum mit sehr wenigen Blattknoten gefunden wird.

4.3 Wichtigkeit der Variablen in Baummodellen

Im Vergleich zu linearen Modellen können wir die *Koeffizienten* der Regressionsbäume nicht mehr direkt interpretieren. Es gibt allerdings verschiedene Ansätze, welche die Einflussnahme der verschiedenen Variablen im Modell ermöglichen.

Eine Idee wäre, zu zählen, wie oft eine Variable zum Aufteilen des Feature Space verwendet wurde. Allerdings ist diese Idee ohne eine zusätzliche Gewichtung zu ungenau, da das wiederholte Verwenden einer Variable den Einfluss auf die Modellgüte nur unzureichend beschreibt.

Beispiel 4.2. Angenommen, die Variable X_j wird im ersten Split verwendet, was zu einem sehr hohen Improvement-Wert führt. Nun wird die Variable X_j für keinen weiteren Split verwendet. Falls dann eine Variable $X_{\tilde{j}}$ mit $\tilde{j} \neq j$ existiert, welche in mehr als einem Split verwendet wird, aber der durch den Improvement-Wert gemessene Einfluss insgesamt kleiner ist als jener der Variable X_j , dann würde ohne eine Gewichtung die Variable $X_{\tilde{j}}$ trotzdem als wichtiger gelten.

Wir sollten also neben der absoluten Anzahl der Variablenverwendung auch den Improvement-Wert des entsprechenden Splits berücksichtigen.

4.4 Regressionsbäume in R

Nachdem wir uns in den vorherigen Abschnitten mit der Theorie und Darstellung von Entscheidungsbäumen auseinandergesetzt haben, wollen wir in diesem Abschnitt lernen, wie man mit Entscheidungsbäumen in R umgeht.

Hierfür betrachten wir wieder den `penguins`-Datensatz aus den vorherigen Übungen:

```
data_penguin <- palmerpenguins::penguins
```

4.4.1 Schätzung der Parameter

Ziel ist es, die Variable `body_mass_g` mit einem Regressionsbaum zu modellieren. Wir können hierfür die `decision_tree()`-Funktion der `{tidymodels}`-Bibliothek verwenden:

```
tree_spec_penguins <- decision_tree(mode = "regression")
```

Ähnlich wie bei der linearen Regression können wir durch die `decision_tree()`-Funktion einen Entscheidungsbaum spezifizieren. Da Entscheidungsbäume sowohl für Klassifikations- als auch für Regressionsaufgaben verwendet werden können, müssen wir mithilfe des `mode`-Arguments spezifizieren, für welche Art von Problem wir den Entscheidungsbaum verwenden

wollen. Das Aufrufen des `tree_spec_penguins` Objekts zeigt dann die Spezifikation des Entscheidungsbaums:

```
tree_spec_penguins
```

Decision Tree Model Specification (regression)

Computational engine: rpart

Um die Parameter des Baumes zu schätzen, verwenden wir wie auch bei der linearen Regression die `fit()`-Funktion:

```
tree_fit_penguins <- tree_spec_penguins %>%  
  fit(data = data_penguin,  
       formula = body_mass_g ~.  
  )
```

Der Rückgabewert der `fit()`-Funktion ist ein Decision-Tree-Objekt, welches neben der ursprünglichen Spezifikation nun auch ein `fit`-Attribut enthält. Dieses `fit` Attribut enthält Informationen über die Variablen und Schwellenwerte, welche in den jeweiligen Splits verwendet wurden. Durch das Aufrufen der `extract_fit_engine()`-Funktion können wir diese Informationen extrahieren:

```
tree_fit_penguins %>% extract_fit_engine()
```

n=342 (2 observations deleted due to missingness)

```
node), split, n, deviance, yval  
  * denotes terminal node
```

```
1) root 342 219307700 4201.754  
  2) species=Adelie,Chinstrap 219 41488530 3710.731  
    4) sex=female 109 8586055 3420.642 *  
    5) sex=male 110 14640890 3998.182 *  
  3) species=Gentoo 123 31004250 5076.016  
    6) sex=female 61 4853135 4670.492 *  
    7) sex=male 62 6250000 5475.000 *
```

In der ersten Zeile des Outputs wird angezeigt, wie viele Datenpunkte zum Schätzen der Baumparameter verwendet wurden:

```
n=342 (2 observations deleted due to missingness)
```

```
...
```

Da in zwei Beobachtungen fehlende Werte vorhanden sind, hat die `fit`-Funktion diese entfernt und die Parameter auf Basis der verbleibenden 342 Beobachtungen geschätzt.

In der nächsten Zeile ist eine Legende gegeben, welche Informationen über den darauffolgenden Output enthält:

```
...
```

```
node), split, n, deviance, yval
      * denotes terminal node
```

```
...
```

- In der ersten Spalte (`node`) ist die Knotennummer gegeben. Der Knoten mit der Nummer 1) steht hierbei für den Wurzelknoten.
- In der zweiten Spalte (`split`) ist die Splittingvariable X_j gegeben.
- In der dritten Spalte (`n`) ist die Anzahl der Observationen in dem entsprechenden Knoten aufgeführt.
- In der vierten Spalte (`deviance`) ist die quadrierte Abweichung der Observationen im Knoten von den tatsächlichen Werten im Knoten aufgeführt. Diese berechnet sich durch $\text{deviance} = \text{MSE} \cdot n$.
- In der fünften Spalte (`yval`) ist der Schätzwert $\hat{y}$ des entsprechenden Knotens gegeben.
- In der letzten Spalte (`* denotes terminal node`) einer Zeile steht ein *, falls es sich in der entsprechenden Zeile um einen Blattknoten handelt.

```
...
```

```
1) root 342 219307700 4201.754
  2) species=Adelie,Chinstrap 219 41488530 3710.731
    4) sex=female 109 8586055 3420.642 *
    5) sex=male 110 14640890 3998.182 *
  3) species=Gentoo 123 31004250 5076.016
    6) sex=female 61 4853135 4670.492 *
    7) sex=male 62 6250000 5475.000 *
```

```
...
```

Die verbleibenden Zeilen enthalten die zuvor beschriebenen Informationen der Legende. In jeder Zeile steht entsprechend ein Knoten des Baums, wobei die Blattknoten mit einem * in der letzten Zeile versehen sind.

Die erste Splitting-Variable ist durch `species` gegeben. Falls eine Beobachtung die Ausprägungen `species=Adelie` oder `species=Chinstrap` enthält (Knoten 2)), so werden diese zur Splitting-Bedingung `sex=female` (Knoten 4)) bzw. `sex = male` (Knoten 5)) geleitet. Hat eine Beobachtung die Ausprägung `species=Gentoo` (Knoten 3)), dann wird ebenso geprüft, ob der entsprechende Pinguin das Geschlecht `male` (Knoten 7)) oder `female` (Knoten 6)) besitzt.

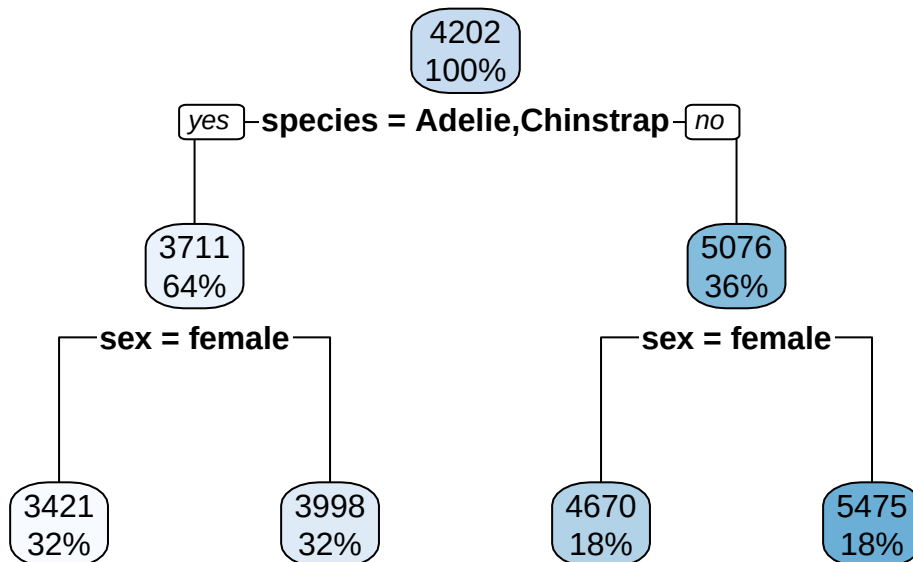
Es wurden in dem gegebenen Baum also lediglich die Variablen `species` und `sex` verwendet, um den Baum aufzuspannen.

4.4.2 Darstellung des Baums

Wir können den geschätzten Baum aus Abschnitt 4.4.1 als Binärbaum mithilfe der `rpart.plot`-Library darstellen²:

```
library(rpart.plot)

tree_fit_penguins %>%
  extract_fit_engine() %>%
  rpart.plot()
```



Das Vorgehen ist hierbei (dank der `{tidymodels}`) Syntax, ähnlich wie beim Extrahieren der Modellparameter eines linearen Modells:

²Diese muss eventuell zuerst mit dem Befehl `install.packages("rpart.plot")` installiert werden

1. Im ersten Schritt übergeben wir das `tree_fit_penguins`-Objekt in die `extract_fit_engine()`-Funktion.
2. Die `extract_fit_engine()`-Funktion extrahiert die Modellparameter, welche dann an die `rpart.plot()` übergeben werden.
3. Die `rpart.plot()`-Funktion visualisiert die Splittingvariablen und Schwellenwerte in einer binären Baumdarstellung.

Der obere Wert in einem Knoten gibt dabei an, wie groß der relative Anteil der Datenpunkte im Vergleich zu allen Datenpunkten auf dieser Ebene des Baums ist. Da auf der Ebene des Wurzelknotens nur ein Knoten ist, ist der relative Anteil in diesem entsprechend 100%. In der zweiten Ebene sind zwei Knoten, wobei der linke Knoten 64% und der rechte Knoten 34% der Datenpunkte enthält. Der untere Wert in einem Knoten gibt den Schätzwert der Zielvariable an. So ist etwa der Schätzwert $\hat{y}$ im linken unteren Blattknoten durch 3421 g gegeben. Unter einem Knoten, welcher kein Blattknoten ist, steht eine Entscheidungsregel. Im Split nach dem Wurzelknoten wird zum Beispiel geprüft, ob ein Pinguin zu den Spezies *Adelie* oder *Chinstrap* gehört.

4.4.3 Wichtigkeit der Variablen

Damit wir die Wichtigkeit der Variablen wie in Abschnitt 4.3 beschrieben berechnen können, verwenden wir das `{vip}`-Paket. `vip` steht in diesem Kontext für *Variable Importance Plots*.

Warning

Das `vip`-Paket() kann nicht über den üblichen Befehl `install.packages("vip")` installiert werden. Eine Installation ist trotzdem mit folgendem Befehl möglich:

```
install.packages(  
  "vip",  
  repos = c(  
    "https://bgreenwell.r-universe.dev",  
    "https://cloud.r-project.org")  
)
```

```
library(vip)
```

Die Funktion `vi()` berechnet für ein gegebenes Baummodell die Wichtigkeit der Variablen. Bei Entscheidungsbäumen ergibt sich die Wichtigkeit einer Variablen aus der Summe der Improvement Werte aller Splits, in denen die Variable verwendet wurde.³

³Streng genommen wird auch noch der Improvement Wert der sog. *Surrogate-Splits* verwendet. Für Details siehe [rpart Dokumentation](#)

Variable	Importance
bill_depth_mm	156249723
flipper_length_mm	156041084
species	146814916
island	94470472
bill_length_mm	75382085
sex	37965241

```
tree_fit_penguins %>%
  extract_fit_engine() %>%
  vi()
```

Auffällig ist hierbei, dass, obwohl der finale Baum nur die Variablen **sex** und **island** enthält, die wichtigsten Variablen **bill_depth_mm** und **flipper_length_mm** sind. Das liegt daran, dass die Wichtigkeit der Variablen auf Basis des ungetrimmten Baumes berechnet wird. Dieses Phänomen ist ein Indikator dafür, dass die Standardwerte **min_n**, **complexity_parameter** und **tree_depth** in diesem Beispiel nicht optimal sein könnten, da die wichtigsten Variablen aus dem finalen Baum entfernt wurden.

4.4.4 Berechnung der Improvement-Werte

Um die Improvement-Werte bzw. die Reduktion der Fehler durch einen Split wie in Abschnitt 4.2.2.3 zu berechnen, können wir auf das **cptable**-Attribut des Baumes zugreifen. Diese können wir durch die Anwendung der **pluck**-Funktion extrahieren:

```
tree_fit_penguins %>%
  extract_fit_engine() %>%
  pluck("cptable")
```

	CP	nsplit	rel error	xerror	xstd
1	0.66944717	0	1.0000000	1.0068986	0.06138148
2	0.09074516	1	0.3305528	0.3341314	0.02213078
3	0.08326927	2	0.2398077	0.2464868	0.01810355
4	0.01000000	3	0.1565384	0.1597440	0.01179351

Hinweis

Anstatt der `extract_fit_engine`- und `pluck`-Funktion kann man die Complexity-Parameter-Tabelle auch durch folgendes Code-Snippet extrahieren:

```
tree_fit_penguins$fit$cptable
```

Aus Konsistenzgründen ist es aber empfehlenswert, die zuvor aufgeführte Tidy-Syntax zu verwenden.

Die `cptable` gibt neben der Fehlerreduktion durch einen Split noch weitere Informationen, welche wir im Folgenden analysieren wollen.

1. Die Spalte **CP** (Complexity Parameter) gibt einen Improvement-Wert des jeweiligen Splits an. In der ersten Zeile wird somit der Improvement-Wert durch das Teilen des Wurzelknotens anhand der Formel in (4.1) berechnet. Die darauffolgenden Improvement-Werte beziehen sich ebenfalls auf die Verbesserung bezüglich des Fehlers des Wurzelknotens, weshalb hier die Formel

$$\frac{|K_{\text{Eltern}}| \cdot \text{MSE}_{K_{\text{Eltern}}} - (|K_{\text{Kind}_1}| \cdot \text{MSE}_{K_{\text{Kind}_1}} + |K_{\text{Kind}_2}| \cdot \text{MSE}_{K_{\text{Kind}_2}})}{|K_{\text{Wurzel}}| \cdot \text{MSE}_{K_{\text{Wurzel}}}} \quad (4.3)$$

verwendet wird.

In Gleichung (4.3) steht im Nenner nicht mehr $|K_{\text{Eltern}}| \cdot \text{MSE}_{K_{\text{Eltern}}}$, sondern $|K_{\text{Wurzel}}| \cdot \text{MSE}_{K_{\text{Wurzel}}}$. Der Wert **CP** beschreibt also den Anteil der Fehlerverbesserung, den ein Split erzeugt, relativ zum Gesamtfehler des Modells ohne jegliche Splits (also nur der Wurzelknoten). Um also etwa den **CP**-Wert in der zweiten Zeile zu berechnen, können wir die tabellarisch-hierarchische Darstellung

```
n=342 (2 observations deleted due to missingness)
```

```
node), split, n, deviance, yval
* denotes terminal node
```

```
1) root 342 219307700 4201.754
 2) species=Adelie,Chinstrap 219 41488530 3710.731
   4) sex=female 109 8586055 3420.642 *
   5) sex=male 110 14640890 3998.182 *
 3) species=Gentoo 123 31004250 5076.016
   6) sex=female 61 4853135 4670.492 *
   7) sex=male 62 6250000 5475.000 *
```

verwenden. Der **CP** Wert berechnet sich dann durch die **deviance** Terme, welche durch das Teilen von Knoten 3) in die Knoten 6) und 7) entstehen:

$$\frac{31004250 - (4853135 + 6250000)}{219307700} = 0.09074517$$

Der letzte Eintrag in der Spalte **CP** (0.01) ist der Default-Parameter der **decision_tree**-Funktion für das Argument **cp** (vgl. Abschnitt 4.2.2.3).

2. Die Spalte **nsplit** beschreibt die Anzahl der Splits. Da der letzte Eintrag 3 ist, heißt das, dass der Featurespace dreimal geteilt wurde.
3. Die Spalte **rel error** beschreibt den relativen Fehler des Entscheidungsbaumes beim entsprechenden Split bezüglich des Wurzelknotens. Dieser kann mithilfe der Formel

$$\text{rel error}_i = \begin{cases} 1, & \text{falls } i = 1 \\ \text{rel error}_{i-1} - \text{CP}_{i-1}, & \text{falls } i > 1 \end{cases}$$

berechnet werden. Um also etwa den **rel err** für den letzten Eintrag ($i = 4$) berechnen wollen, dann lautet die Formel mit eingesetzten Zahlen hierfür

$$0.2398077 - 0.08326927 = 0.1565384$$

4. Die Spalten **xerror** und **xstd** beschreiben den **rel error** und dessen Standardabweichung, welche sich durch eine Cross-Validation ergeben haben.

4.5 Erweiterungen

Baummodelle bilden neben linearen Modellen eine wichtige Grundlage für weiterführende Modelle der Regression, welche zum Standardrepertoire eines Data Scientists gehören. *Random Forests* sind eine dieser Erweiterungen, welche wir in diesem Abschnitt diskutieren wollen.

Die grundlegende Idee kann in zwei Teilen erklärt werden.

4.5.1 Erstellung eines Bootstrap-Datasets

Angenommen, ein Feature-Datensatz besteht aus K Datenpunkten $(x_1, \dots, x_K)$ und J Variablen, also $x_k \in \mathbb{R}^J$ für $k = 1, \dots, K$. Wir nehmen außerdem an, dass zu jedem Datenpunkt x_k mit $k = 1, \dots, K$ ein Zielwert y_k existiert. Der gesamte Datensatz ist somit durch $D = \{(x_k, y_k) | x_k \in \mathbb{R}^J, y \in \mathbb{R}, k = 1, \dots, K\}$ gegeben. Ein Bootstrap-Sample ist dann ein Datensatz $D^{(b)}$ der Größe K , welcher durch zufälliges Ziehen mit Zurücklegen von Samples aus dem Datensatz D generiert wird. Die Datenpunkte in $D^{(b)}$ können also mehrfach im Bootstrap-Datensatz auftauchen!

4.5.2 Trainieren der Baummodelle und Aggregation

Nachdem ein Bootstrap-Sample $D^{(b)}$ erstellt wurde, schätzen wir dann mithilfe dieses Samples einen Regressionsbaum. Beim Schätzen des Baumes verwenden wir allerdings in jedem potenziellen Split nur $m_{\text{try}} < J$ zufällig ausgewählte Variablen, um stark korrelierte Bäume zu vermeiden.

Dieses Vorgehen wiederholen wir dann $B > 1$ mal, sodass wir schlussendlich B Regressionsbäume mithilfe von B Bootstrap-Samples geschätzt haben. Ein Random-Forest-Schätzwert $\hat{y}$ für einen Datenpunkt x berechnet sich dann durch den durchschnittlichen Vorhersagewert aller B Bäume:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B \hat{y}^{(b)},$$

wobei $\hat{y}^{(b)}$ der Vorhersagewert ist, generiert durch Baum b .

4.5.3 Random Forests in R

Ähnlich wie Regressionsbäume können wir auch Random Forests mithilfe des `{tidymodels}`-Pakets schätzen.

```
set.seed(123)

rf_spec_penguins <- rand_forest(
  mode = "regression",
  trees = 100
)

rf_fit_penguins <- rf_spec_penguins %>%
  fit(
    data = data_penguin,
    formula = body_mass_g ~ .
  )
```

Zuerst erstellen wir mithilfe der `rand_forest()`-Funktion eine Modellspezifikation. Als Argumente übergeben wir wieder den Modus `"regression"` und zusätzlich das Argument `trees = 100`, welches spezifiziert, dass $B = 100$. Das Erstellen der Bootstrap-Sample, das Schätzen der Bäume und die Aggregation werden dann im zweiten Schritt durch die `fit()`-Funktion vorgenommen. Als Argumente für die `fit`-Funktion müssen wir wie gewöhnlich die Daten und die Formel übergeben. Dadurch, dass sowohl das Erstellen der Bootstrap-Samples als auch das

Schätzen der Bäume eine Zufallskomponente enthält, sollten wir durch die `set.seed()`-Funktion reproduzierbare Ergebnisse ermöglichen.

Nach dem Schätzen des Random Forests können wir durch das Aufrufen des Objekts `rf_fit_penguins` einige Informationen über den Random Forest extrahieren:

```
rf_fit_penguins
```

```
parsnip model object
```

```
Ranger result
```

```
Call:
```

```
  ranger::ranger(x = maybe_data_frame(x), y = y, num.trees = ~100,      num.threads = 1, verb
```

Type:	Regression
Number of trees:	100
Sample size:	333
Number of independent variables:	7
Mtry:	2
Target node size:	5
Variable importance mode:	none
Splitrule:	variance
OOB prediction error (MSE):	87833.61
R squared (OOB):	0.8645322

- Der Parameter `Mtry` gibt an, wie viele Variablen zufällig für einen Split verwendet werden.
- Der Parameter `Target node size` gibt an, wie viele Datenpunkte mindestens in einem Knoten sein müssen, bevor dieser wieder geteilt wird.
- `OOB prediction error (MSE)` gibt an, wie groß der MSE eines OOS-Datensatzes ist, welcher im Zuge der Schätzung des Random Forests automatisch generiert wird.
- `R squared (OOB)` gibt für das gleiche OOS-Sample den berechneten R^2 an.

Vorhersagen für die gegebenen Sample können wieder mithilfe der `predict()`-Funktion erstellt werden:⁴

```
data_penguin_cleaned <- data_penguin %>% na.omit()

rf_fit_penguins %>%
  predict(data_penguin_cleaned)
```

⁴Exemplarisch wurden hier nur die ersten 5 Werte ausgegeben.

```
# A tibble: 5 x 1
  .pred
  <dbl>
1 3761.
2 3602.
3 3435.
4 3540.
5 3891.
```

4.6 Übungsaufgaben

4.6.1 Theorieaufgaben

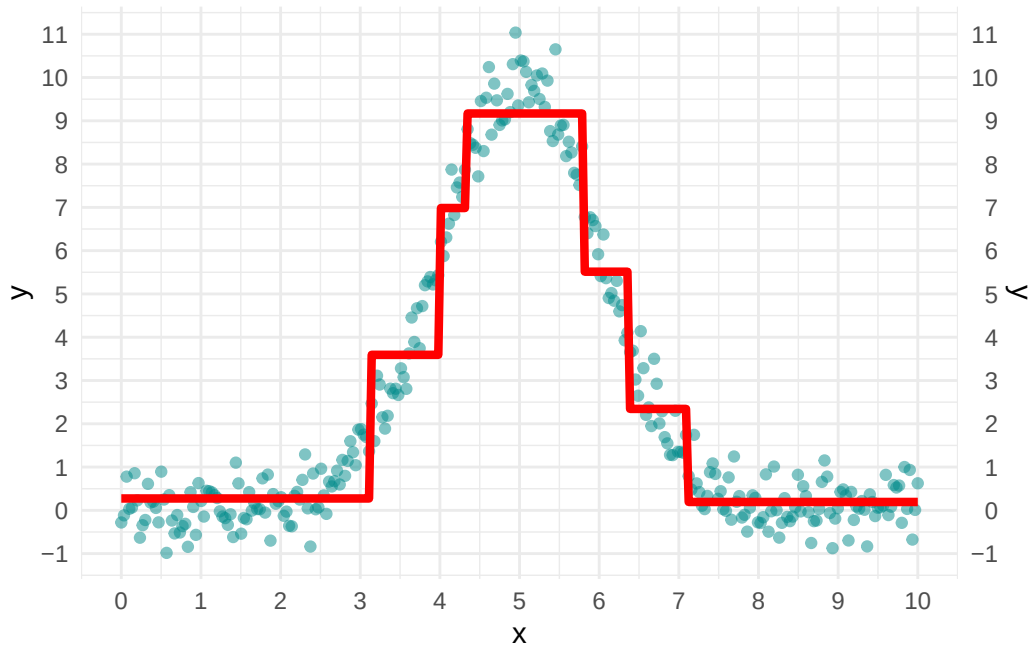
Aufgabe 4.1.

1. Angenommen, ein Regressionsbaum besteht lediglich aus dem Wurzelknoten. Seine Performance wird mit einem linearen Modell verglichen, welches lediglich den Parameter b_0 enthält. Ist zu erwarten, dass das Baummodell den Zusammenhang in den Daten besser erklärt, oder das lineare Modell? Begründe die Antwort.
2. Angenommen, es liegt ein Datensatz vor, welcher lediglich aus einer X - und Y -Variable besteht. Beim Betrachten einer Punktwolke fällt auf, dass der Zusammenhang zwischen den beiden Variablen schon sehr einer Geraden ähnelt. Ist zu erwarten, dass im Vergleich zu einem einfachen linearen Regressionsmodell ein Regressionsbaum den Zusammenhang besser erklärt?

Aufgabe 4.2.

1. Erkläre, warum Cost-Complexity-Tuning sowohl als Pre- als auch als Postpruning-Methode gesehen werden kann.
2. Warum ist es keine gute Idee, Cost-Complexity-Pruning als Prepruning-Methode zu verwenden?

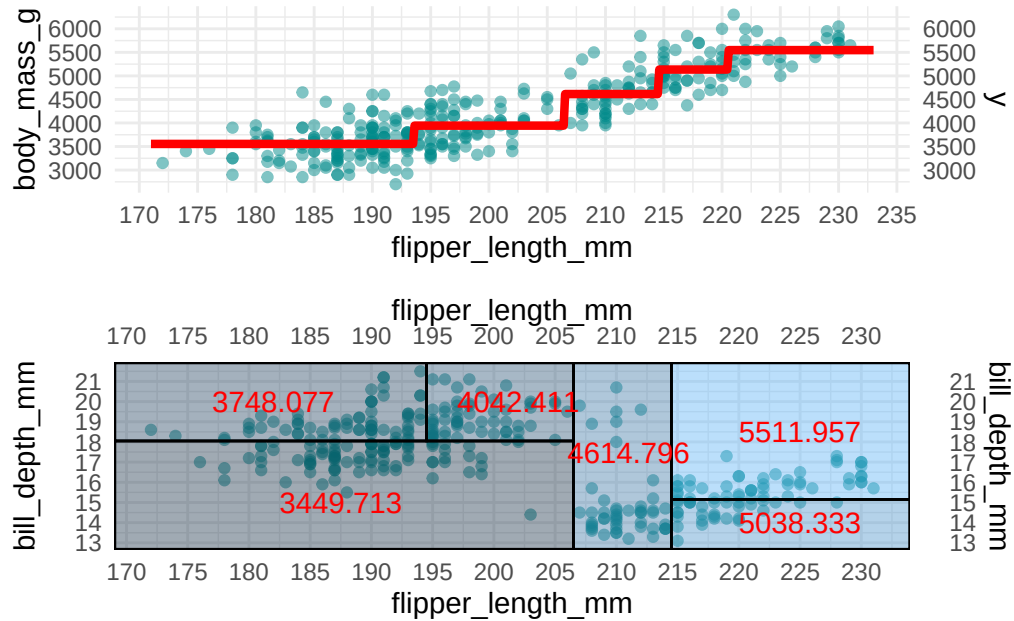
Aufgabe 4.3. Gegeben sei folgende Grafik, welche die Prognosewerte eines Regressionsbaums zeigt.



1. Bestimme, wie viele Blattknoten der abgebildete Baum enthält.
2. Formuliere die Funktionsvorschrift des Regressionsbaums als stückweise definierte Funktion. Runde die visuell abgelesenen Werte auf eine Nachkommastelle.
3. Ist zu erwarten, dass bei einer einfachen linearen Regression eine ähnlich gute Anpassung an die Daten stattfinden würde?

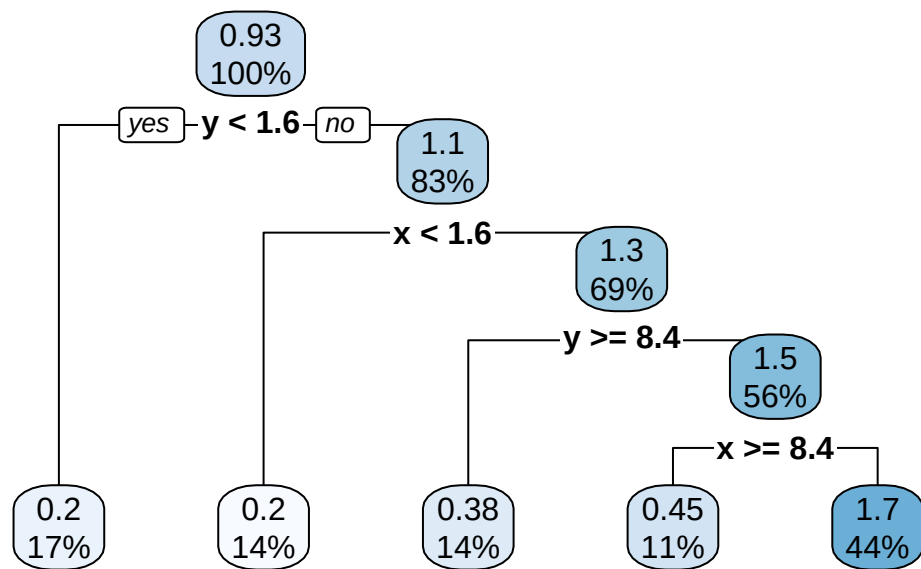
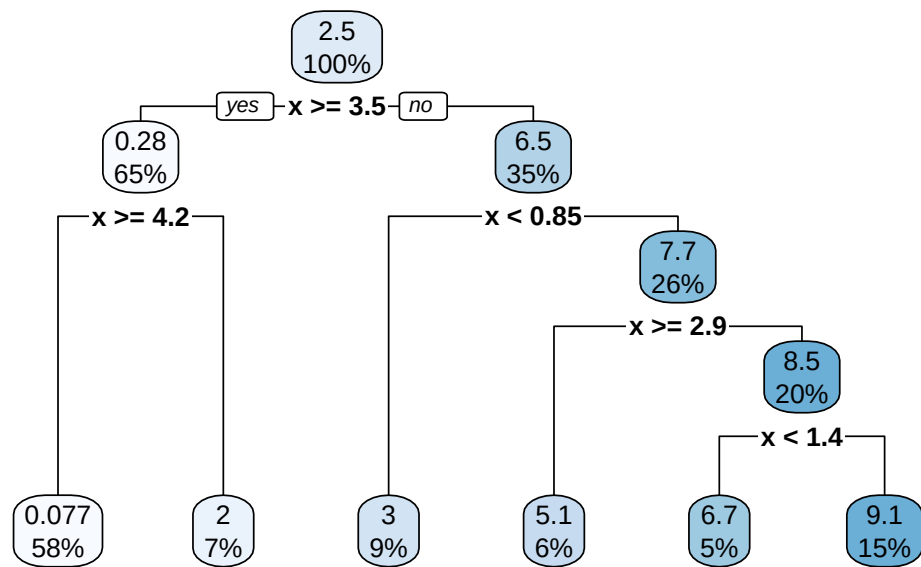
Aufgabe 4.4.

1. Gegeben seien folgende beiden Feature Spaces geschätzter Baummodelle:



Skizziere für beide Abbildungen den entsprechenden Binärbaum unter der Angabe der Splitting-Variable, des Schwellenwertes und des Schätzwertes in den Blattknoten. *Hinweis: In der unteren Abbildung sind die Schätzwerte in den entsprechenden Ausschnitten in Rot abgebildet.*

- Gegeben seien folgende beiden Bäume in Binärbaumdarstellung:



Skizziere für beide Abbildungen die entsprechenden Featurespaces unter der Angabe der Splitting-Variablen, des Schwellenwertes und des Schätzwertes auf den Geraden bzw. in den Regionen.

Aufgabe 4.5. Gegeben seien folgende tabellarische/hierarchische Darstellungen eines Baums:

n= 300

node), split, n, deviance, yval

* denotes terminal node

```
1) root 300 3516.58000 2.4522780
  2) x>=3.461538 196 90.79800 0.2780154 *
  3) x< 3.461538 104 752.97340 6.5499260
    6) x< 0.8528428 26 28.73169 2.9676420 *
    7) x>=0.8528428 78 279.37280 7.7440200 *
```

Außerdem sei folgende CP Tabelle gegeben, welche durch den gleichen Baum erzeugt wurde:

	CP	nsplit	rel error	xerror	xstd
1	a	0	b	1.0080905	0.07565504
2	0.1265061	1	0.2399409	0.2481654	0.02622409
3	0.0500000	2	c	0.1164630	0.01292697

Berechne die fehlenden Werte a, b und c.

4.6.2 R Aufgaben

In dieser Übung verwenden wir den bereits bekannten `hotel_rates`-Datensatz aus den letzten Übungen.

4.6.2.1 Modellvergleiche

Verwende die unten stehende Version des `hotel_rates`-Datensatzes für die folgenden Aufgaben.

```
hotel_rates_filtered <- hotel_rates %>%
  mutate(
    arrival_month = factor(month(arrival_date)),
    num_guests = adults+children,
    is_repeated_guest = factor(is_repeated_guest),
  ) %>%
  select(num_guests, total_of_special_requests,
         required_car_parking_spaces, arrival_month,
```

```
is_repeated_guest, lead_time, stays_in_week_nights,
avg_price_per_room, country) %>%
na.omit()
```

Aufgabe 4.6. Generiere mithilfe des `hotel_rates_filtered`-Datensatzes einen 5-fold Cross-Validation (CV)-Split. Verwende dabei das Seed 123.

Aufgabe 4.7. Verwende das CV-Objekt aus Aufgabe 4.6 um jeweils eine multiple lineare Regression, einen Regressionsbaum und einen Randomforest (bestehend aus 500 Bäumen) zu schätzen, und vergleiche die OOS-Performance anhand der Metriken RMSE und R^2 . Die Zielvariable soll hierbei `avg_price_per_room` sein und die erklärenden Variablen alle verbleibenden. Setze beim Bearbeiten dieser Aufgabe ebenfalls einen Random Seed auf 123, um auch beim Random-Forest-Modell reproduzierbare Ergebnisse zu erzielen.

4.6.2.2 Einfluss der Hyperparameter

In Abschnitt 4.2.2 haben wir gesehen, dass die Parameter `cost_complexity` (CP), `tree_depth` (Baumtiefe) und `min_n` (Mindestanzahl der Datenpunkte für einen weiteren Split) die Form des Baumes stark beeinflussen können.

Wir wollen deshalb in den folgenden Übungsaufgaben diesen Einfluss untersuchen.

```
hotel_rates_adj <- hotel_rates %>%
mutate(
  arrival_month = factor(month(arrival_date)),
  num_guests = adults+children,
  is_repeated_guest = factor(is_repeated_guest),
) %>%
select(-c(adults, children, agent))
```

Aufgabe 4.8. Erstelle mithilfe des oben eingeführten Datensatzes `hotel_rates_adj` einen Trainings- und Test-Split im Verhältnis 5:1 unter Verwendung des Seeds 123.

Aufgabe 4.9. In dieser Aufgabe wollen wir verschiedene Versionen eines Regressionsbaums trainieren. Der Fokus soll hierbei auf der Wahl der verschiedenen Hyperparameter und deren Einfluss auf die Modellperformanz auf den Testdaten liegen.

Verwende für jeden der folgenden beschriebenen Regressionsbäume die Formel

```
formula_hr <- avg_price_per_room~.
```

Hinweis: Man kann in der `fit`-Funktion dem Argument `formula` damit den Wert `formula_hr` zuweisen und muss nicht immer die gleiche Spezifikation ausschreiben.

1.
 - i) Verwende den Trainings-Split aus Aufgabe 4.8 um einen Regressionsbaum mithilfe der Formel `formula_hr` zu trainieren. Bei der Modellspezifikation sollen alle Hyperparameter des Baumes auf deren Standardeinstellung gelassen werden.
 - ii) Stelle den Baum grafisch dar (`rpart.plot()`) und beantworte die folgenden Fragen:
 - a) Welcher Blattknoten enthält die kleinste bzw. größte Anzahl an Datenpunkten?
 - b) Welcher ist der größte bzw. kleinste Schätzwert im dargestellten Baum (bezogen auf alle Knoten)?
 - c) Welche Tiefe hat der resultierende Baum?
 - iii) Bestimme mithilfe der `vip()`-Funktion welche Variable im Modell den größten und kleinsten Importance-Wert besitzt.
 - iv) Berechne auf Basis der Testdaten den RMSE und R^2 .
2. Verwende das Trainingsset aus Aufgabe 4.8 um einen weiteren Regressionsbaum zu trainieren.
 - i) Verwende hierbei die Hyperparameter:

```
tree_params <- tibble(  
  cost_complexity = 0.0000000915,  
  tree_depth = 15,  
  min_n = 32  
)
```

- ii) Welche Tiefe besitzt dieser neue Baum?
 - iii) Berechne für diesen Baum ebenso auf den Testdaten den RMSE und R^2 und vergleiche diese mit der Testperformanz des Baumes aus Teilaufgabe 1. iv.

5 Klassifikation

Neben der Regression beschäftigen wir uns im Zuge des Supervised-Learnings auch mit der Klassifikation. Im Vergleich zu Regressionsaufgaben ist die Zielvariable bei der Klassifikation nicht mehr metrisch, sondern nominal. Es gibt für die Zielvariablen also keine bestimmte Ordnung mehr, und zusätzlich gehen wir auch davon aus, dass die Anzahl der Ausprägungen der Zielvariable endlich ist.

5.1 Einführung

In dieser Veranstaltung setzen wir uns vorwiegend mit der binären Klassifikation auseinander. Die Zielvariable Y nimmt also nur zwei Werte an. Normalerweise kodiert man diese Werte mit 0 und 1, sodass sich Y als Vektor $Y \in \{0, 1\}^K$ darstellen lässt. Wir sind nun daran interessiert, für einen Datenpunkt $x \in \mathbb{R}^J$ die Wahrscheinlichkeit

$$\mathbb{P}(Y = 1 | X = x) \tag{5.1}$$

zu schätzen. Falls die geschätzte Wahrscheinlichkeit $\hat{p}$ größer als ein bestimmter, vordefinierter Schwellenwert $q \in (0, 1)$ ist, dann weisen wir das Sample der Klasse 1 zu. Intuitiv macht hierbei der Wert $q = 0.5$ Sinn, da wir uns bei einer Wahrscheinlichkeit $p > 0.5$ sicherer sind, dass der Datenpunkt zur Klasse 1 als zur Klasse 0 gehört.

5.2 Evaluation eines binären Klassifikationsmodells

Da eine Evaluation mithilfe von Metriken wie RMSE und R^2 bei der binären Klassifikation wenig Sinn ergibt, wollen wir in diesem Abschnitt verschiedene alternative Metriken betrachten.

5.2.1 Confusionmatrix

Als Grundlage für die Metriken, welche wir im Folgenden betrachten wollen, ist die sogenannte *Confusion Matrix*. Diese Kreuztabelle gibt an, wie viele Datenpunkte korrekt bzw. inkorrekt klassifiziert wurden. Hierbei wird insbesondere zwischen den Klassen *Negativ* und *Positiv* unterschieden. Im vorherigen Abschnitt wurden die Klassen mit 1 und 0 encodiert. Meistens steht 1 im Kontext der Confusion Matrix für **Positive** und 0 für **Negative**.

		Predicted Condition	
		Predicted Positive (PP)	Predicted Negative (PN)
Actual Condition	Positive (P)	True Positive (TP)	False Negative (FN)
	Negative (N)	False Positive (FP)	True Negative (TN)

Figure 5.1

Die Felder der Confusion Matrix können wir am besten anhand eines Beispiels illustrieren.

Beispiel 5.1. Ein Forscherteam des Universitätsklinikums hat einen Test für das Feststellen einer seltenen Herzkrankheit entwickelt. Falls die Ärzte diese Herzkrankheit korrekt feststellen, kann diese durch einen operativen Eingriff geheilt werden. Die Wahrscheinlichkeit eines erfolgreichen Eingriffs ist hierbei größer als die Wahrscheinlichkeit, dass eine kranke Person den Krankheitsverlauf überlebt. Ein positives Testergebnis wird mit 1 bzw. **Predicted Positive** encodiert und ein negatives Ergebnis mit 0 bzw. **Predicted Negative**. Nun gibt es folgende Möglichkeiten:

1. Der Test ist positiv und die untersuchte Person hat die Krankheit. Dann spricht man von einem **True Positive**.
2. Der Test ist positiv und die untersuchte Person hat die Krankheit nicht. Dann spricht man von einem **False Positive**.
3. Der Test ist negativ und die untersuchte Person hat die Krankheit nicht. Dann spricht man von einem **True Negative**.
4. Der Test ist negativ und die untersuchte Person hat die Krankheit. Dann spricht man von einem **False Negative**.

Neben einer korrekten Klassifikation (TP und TN) sollten wir uns besonders mit den beiden fehlerhaften Klassifikationsszenarien auseinandersetzen. In Beispiel 5.1 ist eine fälschlicherweise als negativ klassifizierte Person (FN) also ein Patient oder eine Patientin, welche die Herzkrankheit besitzt, aber nicht diagnostiziert wird. Diese Art von Fehler ist im Beispiel

besonders gravierend, da die Person eine höhere Überlebenswahrscheinlichkeit durch den operativen Eingriff besitzt. Falls eine Person fälschlicherweise als positiv klassifiziert wird (FP), dann bedeutet das, dass der Test positiv ausfällt, obwohl die Person nicht an der Herzkrankheit erkrankt ist. Unter gewöhnlichen Umständen wäre eine solche Fehlklassifikation nicht ganz so gravierend, da die Person schließlich nicht an der seltenen Krankheit erkrankt ist. Falls allerdings auf Basis des Testergebnisses trotzdem ein operativer Eingriff vorgenommen wird, dann setzt man den Patienten oder die Patientin einem unnötigen Risiko aus.

Wie gut also ein solcher Test tatsächlich funktioniert, sollte man also nicht nur an der absoluten oder relativen Anzahl der korrekt klassifizierten Datenpunkte festmachen, sondern auch daran, wie hoch die entsprechenden Fehlerquoten sind.

Präziser lässt sich diese Aussage mit folgenden Kennzahlen ausdrücken:

- Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Die Accuracy misst den relativen Anteil der korrekt klassifizierten Datenpunkte an der Gesamtanzahl der Datenpunkte. Sie beschreibt die Wahrscheinlichkeit für ein korrektes Testergebnis.

- Sensitivität:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

Die Sensitivität misst den relativen Anteil der als korrekt positiv klassifizierten Datenpunkte an der Gesamtanzahl der positiven Datenpunkte. Sie beschreibt die bedingte Wahrscheinlichkeit, dass ein positives Ergebnis vorhergesagt wird, unter der Annahme, dass das Ergebnis tatsächlich positiv ist.

- Spezifität:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Die Spezifität ist das Gegenstück zur Sensitivität. Sie misst den relativen Anteil der korrekterweise negativ klassifizierten Datenpunkte an der Gesamtanzahl der negativen Datenpunkte. Spezifität kann auch als die bedingte Wahrscheinlichkeit interpretiert werden, dass ein negatives Ergebnis vorhergesagt wird, unter der Annahme, dass das Ergebnis tatsächlich negativ ist.

- Präzision:

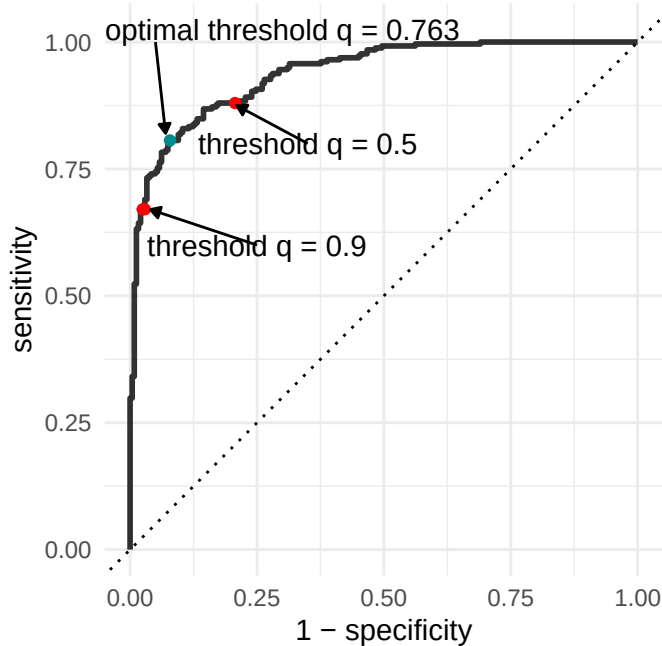
$$\text{Precision} = \frac{TP}{TP + FP}$$

Die Präzision misst den relativen Anteil der korrekterweise als positiv klassifizierten Datenpunkte an der Gesamtzahl der als positiv klassifizierten Datenpunkte. Besonders bei unbalancierten Datensätzen¹ ist die Präzision eine wichtige Metrik, da die Accuracy bei unbalancierten Datensätzen zu Trugschlüssen führen kann.

¹Datensätze, bei welchen weitaus mehr Negatives als Positives vertreten sind

5.2.2 ROC-Kurve

In Abschnitt 5.1 haben wir kurz den Entscheidungsprozess bei einer Klassifikation in Abhängigkeit des Schwellenwerts q eingeführt. Da der Schwellenwert q allerdings stark die Confusion-Matrix beeinflussen kann, sollte man sich verschiedene Schwellenwerte in Abhängigkeit von ausgewählten Metriken betrachten. Eine dieser Gegenüberstellungen ist durch die sogenannte *Receiver-Operator-Characteristic*-Kurve gegeben. Diese zeigt auf der x -Achse die False-Positive-Rate ($1 - \text{specificity}$) und auf der y -Achse die Sensitivität an. Die Kurve ergibt sich dann durch das Variieren des Schwellenwerts q :



Die ROC-Kurve ist monoton steigend, d. h., je größer die False-Positive-Rate ($1 - \text{specificity}$) ist, desto größer wird die Sensitivität. Der optimale Threshold ist dann durch jenen Punkt gegeben, welcher am nächsten zum Punkt $(0,1)$ ist. Ein Entscheidungsschwellenwert von 0.5 ist also nicht zwingend der optimale Schwellenwert für eine Entscheidung, wenn das Ziel ist, gleichzeitig die Sensitivität zu maximieren und die False-Positive-Rate zu minimieren.

Die Winkelhalbierende beschreibt den Fall, dass die Sensitivität gleich der False-Positive-Rate ist. Falls diese beiden Metriken gleich sind, dann kann das unterliegende Modell nicht zwischen True Positives und False Positives unterscheiden. Dieser Fall wird dann auch gerne als *Münzwurf* beschrieben, da die Wahrscheinlichkeit für eine korrekte, bzw. falsche Klassifizierung 50% beträgt.

Die ROC-Kurve kann allerdings auch irreführend sein, weshalb man neben dieser Metrik auch noch andere Metriken betrachten sollte. Besonders, falls die Klassen unbalanciert sind, kann

die ROC-Kurve irreführend sein, da sie keine Informationen bezüglich der Präzision des Modells enthält.

Neben dem Ablesen der Sensitivität und der False-Positive-Rate an der Kurve selbst können wir auch die Fläche unter der Kurve berechnen. Die sogenannte Area under the Receiver Operator Characteristic (AUROC) nimmt dabei Werte zwischen 0 und 1 an, wobei 1 einem perfekten Modell entspricht und 0 einem Modell, welches alle Vorhersagen falsch trifft.

5.3 Die Logistische Regression

Als Erweiterung der linearen Regression erlaubt es uns die logistische Regression ebenso eine einfache Parametrisierung der geschätzten Wahrscheinlichkeit $\hat{p}$. Betrachten wir zuerst den Fall, dass unser Datensatz aus lediglich einer unabhängigen Variable besteht. Im Falle der linearen Regression schätzen wir dann die Parameter b_0 und b_1 der Gleichung (vgl. Gleichung 2.1).

$$Y = b_0 + b_1 X + \varepsilon$$

5.3.1 Modellgleichung

Wir können diese Modellgleichung dann mithilfe der logistischen Funktion

$$\sigma(x) = \frac{1}{1 + \exp(-x)} \in (0, 1), x \in \mathbb{R}$$

so transformieren, dass die Werte der Regressionsgleichung zwischen 0 und 1 liegen. Die Werte dieser transformierten Regressionsgleichung können dann als Wahrscheinlichkeiten interpretiert werden. Wir erhalten also nach dem Schätzen der Parameter die Modellgleichung

$$\mathbb{P}(Y = 1 | X = x) = \frac{1}{1 + \exp(-(b_0 + b_1 x))}$$

Diese Schätzgleichung lässt sich dann analog zum MLR-Modell erweitern zu

$$\mathbb{P}(Y = 1 | X = x_k) = \frac{1}{1 + \exp\left(-(b_0 + \sum_{j=1}^J b_j x_{k,j})\right)}$$

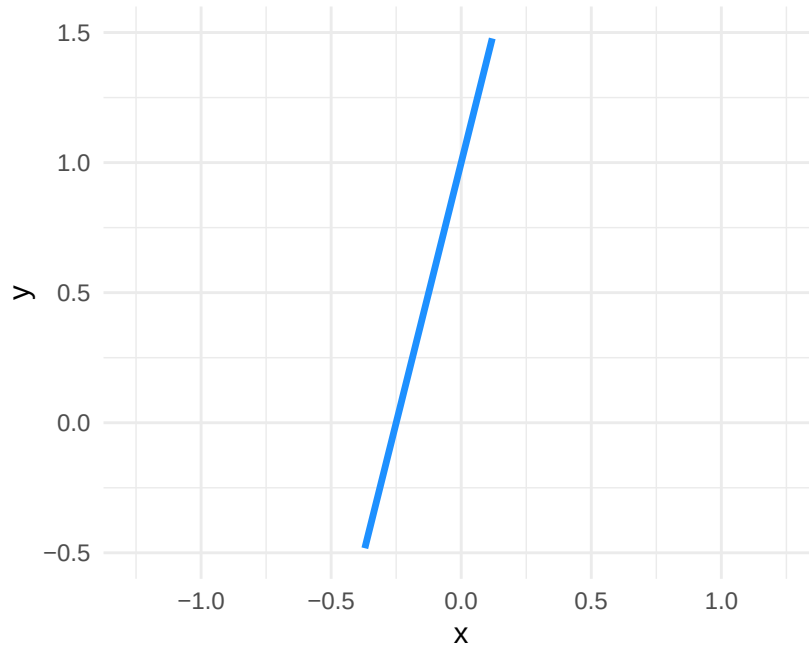
Die Parameter dieser Schätzgleichung werden mithilfe der Maximum-Likelihood-Methode geschätzt, da eine analytische Lösung wie für die Parameter des MLR nur schwer zu berechnen ist.

Beispiel 5.2. Betrachten wir folgendes Beispiel:

Die Schätzgleichung einer einfachen linearen Regression sei durch

$$y = \hat{b}_0 + \hat{b}_1 x = 4x + 1$$

gegeben:

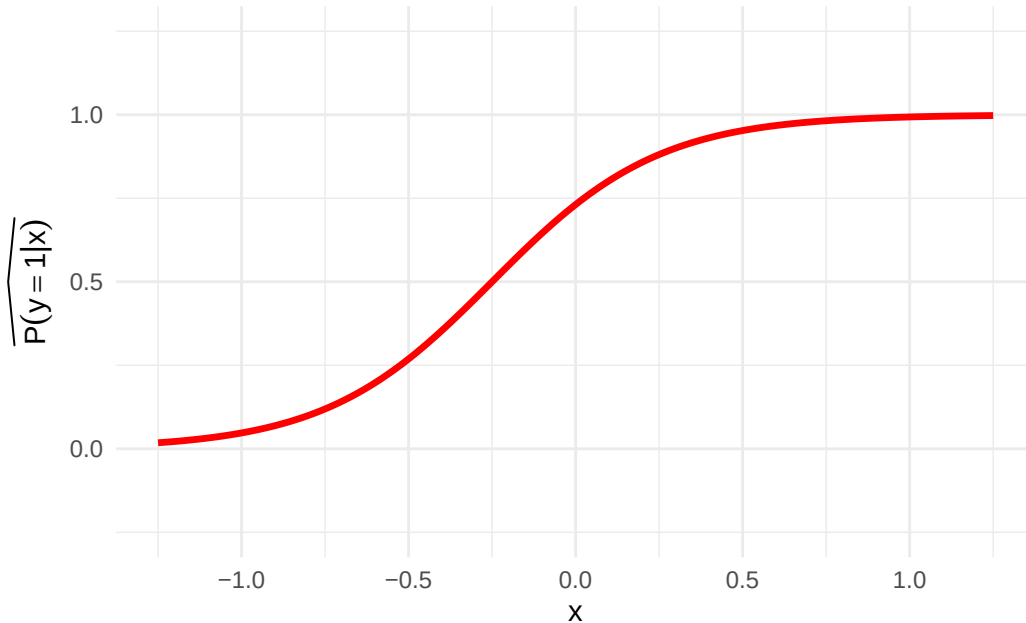


Dann eignet sich diese Funktion nicht, um eine Wahrscheinlichkeit vorherzusagen, da die Funktion für $x < -0.25$ negative Werte annimmt und für $x > 0$ Werte annimmt, welche größer als 1 sind.

Durch eine Transformation dieser Schätzgleichung zu einer logistischen Schätzgleichung der Form

$$\mathbb{P}(\widehat{y = 1} | x) = \frac{1}{1 + \exp(-(\hat{b}_0 + \hat{b}_1 x))} = \frac{1}{1 + \exp(-(4x + 1))}$$

erreichen wir, dass die Funktionswerte nur noch im Intervall $(0, 1)$ liegen:



Je größer also die Werte für x sind, desto höher ist die Wahrscheinlichkeit, dass wir die Klassifikation $y = 1$ annehmen.

5.3.2 Interpretation der Koeffizienten

Im Vergleich zur multiplen linearen Regression lassen sich die Koeffizienten des logistischen Modells nicht mehr direkt interpretieren. Auch die Überprüfung der Signifikanz einzelner Variablen ist nicht mehr mithilfe des t -Tests umsetzbar.

Grundlegend können wir allerdings folgende Aussagen treffen: Das Vorzeichen eines geschätzten Parameters gibt Auskunft darüber, ob die geschätzte Wahrscheinlichkeit beim Erhöhen der Variable (um eine Einheit) steigt oder sinkt.

- Falls $b_j > 0$, $j \in \{1, \dots, J\}$, so steigt die geschätzte Wahrscheinlichkeit $\widehat{\mathbb{P}(Y = 1|X)}$.
- Falls $b_j < 0$, $j \in \{1, \dots, J\}$, dann sinkt die geschätzte Wahrscheinlichkeit $\widehat{\mathbb{P}(Y = 1|X)}$.

Um den Einfluss einer Variable zu quantifizieren, betrachten wir die sogenannten *Odds*. Die Odds sind definiert als

$$\text{Odds} = \frac{\mathbb{P}(Y = 1|X = x)}{\mathbb{P}(Y = 0|X = x)} = \exp(b_0 + \sum_{j=1}^J b_j x_j) \quad (5.2)$$

Die Odds können Werte im Intervall $(0, \infty)$ annehmen. Sie können interpretiert werden als die Chance dessen, dass ein Event eintritt ($y = 1$), gegenüber dem Nichteintreten des Events ($y = 0$).

Beispiel 5.3. Bei einem Wettanbieter für Fußballwetten sind für ein Spiel der Mannschaft A gegen Mannschaft B die dezimalen Odds (Quoten) für einen Sieg von Mannschaft A mit 1.75 und für einen Sieg von Mannschaft B mit 2.10 gegeben. Es wird angenommen, dass kein Unentschieden möglich ist. Die dezimalen Odds geben an, wie viel ein Spieler für einen eingesetzten Euro im Falle eines Gewinns zurückerhält (inklusive des Einsatzes). Die vom Wettanbieter implizierten Wahrscheinlichkeiten werden wie folgt berechnet:

1. Die implizierte Wahrscheinlichkeit für einen Sieg von A ist:

$$\mathbb{P}_{\text{impl}}(A) = \frac{1}{\text{Odds}(A)} = \frac{1}{1.75} \approx 0.5714$$

2. Die implizierte Wahrscheinlichkeit für einen Sieg von B ist:

$$\mathbb{P}_{\text{impl}}(B) = \frac{1}{\text{Odds}(B)} = \frac{1}{2.10} \approx 0.4762$$

Die Summe der implizierten Wahrscheinlichkeiten ist:

$$\mathbb{P}_{\text{impl}}(A) + \mathbb{P}_{\text{impl}}(B) \approx 0.5714 + 0.4762 = 1.0476$$

Diese Summe ist größer als 1, was die Wettmarge (Overround) des Anbieters widerspiegelt. Die tatsächlichen Wahrscheinlichkeiten, die der Wettanbieter zugrunde legt (nach Bereinigung der Marge), erhält man durch Normalisierung:

$$\mathbb{P}(A) = \frac{\mathbb{P}_{\text{impl}}(A)}{\mathbb{P}_{\text{impl}}(A) + \mathbb{P}_{\text{impl}}(B)} = \frac{0.5714}{1.0476} \approx 0.5455$$

$$\mathbb{P}(B) = \frac{\mathbb{P}_{\text{impl}}(B)}{\mathbb{P}_{\text{impl}}(A) + \mathbb{P}_{\text{impl}}(B)} = \frac{0.4762}{1.0476} \approx 0.4545$$

Somit rechnet der Wettanbieter mit einer Wahrscheinlichkeit von ca. 54.55%, dass Mannschaft A gewinnt, und mit ca. 45.45%, dass Mannschaft B gewinnt.

Falls Odds = 1, dann sind beide Events gleich wahrscheinlich, während Odds > 1 bzw. < 1 implizieren, dass die Wahrscheinlichkeit für das Eintreten des untersuchten Events > 50% bzw. < 50% ist.

Der Einfluss der Modellparameter auf die Odds kann ebenso interpretiert werden:

- Steigt eine Variable X_j um eine Einheit, so erhöhen bzw. sinken die Odds um $\exp(b_j)$.

Falls zum Beispiel $\exp(\hat{b}_j) = 2$, so verdoppeln sich die Odds beim Erhöhen der Variable X_j um eine Einheit.

5.3.3 Statistische Signifikanz

Im Vergleich zur linearen Regression können wir bei der logistischen Regression den t -Test bzw. F -Test nicht auf ein geschätztes Modell anwenden.

Für die geschätzten Koeffizienten verwendet man den *Wald*-Test. Hierbei werden die Hypothesen

$$H_0 : b_j = 0 \quad \text{vs.} \quad H_1 : b_j \neq 0$$

geprüft. Die Teststatistik ist durch $W = \frac{\hat{b}_j}{\text{sd}(\hat{b}_j)}$ gegeben und zum Berechnen des p -Werts verwenden wir die Standardnormalverteilung.

Die statistische Signifikanz des Gesamtmodells können wir zum Beispiel mithilfe des Likelihood-Ratio-Tests evaluieren. Das Hypothesenpaar ist durch

$$H_0 : b_j = 0 \quad \forall j = 1, \dots, J \quad \text{vs.} \quad H_1 : b_j \neq 0 \quad \text{für mind. ein } j.$$

gegeben.

Die Teststatistik wird durch den Term

$$\nu = -LL_v$$

wobei $-LL_v$ die Log-Likelihood des Vollmodells ist, also

$$LL_v = \sum_{k=1}^K \hat{y}_k \log(\mathbb{P}(Y = \widehat{y_k} | X = x_k)) + (1 - \hat{y}_k) \log(1 - \mathbb{P}(Y = \widehat{y_k} | X = x_k))$$

Unter der Annahme, dass $\nu \sim \chi_{n-k}^2$ ist, kann dann der entsprechende p -Wert berechnet werden.

5.4 Klassifikationsbäume

In Chapter 4 haben wir uns bereits intensiv mit Regressionsbäumen auseinandergesetzt. Der Übergang von Regressions- zu Klassifikationsbäumen ist letztlich auch nicht mehr komplex! Anstelle einer reellwertigen Zielvariable betrachten wir in den Blattknoten nun geschätzte Klassen bzw. Klassenwahrscheinlichkeiten. Der Algorithmus zum Schätzen eines Klassifikationsbaums ist hierbei im Vergleich zu Regressionsbäumen fast unverändert. Lediglich der Improvement-Wert, welcher bei Regressionsbäumen mithilfe des MSE berechnet wird, muss

durch eine passende Metrik im Klassifikationskontext ersetzt werden. Diese Metrik ist durch die Gini-Impurity gegeben. Für einen Knoten K_n ist diese definiert als

$$1 - (p_1^2 + p_0^2)$$

wobei p_1 die relative Häufigkeit der Klasse 1 im Knoten ist und p_0 die relative Häufigkeit der Klasse 0 im Knoten. Es wird dann getestet, für welches p_i , $i = 0, 1$ das Entfernen zu einem größeren Impurity Index führt. D. h., falls $p_1 \geq p_0$, wird der Knoten als Klassifikationswert Klasse 1 zurückgegeben. Die Entscheidung bezüglich der Splittingvariablen funktioniert gleich: Teste, für welche Variable sich die größte Reduktion der Gini-Impurity ergibt, und verwende diese für einen weiteren Split.

Die Wichtigkeit der Variablen kann dann ebenso bezüglich dieses Impurity-Wertes gemessen werden, sodass wir effektiv die gleiche Permutation-Feature-Importance-Methode verwenden können.

5.5 Klassifikation in R

In diesem Abschnitt wollen wir die verschiedenen Klassifizierungsalgorithmen und deren Evaluation in R betrachten. Als Grundlage für diese Einführung dient der `penguins`-Datensatz, welchen wir auch schon in vorherigen Übungen behandelt haben. Ziel wird es sein, die Variable Geschlecht vorherzusagen, da diese im Datensatz binär mit "male" und "female" encodiert sind.

```
data_penguin <- palmerpenguins::penguins %>%
  select(-year) %>%
  na.omit() %>%
  mutate(sex = factor(
    if_else(sex=="male",1,0)
  )
)
```

Nachdem wir den Datensatz als `data_penguin` gespeichert haben, entfernen wir alle `NA`-Werte und wandeln mithilfe der `mutate()`-Funktion die Variable `sex` in eine Faktorvariable um. Die Ausprägungen setzen hierbei auf 1, falls ein Pinguin männlich ist, und 0, falls ein Pinguin weiblich ist.

5.5.1 Logistische Regression

Ein logistisches Modell können wir mithilfe der `logistic_reg()`-Funktion spezifizieren. Nach dem Spezifizieren können wir dann die Modellparameter einfach wieder mithilfe der `fit()`-Funktion schätzen. Als Argumente übergeben wir der `fit()`-Funktion die Daten selbst und die Formel, bezüglich welcher die Parameter geschätzt werden sollen. Da wir die Variable `sex` prognostizieren wollen, verwenden wir deshalb die Formel `sex ~ .`.

```
log_spec <- logistic_reg()

log_fit <- log_spec %>%
  fit(data = data_penguin,
      formula = sex ~.)
```

Nach dem Schätzen der Parameter können wir wieder mithilfe der `extract_fit_engine()`-Funktion die Parameter extrahieren und die Modellzusammenfassung durch die `summary()`-Funktion erzeugen.

```
log_fit %>% extract_fit_engine() %>%
  summary()
```

Call:

```
stats::glm(formula = sex ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-80.376672	12.329735	-6.519	7.08e-11	***
speciesChinstrap	-7.402697	1.662534	-4.453	8.48e-06	***
speciesGentoo	-8.427611	2.597027	-3.245	0.00117	**
islandDream	0.324158	0.809135	0.401	0.68870	
islandTorgersen	-0.507858	0.855746	-0.593	0.55287	
bill_length_mm	0.614436	0.131968	4.656	3.22e-06	***
bill_depth_mm	1.646446	0.335798	4.903	9.43e-07	***
flipper_length_mm	0.026654	0.048307	0.552	0.58111	
body_mass_g	0.005819	0.001087	5.352	8.71e-08	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 461.61 on 332 degrees of freedom

Residual deviance: 126.05 on 324 degrees of freedom
AIC: 144.05

Number of Fisher Scoring iterations: 7

Die Variable `island` scheint nicht statistisch signifikant zum Niveau $\alpha = 0.1$ zu sein, was auch den Erwartungen entspricht, da auf jeder Insel sowohl männliche als auch weibliche Pinguine leben. Allerdings stellt sich dann natürlich wieder die Frage, warum die Variable `species` statistisch signifikant ist! Eine mögliche Ursache wäre Multikollinearität, wobei wir das an dieser Stelle nicht weiter untersuchen wollen. Außerdem ist die Variable `flipper_length_mm` nicht statistisch signifikant zum Niveau $\alpha = 0.1$. Ein Grund hierfür könnte sein, dass die durchschnittliche Länge der Flossen bei männlichen und weiblichen Pinguinen gleich groß ist.

Um das Modell zu evaluieren, sollten wir zunächst eine Confusion Matrix erzeugen. Dafür müssen wir zunächst Vorhersagen generieren. Wir wenden dafür die `augment()`-Funktion direkt auf das `log_fit`-Objekt unter Verwendung des Arguments `data_penguin` an. Der Rückgabewert ist ein Tibble, welches neben dem übergebenen Datensatz noch die Spalten `.pred_class`, `.pred_0` und `.pred_1` enthält.

- Die Spalte `.pred_class` enthält die vorhergesagte Klasse, ermittelt durch den Schwellenwert $q = 0.5$.
- Die Spalte `.pred_0` enthält die geschätzte Wahrscheinlichkeit, dass der Datenpunkt zur Klasse 0 (`sex = "female"`) gehört.
- Die Spalte `.pred_1` enthält die geschätzte Wahrscheinlichkeit, dass der Datenpunkt zur Klasse 1 (`sex = "male"`) gehört.

```
log_fit %>%  
  augment(data_penguin) %>%  
  glimpse()
```

```
Rows: 333  
Columns: 10  
$ .pred_class      <fct> 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1~  
$ .pred_0          <dbl> 3.604544e-01, 7.103332e-01, 9.151390e-01, 7.923568e-~  
~  
$ .pred_1          <dbl> 0.639545554, 0.289666770, 0.084861038, 0.207643155, ~  
$ species          <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~  
$ island           <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~  
$ bill_length_mm   <dbl> 39.1, 39.5, 40.3, 36.7, 39.3, 38.9, 39.2, 41.1, 38.6~  
$ bill_depth_mm    <dbl> 18.7, 17.4, 18.0, 19.3, 20.6, 17.8, 19.6, 17.6, 21.2~  
$ flipper_length_mm <int> 181, 186, 195, 193, 190, 181, 195, 182, 191, 198, 18~  
$ body_mass_g      <int> 3750, 3800, 3250, 3450, 3650, 3625, 4675, 3200, 3800~  
$ sex              <fct> 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0~
```


Um nun eine Confusion-Matrix zu erzeugen, selektieren wir zuerst die relevanten Spalten `.pred_class` und `sex` und übergeben diese der `table()`-Funktion. Die `table()`-Funktion erzeugt eine Kreuztabelle, welche einer Confusion Matrix entspricht.

```
log_fit %>%
  augment(data_penguin) %>%
  select(.pred_class, sex) %>%
  table()
```

```
      sex
.pred_class 0  1
0    153  13
1     12 155
```

Nun haben wir allerdings noch folgendes Problem: In Figure 5.1 stehen die Positives in der ersten Zeile bzw. der ersten Spalte. Die resultierende Confusion-Matrix zeigt die Werte allerdings sortiert nach Zeilen- bzw. Spaltennamen an. Wir können diese aber einfach neu anordnen, indem wir den Befehl auf die zuvor selektierten Spalten anwenden:

```
mutate(across(c(.pred_class, sex), ~ factor(., levels = c("1", "0"))))
```

Da die Spalten `.pred_class` und `sex` Faktorvariablen sind, können wir die Werte mithilfe des `levels`-Arguments neu sortieren. Durch das Spezifizieren von `levels = c("1", "0")` erreichen wir, dass die Reihenfolge der Zeilen und Spalten getauscht wird. Es gilt also nach dieser Transformation `"1"<"0"`.

```
cm_lr <- log_fit %>%
  augment(data_penguin) %>%
  select(.pred_class, sex) %>%
  mutate(across(c(.pred_class, sex), ~ factor(., levels = c("1", "0")))) %>%
  table()
```

```
cm_lr
```

```
      sex
.pred_class 1  0
1    155  12
0     13 153
```

Nun haben wir zwar erreicht, dass die Einträge richtig sortiert sind, allerdings spezifizieren die Spalten in Figure 5.1 die Predictions und die Zeilen die tatsächlichen Werte.

Um die Zeilen und Spalten zu vertauschen, können wir die resultierende Tabelle einfach mithilfe der `t()`-Funktion transponieren:

```
cm_lr <- cm_lr %>% t()

cm_lr
```

```
      .pred_class
sex    1      0
  1 155    13
  0   12 153
```

Auf Basis dieser Confusion Matrix ergeben sich nun folgende Metriken:

Metric	Value
Accuracy	0.924
Sensitivity	0.923
Specificity	0.927
Precision	0.928

Die oben aufgeführten Metriken können wir natürlich auch mithilfe von R berechnen. Damit wir nicht jede Metrik einzeln berechnen müssen, erstellen wir ein `metric_set`-Objekt mithilfe der `metric_set()`-Funktion, welcher wir die entsprechenden Metriken als Argumente übergeben. Um das Modell dann auf Basis dieser Menge an Metriken zu evaluieren, übergeben wir dem Objekt die Argumente `truth = sex` und `estimate = .pred_class`. Wichtig ist, dass wir wieder die Reihenfolge der Ausprägungen anpassen, um die Darstellung der Confusion-Matrix aus Figure 5.1 zu erhalten:

```
multi_metric <- metric_set(accuracy,sensitivity,specificity,precision)

lr_metrics <- log_fit %>%
  augment(data_penguin) %>%
  mutate(across(c(.pred_class, sex), ~ factor(., levels = c("1", "0")))) %>%
  multi_metric(truth = sex,
               estimate = .pred_class)

lr_metrics
```

```
# A tibble: 4 x 3
  .metric      .estimator .estimate
  <chr>        <chr>      <dbl>
1 accuracy    binary      0.925
2 sensitivity binary      0.923
3 specificity binary      0.927
4 precision   binary      0.928
```

Die eben berechneten Metriken basieren alle auf dem Schwellenwert $q = 0.5$. Wir sollten deshalb auch die ROC-Kurve betrachten, um ein Gefühl für verschiedene Schwellenwerte zu bekommen. Die Werte der ROC-Kurve können wir mithilfe der `roc_curve()`-Funktion berechnen: Als Argument übergeben wir der Funktion die geschätzten Wahrscheinlichkeiten, dass ein Sample zur Klasse 1 gehört, und die Variable `sex`, welche die tatsächlichen Werte enthält.

Der Rückgabewert ist dann ein Tibble, welches neben dem Schwellenwert q , gegeben durch die Spalte `.threshold`, die beiden Spalten `specificity` und `sensitivity` enthält.

```
lr_roc <- log_fit %>%
  augment(data_penguin) %>%
  mutate(across(c(.pred_class, sex), ~ factor(., levels = c("1", "0")))) %>%
  roc_curve(.pred_1, truth = sex)

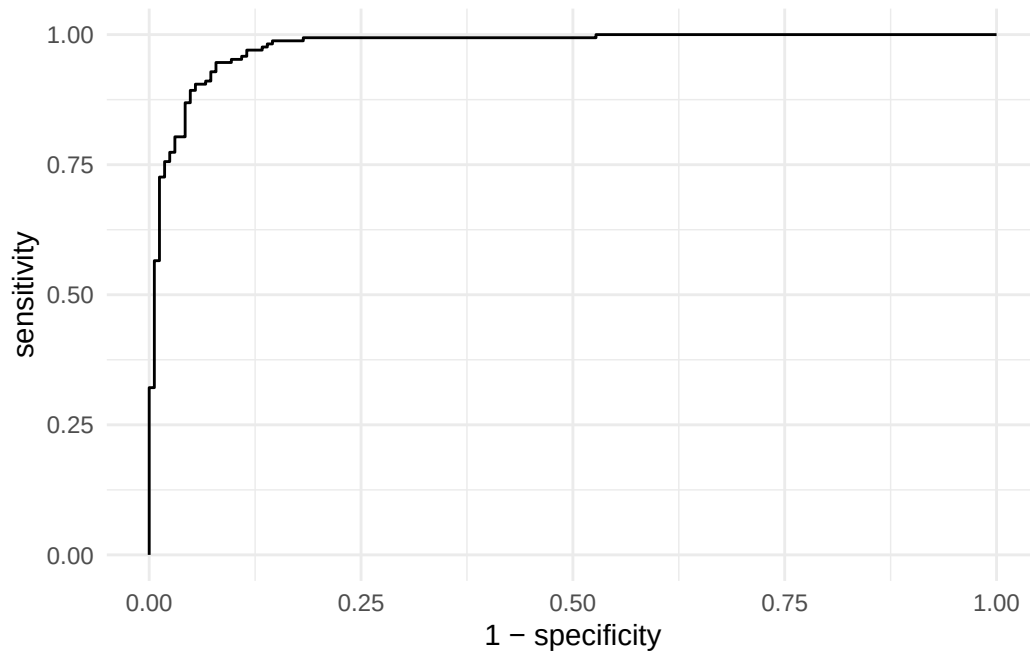
lr_roc %>% glimpse()
```

```
Rows: 335
Columns: 3
$ .threshold <dbl> -Inf, 5.839409e-06, 8.115530e-06, 9.277271e-06, 1.030156e-
~
$ specificity <dbl> 0.000000000, 0.000000000, 0.006060606, 0.012121212, 0.0181~
$ sensitivity <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
```

Nun können wir die Kurve mithilfe von `ggplot` erzeugen:

```
p1 <- lr_roc %>%
  ggplot(aes(x=1-specificity, y=sensitivity))+
  geom_path()+
  theme_minimal()

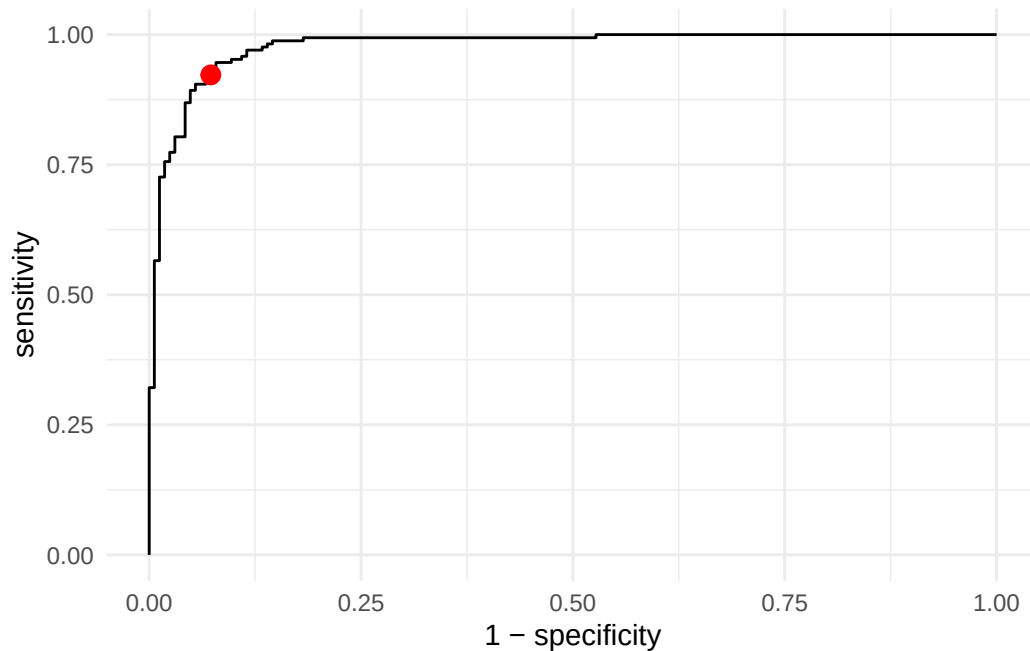
p1
```



Die `geom_path()`-Funktion verbindet die Punkte in der übergebenen Reihenfolge. Wir können diese Grafik nun beliebig erweitern, um z. B. die Sensitivität und Spezifität für den Schwellenwert $q = 0.5$ hervorzuheben.

```
threshold_50 <- lr_roc %>%
  filter(.threshold == median(.threshold))

p1 +
  annotate(
    geom = "point",
    x = 1-threshold_50$specificity,
    y = threshold_50$sensitivity,
    color = "red",
    size = 3
  )
```



Der Schwellenwert $q = 0.5$ maximiert wahrscheinlich nicht gleichzeitig die Sensitivität und Spezifität, weshalb wir diesen optimalen Wert auch berechnen und einzeichnen können. Der Punkt, welcher beide Werte simultan maximiert, kann mithilfe des `lr_roc` Objektes einfach berechnet werden: Indem wir die beiden Spalten `specificity` und `sensitivity` aufsummieren und das Maximum dieser Summe berechnen, ermitteln wir den optimalen Schwellenwert.

```
threshold_opt <- lr_roc %>%
  mutate(sssum = sensitivity+specificity) %>%
  filter(sssum == max(sssum))

threshold_opt
```

```
# A tibble: 1 x 4
  .threshold specificity sensitivity sssum
    <dbl>         <dbl>         <dbl> <dbl>
1     0.442         0.921         0.946  1.87
```

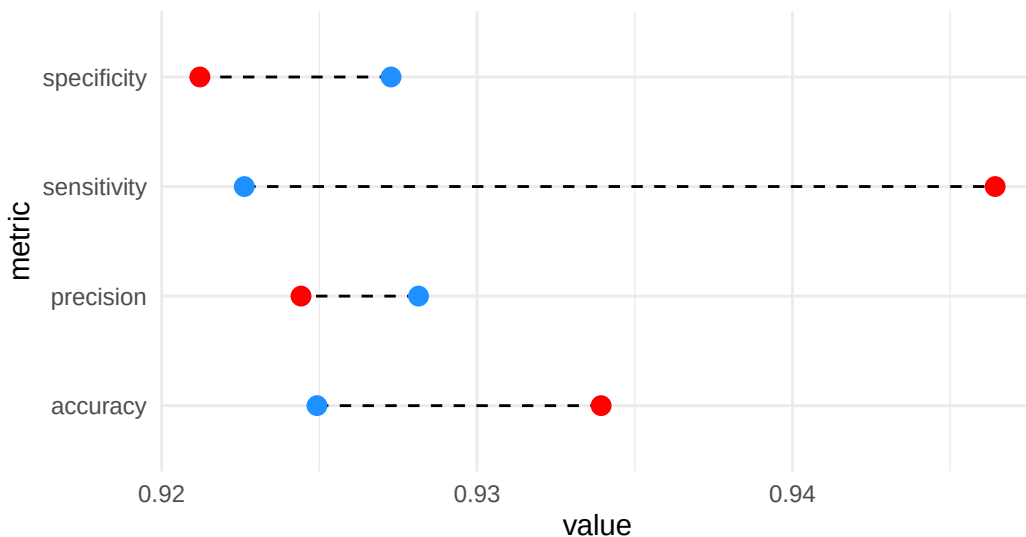
Da der optimale Schwellenwert $q = 0.442$ beträgt, wollen wir unsere Ergebnisse entsprechend anpassen:

```
lr_metrics_new <- log_fit %>%
  augment(data_penguin) %>%
```

```
mutate(
  .pred_class = if_else(.pred_1 >= threshold_opt$.threshold, "1", "0"),
  across(c(.pred_class, sex),
    ~ factor(., levels = c("1", "0"))) %>%
  multi_metric(truth = sex,
    estimate = .pred_class)
```

Wir können dann die Ergebnisse auf Basis dieses neuen Schwellenwerts mit den Ergebnissen auf Basis des alten Schwellenwerts grafisch vergleichen:

Vergleich zwischen einem **logistischen Modell mit $q=0.5$** u
einer **logistischen Regression mit $q=0.44$**
bezüglich verschiedener Metriken



Der Grafik können wir entnehmen, dass durch das Anpassen des Schwellenwerts zwar die Sensitivität stark gestiegen ist, allerdings die Spezifität leicht gesunken ist. Ebenso hat sich die Accuracy verbessert, aber die Präzision leicht verschlechtert.

5.5.2 Klassifikationsbäume

Das Fitten von Klassifikationsbäumen im `{tidymodels}`-Framework funktioniert analog zum Fitten von Regressionsbäumen. Lediglich das Argument `mode` muss auf `"classification"` statt `"regression"` gesetzt werden.

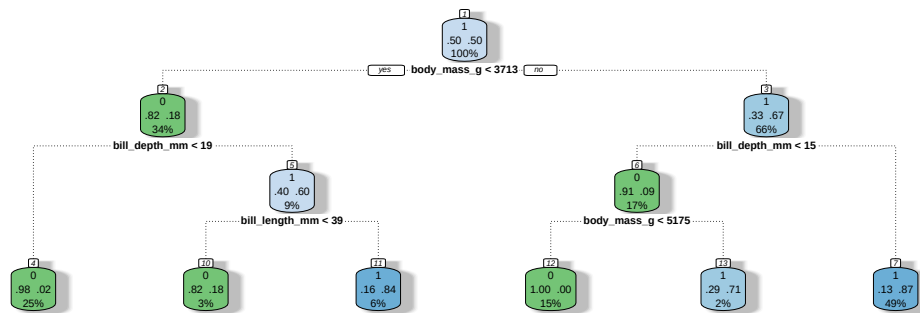
Wir trainieren zuerst einen Baum, ohne die Hyperparameter gesondert zu spezifizieren:

```
tree_spec_vanilla <- decision_tree(mode = "classification")

tree_fit_vanilla <- tree_spec_vanilla %>% fit(
  formula = sex~.,
  data = data_penguin
)
```

Das trainierte Baummodell können wir dann wieder mithilfe der `fancyRpartplot()`-Funktion darstellen:

```
tree_fit_vanilla %>%
  extract_fit_engine() %>%
  rattle::fancyRpartPlot(caption = "")
```



Die Knoten des dargestellten Baums sehen etwas anders aus im Vergleich zu einem Regressionsbaum: Zwar stehen der Vorhersagewert (0 bzw. 1) und die relative Anzahl der Samples im Knoten weiterhin im Knoten, allerdings wird jetzt auch noch zusätzlich die Impurity angezeigt. Die Terme in der Mitte der Knoten geben an, wie groß der Anteil der jeweiligen Klassen in den verschiedenen Knoten sind. So sind zum Beispiel im Knoten mit der ID 4 (linker unterer Blattknoten) 98% der Samples im Knoten aus der Klasse 0 und nur 2% aus der Klasse 1. Je ausgeglichener diese beiden Werte sind, desto weniger rein ist der Knoten.

Um eine noch bessere Performance zu erzielen, sollten die Hyperparameter `min_n`, `tree_depth` und `cost_complexity` angepasst werden. Für den folgenden Baum setzen wir deshalb die Baumtiefe auf 13, die Mindestanzahl der Samples in einem Knoten benötigt für einen weiteren Split auf 3 und den Cost-Complexity-Parameter auf $3.67 \cdot 10^{-7}$.

```
tree_spec <- decision_tree(mode = "classification",
                           tree_depth = 13,
                           min_n = 3,
                           cost_complexity = 3.67e-7)

tree_fit <- tree_spec %>% fit(
  formula = sex~.,
  data = data_penguin
)
```

Für den resultierenden Baum ergeben sich dann die Metriken

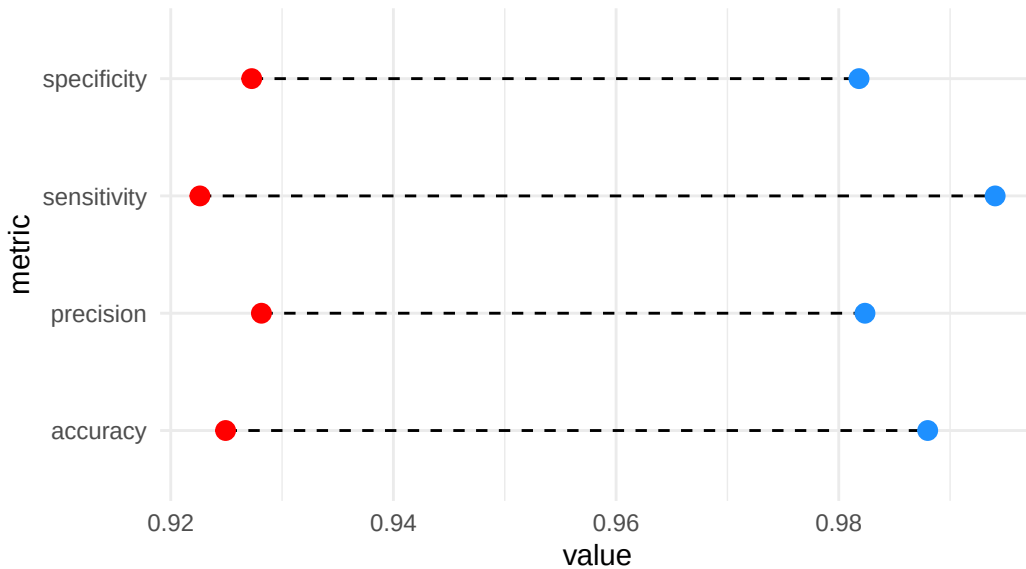
```
tree_metrics <- tree_fit %>%
  augment(data_penguin) %>%
  mutate(across(c(.pred_class, sex), ~ factor(., levels = c("1", "0")))) %>%
  multi_metric(truth = sex,
              estimate = .pred_class)

tree_metrics
```

```
# A tibble: 4 x 3
  .metric      .estimator .estimate
  <chr>        <chr>        <dbl>
1 accuracy    binary          0.988
2 sensitivity binary          0.994
3 specificity  binary          0.982
4 precision   binary          0.982
```

Wir können diese Metriken grafisch mit denen der logistischen Regression vergleichen:

Vergleich zwischen einem **Klassifikationsbaums** und einer **logistischen Regression** bezüglich verschiedener Me



Der Klassifikationsbaum eignet sich also entsprechend der Metriken besser für die Vorhersage des Geschlechts eines Pinguins.

5.6 Übungsaufgaben

Aufgabe 5.1.

1. Im Kontext der logistischen Regression ist die Wahrscheinlichkeit, dass ein Datenpunkt zur Klasse 1 gehört, definiert als

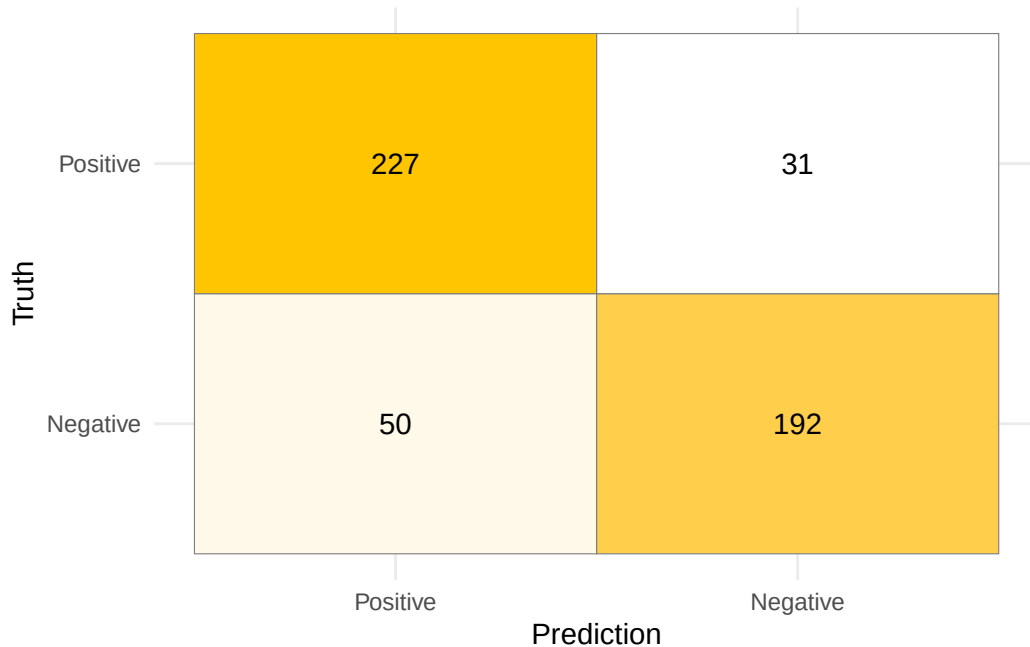
$$\mathbb{P}(Y = 1|X = x) = \frac{1}{1 + \exp\left(-\left(b_0 + \sum_{j=1}^J b_j x_j\right)\right)}$$

Zeige, dass

$$\mathbb{P}(Y = 0|X = x) = \frac{\exp\left(-\left(b_0 + \sum_{j=1}^J b_j x_j\right)\right)}{1 + \exp\left(-\left(b_0 + \sum_{j=1}^J b_j x_j\right)\right)}$$

2. In Abschnitt 5.3.2 haben wir in Gleichung (5.2) die Odds definiert. Zeige, dass der Quotient der Wahrscheinlichkeiten tatsächlich $\exp(b_0 + \sum_{j=1}^J b_j x_j)$ ergibt.

Aufgabe 5.2. Gegeben sei folgende Confusion Matrix:



Berechne für diese Confusion-Matrix die Metriken

- Accuracy
- Sensitivität
- Spezifität
- Precision

Runde die Ergebnisse auf drei Nachkommastellen.

Für die folgenden Aufgaben wollen wir einen Datensatz untersuchen, welcher anonymisierte Patientendaten von 300 Personen enthält. Der Datensatz **Heart Disease Prediction Dataset** kann auf der Plattform [Kaggle](#) heruntergeladen werden.

Der Datensatz enthält folgende Variablen:

Variable	Beschreibung	Ausprägung
age	Alter des Patienten (in Jahren)	Zahl
sex	Geschlecht des Patienten	1 = männlich, 0 = weiblich
cp	Typ der Brustschmerzen	1–4
trestbps	Blutdruck in Ruhe bei Aufnahme ins Krankenhaus (in mm Hg)	Zahl

Variable	Beschreibung	Ausprägung
chol	Serum-Cholesterin in mg/dl	Zahl
fbs	Nüchternblutzucker > 120 mg/dl	1 = ja, 0 = nein
restecg	Ruhe-EKG-Ergebnisse	0–2
thalach	Maximale erreichte Herzfrequenz	Zahl
exang	Belastungsinduzierte Angina	1 = ja, 0 = nein
oldpeak	ST-Streckensenkung durch Belastung im Vergleich zur Ruhe	Zahl

Zusätzlich enthält der Datensatz noch die Variable **target**, welche beschreibt, ob eine Person an einer Herzkrankheit leidet. Die Variable **target** hat die Ausprägungen 1 und 0, wobei 1 für ein positives Testergebnis bzw. Krankheitsbild steht und 0 für ein negatives.²

Aufgabe 5.3.

1. Lese den Datensatz ein und wandle die Zielvariable **target** in eine Faktorvariable um.
2. Finde heraus, wie viele Patienten mit bzw. ohne Herzkrankheit im Datensatz vorhanden sind. Ist der Datensatz balanciert?
3. Sind in dem gegebenen Datensatz mehr Männer oder Frauen von der Herzkrankheit betroffen? Betrachte nicht nur die absolute Anzahl, sondern auch die relative Anzahl in den beiden Geschlechtsgruppen.
4. Erstelle jeweils einen Boxplot für das Alter jener Personen, welche erkrankt bzw. nicht erkrankt sind. Lässt sich ein Zusammenhang zwischen dem Alter der Personen und der Krankheit anhand der Boxplots erkennen?

Aufgabe 5.4. In dieser Übung wollen wir ein logistisches Regressionsmodell schätzen, die geschätzten Parameter untersuchen und verschiedene Metriken auf Test Daten berechnen.

1. Teile den Datensatz in Trainings- und Testdaten ein. Verwende das Seed 123 und die Defaultparameter der **initial_split**-Funktion bezüglich des Anteils der Trainings- und Testdaten.
2.
 - i. Schätze ein logistisches Regressionmodell auf Basis der Trainingsdaten.
 - ii. Erstelle eine Summary des geschätzten Modells und beurteile, welche Variablen statistisch signifikant sind zum Mindestniveau $\alpha = 0.1$.
 - iii. Angenommen die Variable **cp** erhöht sich um eine Einheit, während alle anderen Variablen gleich bleiben. Wie wirkt sich das auf die Odds aus?
3. Nun wollen wir das Modell auf den Testdaten evaluieren.

²Positives Krankheitsbild heißt hier, dass eine Person erkrankt ist.

- i. Erstelle eine Confusion Matrix wie in Abschnitt [Abschnitt 5.2.1](#) auf Basis der Testdaten.
- ii. Verwende das `multi_metric` Objekt aus Abschnitt [5.5.1](#) um eine Übersicht der verschiedenen Metriken zu erstellen.
- iii. Hat das Modell größere Schwierigkeiten Personen als krank zu identifizieren, welche tatsächlich krank sind, oder Personen als nicht-erkrankt zu identifizieren, welche tatsächlich nicht krank sind?

Aufgabe 5.5.

1.
 - i. Trainiere nun einen Klassifikationsbaum mit folgenden Parametern auf den Trainingsdaten.
 - `cost_complexity = 6.43e-4`,
 - `tree_depth = 14`,
 - `min_n = 12`
 - ii. Berechne nun die gleichen Metriken wie in Teilaufgabe 3. ii. aus Aufgabe [5.4](#) auf Basis der Testdaten.
 - iii. Welches der beiden Modelle (logistische Regression vs. Entscheidungsbaum) eignet sich in diesem Kontext besser, um eine Erkrankung vorherzusagen?
2. Da beim Erstellen der Vorhersagen der Schwellenwert $q = 0.5$ verwendet wurde, sollen wir in dieser Teilaufgabe die ROC Kurve untersuchen.
 - i. Erstelle eine ROC-Kurve auf Basis der Testdaten wie in Abschnitt [5.5.1](#).
 - ii. Warum sind in der dargestellten ROC Kurve nur sehr wenige Spezifitäts und Sensitivitäts Kombinationen dargestellt?
 - iii. Bestimme anhand der folgenden Tabelle den optimalen Schwellenwert q (welcher simultan die Spezifität und Sensitivität maximiert) und stelle diesen in der ROC-Kurve der vorherigen Teilaufgabe dar.
 - iv. Ist der optimale Schwellenwert größer oder kleiner als 0.5?

.threshold	specificity	sensitivity
-Inf	0.000	1.000
0.082	0.000	1.000
0.143	0.429	0.951
0.235	0.457	0.951
0.417	0.543	0.927
0.686	0.571	0.854
0.892	0.657	0.610
0.917	0.943	0.024
Inf	1.000	0.000